MSc in Computing - Team Project

Final Report

Group 2 – SafePath

More than Navigation, Smarter, Safer journeys

Team Members

D24126339 – Huda Ibrahim

D24127853 – Hina Kausar

D24126048 – Shalini Kuruguntla

D24125575 – Sai Priyanka Basa Shanker

D24125555 – Karan Joseph

17/12/2025

Table of Contents

Index of Figures

Index of Tables

Introduction

SafePath is a safety-first navigation web application for cyclists and pedestrians that addresses a common limitation in mainstream route planners: they optimize primarily for efficiency (time and distance) while offering limited support for *personal safety* decision-making for Vulnerable Road Users (VRUs). The project was initially developed as SafetyCyclePath and later renamed SafePath to reflect a clearer and more modern identity.

The primary aim of SafePath is to reduce both actual risk and perceived anxiety during urban travel by making safety information visible, comparable, and actionable. The interface is designed to support effective and efficient decision-making in context, consistent with the usability framework defined in ISO 9241-11 (“ISO 9241-11:2018,” 2018). SafePath supports these features through safety-aware routing presets (Safest/Fastest), map-first exploration with clear safety cues, and community hazard reporting, including photo-based evidence.

From a data perspective, SafePath calls the OSRM API to compute routes on a road network that was built from OpenStreetMap (OSM) data, made available under the Open Database License (OSRM - Documentation, n.d.). To model safety risks, the system incorporates publicly available street-level crime data accessed through the police API (“About data.police.uk”). In addition, SafePath integrates TomTom Traffic incident services to obtain real-time hazards and incidents (e.g., accidents, closures, and other disruptions) for more context-aware routing decisions (“Traffic Incidents service | Traffic API”). These sources are translated into map-aligned attributes that contribute to a composite safety scoring approach, enabling users to compare route alternatives using clear indicators and understandable trade-offs.

User evaluation is central to SafePath because a safety-first navigation app must be user friendly and trusted. Usability evaluation and usability testing help identify where the interface supports user goals and where it causes confusion or hesitation (“What is Usability Evaluation,” 2016). Accordingly, SafePath uses mixed evaluation methods (surveys/questionnaires, walkthrough-style sessions, and interviews) to capture both quantitative outcomes and qualitative user reasoning (Lazar et al., 2017).

This report is organized into five sections: Section 1 defines user scenarios and personas (The Characters). Section 2 explains the technical problem and context (The Setting). Section 3 presents the technical solution (The Plot), including system architecture, routing workflow, and a discussion of data bottlenecks and the engineering choices made to mitigate them. Section 4 reports the user evaluation findings (The Results), integrating quantitative measures and qualitative feedback. Section 5 concludes with a review of outcomes, limitations, and future work.

User Scenario: The Characters

Who is Your Target User

SafePath system was designed with a specific goal in mind: to help people who live in cities and deal with safety issues every day to understand it. We used user research, surveys, and persona development to find key categories of vulnerable road users (VRUs) whose experiences contributed to developing the project's vision. The system is aimed at people who live in cities and rely on walking or biking to go around every day, but who have problems with safety, awareness, and infrastructure. Early UX research and surveys found the following key user groups:

User Personas

SafePath primarily serves pedestrians and cyclists in urban areas, particularly those who rely on walking or cycling as a sustainable and cost-effective means of transportation. Personas assist developers and artists understand the types of individuals for whom they are creating content. You don't just consider a general "user." You consider real persons with real requirements. Within this group, we identified five main user categories:

- University students and young commuters frequently walk or cycle to class and come home after dark.
- Working professionals looking for a safer daily commute can cycle or walk to neighbouring offices or transportation hubs.
- International residents and visitors who are unfamiliar with community safety patterns want credible counsel.
- Community people can report problems like poor lighting and hazardous crossings to improve public safety data.
- City planners and local councils use aggregated reports to improve infrastructure and public safety measures.

As part of the Explore phase, we created three unique personas based on study results, user survey data, and problems

Just Eat Courier (bike/e-bike)

Figure 1 - Persona #1 - Daily Cyclist

A daily courier uses SafePath to keep deliveries on time without being pushed into risky shortcuts. SafePath helps them choose safer, well-lit routes, avoid hazard hotspots (potholes, roadworks, dangerous junctions), and reroute quickly when incidents or closures appear reducing crashes, delays, and stress during peak hours. He benefits from safer route options to avoid crash-prone junctions and hazard clusters, plus faster rerouting during closures so that he can stay on time with fewer near-misses, less stress, and less bike damage.

College Student– Emily Carter

Figure 2 - Persona #2 Daily Walker

A college student who walks to and from the Luas relies on SafePath to feel safer on routine commutes, especially early morning or after dark. SafePath prioritises well-lit, safer streets, offers quick reroutes if an area feels unsafe, and supports trip sharing so she can walk with more confidence when alone. She benefits from well-lit, lower-risk walking routes to/from Luas stops, quick "safer alternative" reroutes, and trip-sharing, so she feels more confident commuting alone, especially after dark.

Wheelchair User (city commuter)

Figure 3 - Persona #3 - Wheelchair Commuter

A wheelchair user depends on SafePath to find routes that are truly accessible, not just "short." SafePath highlights step-free paths, smoother surfaces, safer crossings, and warns about barriers like missing curb cuts, blocked ramps, steep slopes, or roadworks, so she can travel independently with fewer surprises and safer alternatives. She benefits from step-free, accessible routing with barrier alerts (blocked ramps, missing curb cuts, steep slopes) and surface/slope awareness, so trips are more reliable, independent, and safe without surprise detours.

Collective Insights

Analysing the three identities, Mark, Emily, and Grace, the most elementary expectations regarding the need for the app are the following:

All users rate the importance of being safe and reliable over speed when it comes to route selection; therefore, the need for a safety-first route guidance system is prominent. The user-friendly design of the application will provide clear and understandable mapping, optimal visual guidance, and minimal confusion while travelling. One recurring user observation was that the app doesn't just improve individual safety, it also creates a stronger sense of community participation, because reporting and sharing safety information feels like actively helping others stay safe.

People want to feel supported, especially when travelling at night or through unfamiliar areas. By enabling users to connect with others who face similar safety concerns, the system turns safety from a solo task into a shared experience—often increasing confidence and providing a sense of satisfaction from contributing to a wider community effort. Research consistently shows that perceived safety drops at night and is strongly influenced by environmental and social factors, which makes companionship and social reassurance particularly valuable in these contexts.

Finaly, everyone requires the system to suggest reliable data regarding route selection. They should feel that the route selection system is intelligent and responding conveniently by considering the real situation regarding the selection of routes.

Why are they important

These users represent the growing number of people who prefer active, environmentally friendly ways of transportation. While walking and cycling promote sustainability and health, many people avoid them due to perceived or actual risks. Every day, we face unsafe crossings, poorly lit streets, high-crime neighbourhoods, and a lack of riding lanes. SafePath's focus on this diverse, yet safety-conscious demographic directly promotes the aims of urban safety, inclusion, and environmental sustainability.

UX Research and Surveys

User survey and data collected by the team revealed that 75 % of respondents preferred safe routes over fast ones, and 73 % were willing to report hazards.

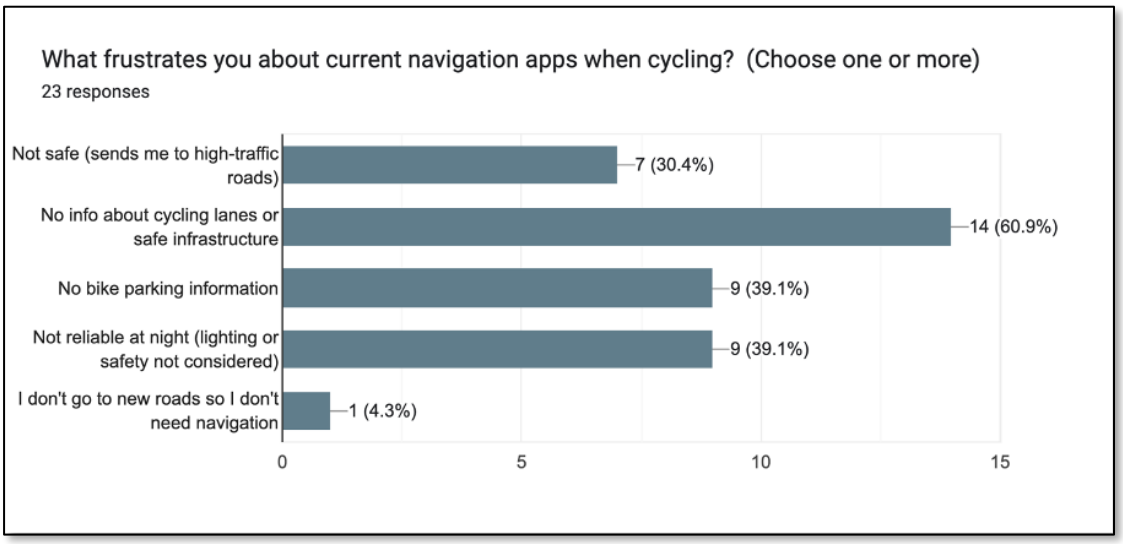

Figure 4 - Frustrations about Current Apps

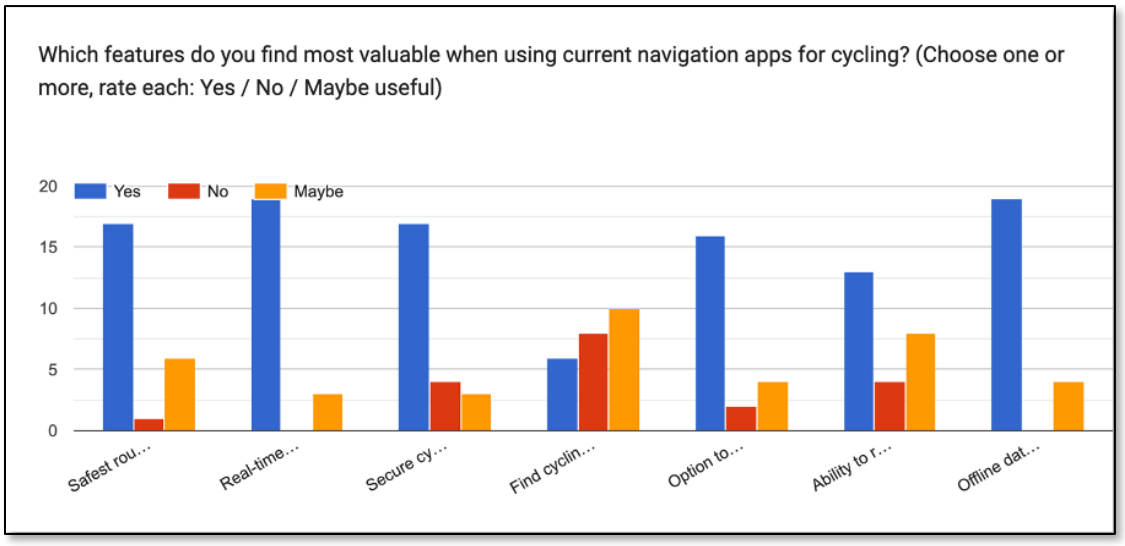

Figure 5 - Valuable Navigation Apps Features

Key Findings

- 91% are beginners or intermediate riders → focus on ease and confidence.
- Top frustrations:
  - Lack of info on cycle lanes/safe infrastructure (61%)
  - Missing bike parking and night safety features (~39%)
- Desired features: real-time hazard alerts, secure parking, and offline data.
- 75% prefer safest routes over fastest.
- 96% would use cycle parking finder.
- 75% open to connecting with others if privacy is protected.

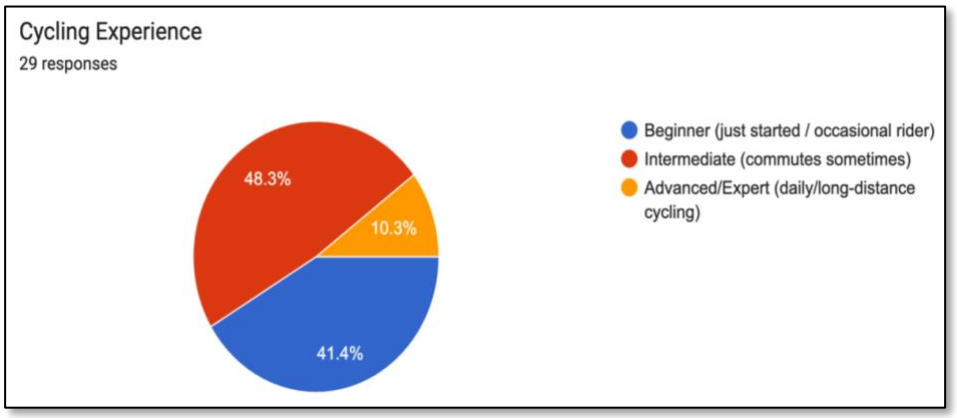

Figure 6- Cycling Experience

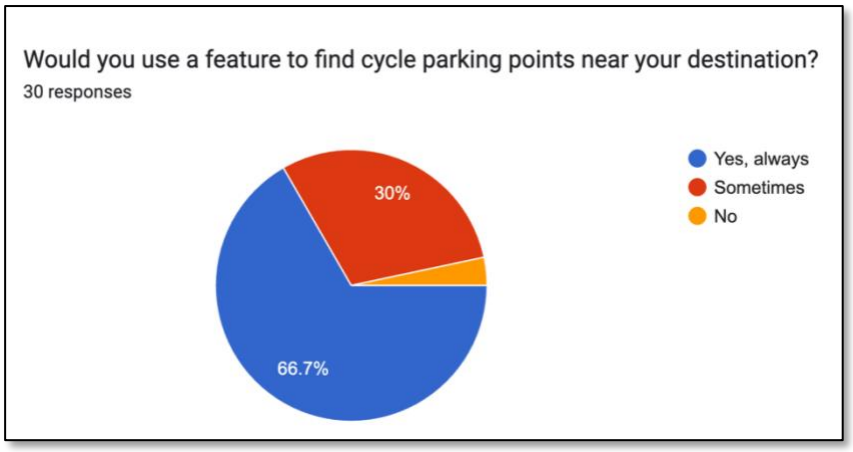

Figure 7 - Pike Park Feature

Pedestrian Insights (48 responses)

- Main Takeaways:
- 61% walk daily; walking is part of routine mobility.
- Top frustrations:
  - No hazard alerts (50%), poor lighting info (45%), and lack of safety awareness (39%).
- Most valuable SafePath features:
  - Safe routes (61%)
  - Real-time safety alerts (61%)
  - Safer crossings (50%)
- 85% prefer safe or context-based routes over fastest.
- 100% open to real-time alerts (with smart control).
- 75% open to connecting with walking groups (if privacy protected).
- 73% willing to report hazards, supporting community-driven safety.

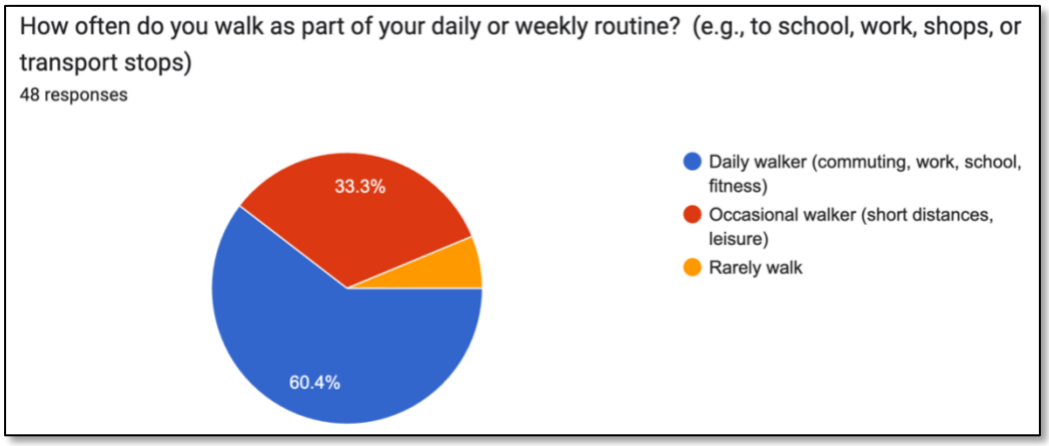

Figure 8- Walking Experience

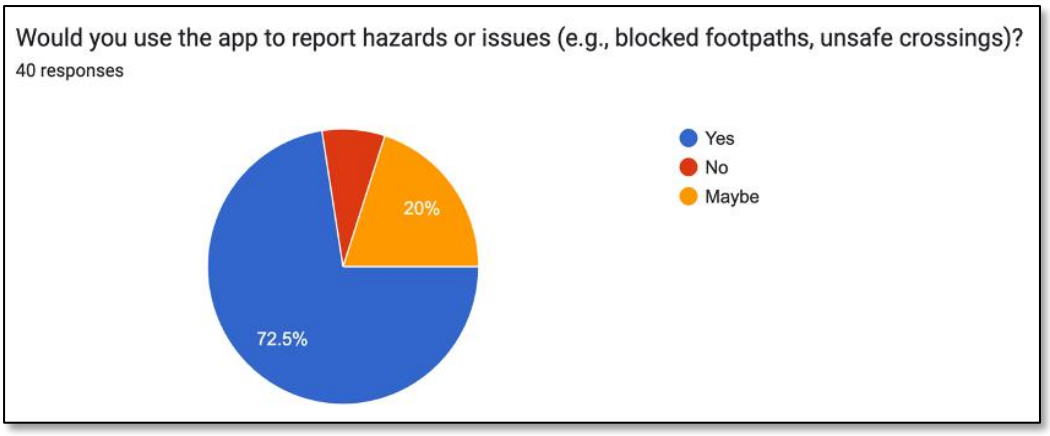

Figure 9 - Reports Hazard Feature

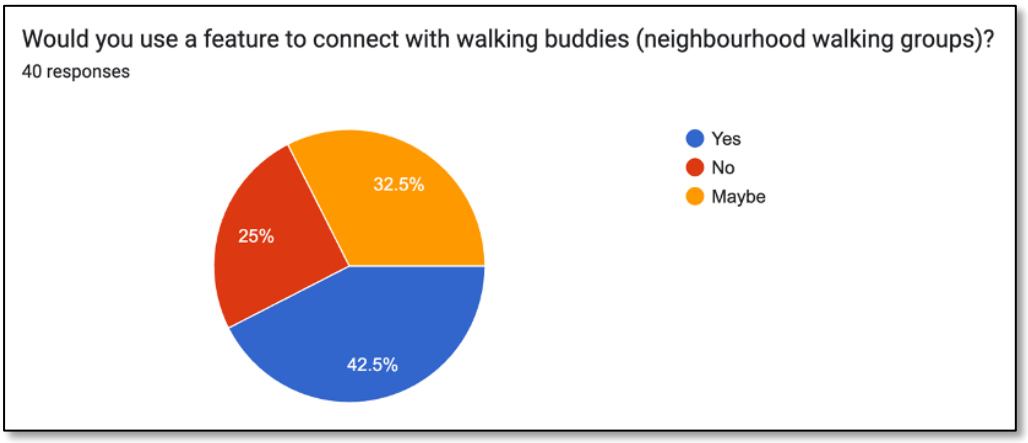

Figure 10 - Find buddy Feature

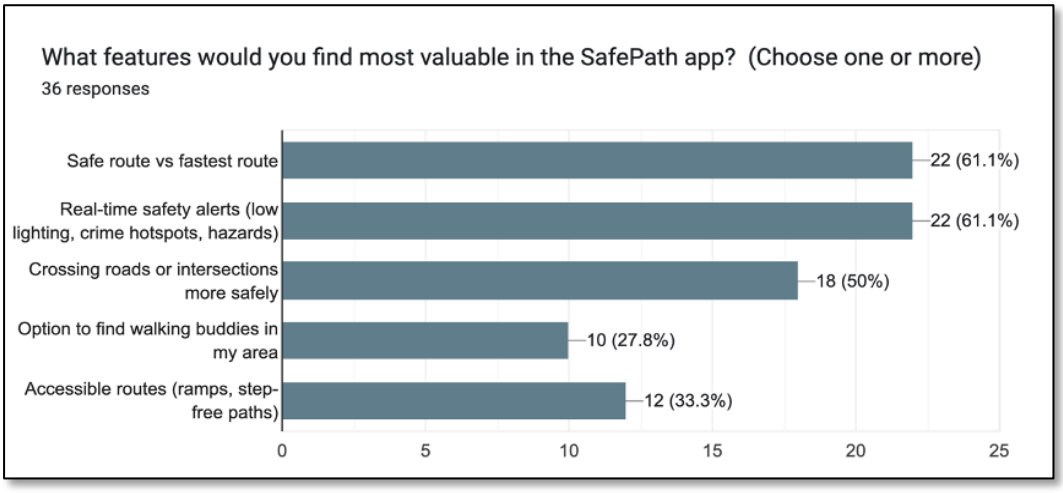

Figure 11 - User Most Valuable Features

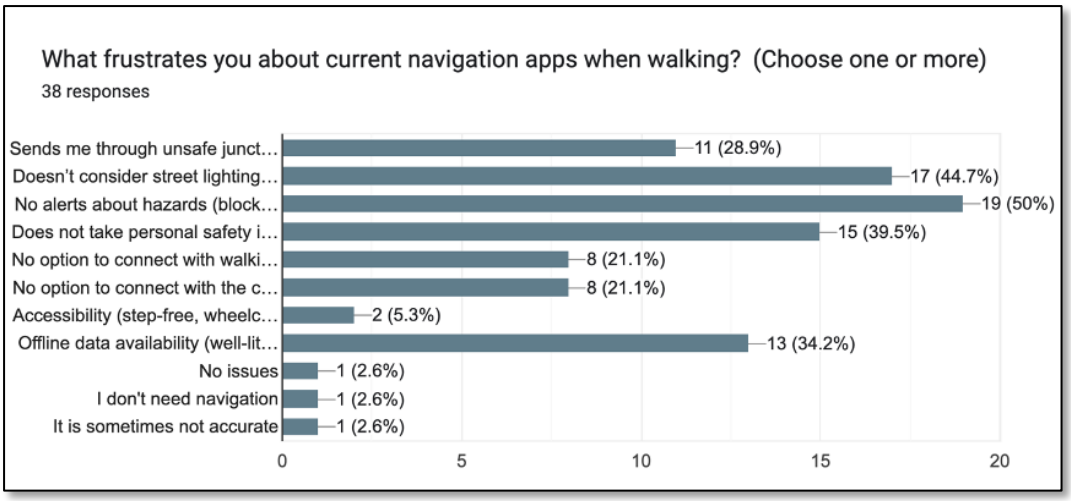

Figure 12 - Current Navigation Apps Experience

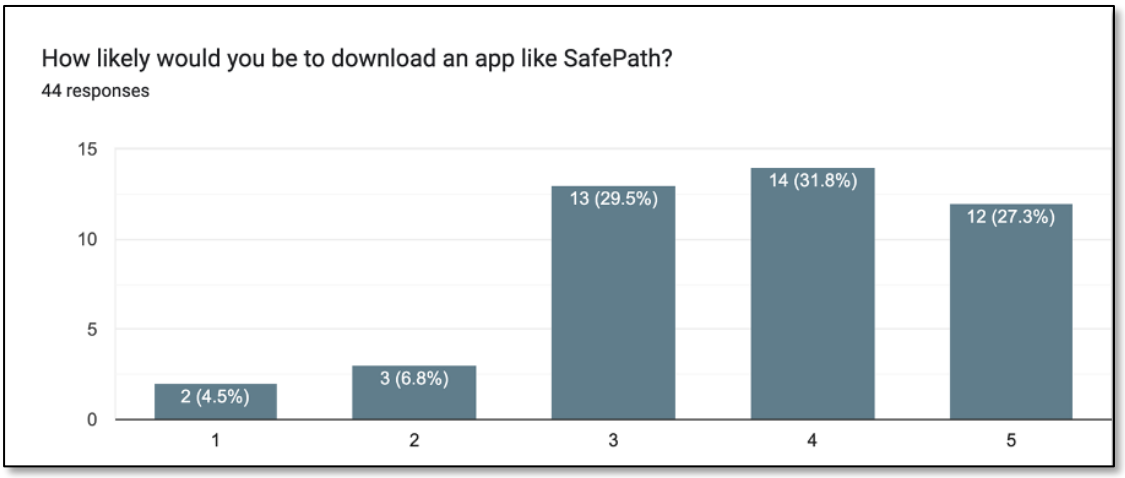

Figure 13 - User Interest in App Download

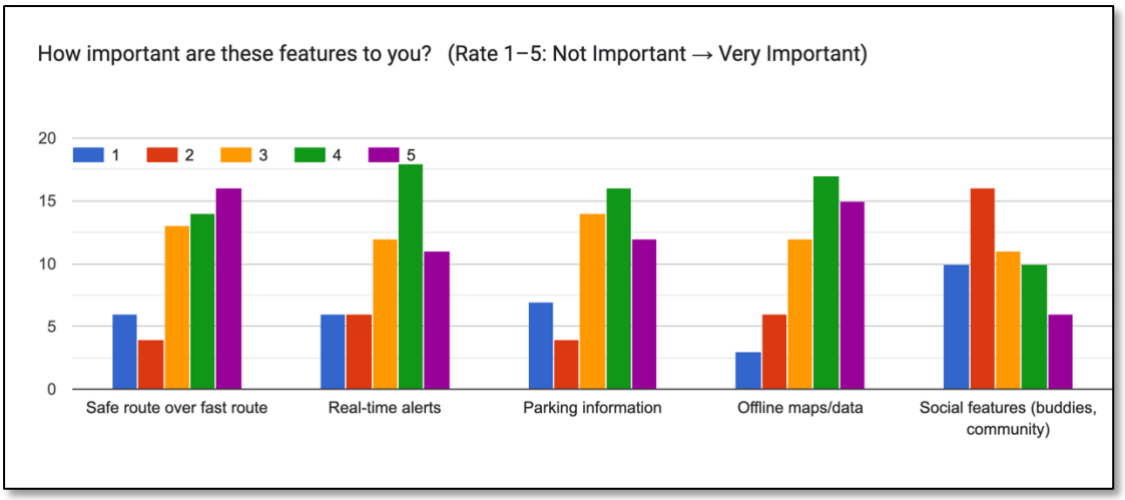

Figure 14 - App Features Importance Diagram

- **Main Takeaways:**
- Preferred alert type**:** non-intrusive — 41% only when app is open, 34% push notifications.
- Most important features: Safe routes, real-time alerts, offline maps.
- 59% likely or very likely to download SafePath; 90% positive or neutral overall.
- Encouraging factors:
    - Free to use (82%)
    - Unique safety features (52%)
    - Trust from local councils or universities (32%)
- Awareness channels:
    - Social media (59%) and friends' recommendations (55%) are key.

These findings confirm strong user motivation for a system that values safety and community participation.

Current navigation apps, such as Google Maps, Waze, and Strava, focus on time and distance rather than safety. They fail to consider real-world facts such as criminal density, lighting quality, and accident frequency. As a result, even the quickest path may expose a pedestrian to hazardous conditions.
SafePath fills this void by aggregating several datasets, crime, infrastructure, lighting, weather, and user reports, and assigning a Safety Index to each route segment. Users can evaluate the fastest and safest routes and choose based on their comfort and context. The system also promotes friendship with a "Find a Mate" feature that matches users with comparable routes and schedules, improving both physical and psychological safety.

What problem are you solving for them

SafePath solves a big problem that exists in today's available navigation solutions: their solutions are focused solely upon reaching a point as quickly and in the shortest distance possible and do not prioritize reaching that point safely. Today's solutions for navigation, such as Google Maps or Waze, are simply concerned with reaching a point in the shortest time possible and covering the shortest distance possible and do not take into consideration whether that route passes through an area that might be dangerous and/or lacks adequate illumination and has a readily apparent crime rate to consider

For pedestrians and cyclists in urban areas, this means that:

- **A safety factor is not considered:** You might be led down through dark alleys or crime-prone areas just because it is faster. Think about being routed through a deserted and dimly lit street at 10pm just because it will cut 3 minutes off your trip.
- **Lack of warnings for hazards:** Until you experience them yourself, you won't know where broken streetlights are, where there are potholes, dangerous crosswalks, or where there have been accidents. A cyclist could ride into a pothole that was reported days ago.
- **The risks associated with cycling are overlooked:** Your apps will not take into consideration, for instance, the aspects of absent cycling lanes, busy junctions where visibility is low, or roads where cyclists are often encountering accidents. The "fastest way to a destination" often leads you straight into traffic with no cycling space at all.
- **Night walking is not secure:** It is impossible to locate well-lit and populated routes for students going back from late classes or employees leaving from the evening shifts.
- **Lack of community support:** Once you notice a danger or an unsafe path, there isn't a way to warn others not to travel the same route.

What the Research Shows

Results from user surveys showed that 75% of users prefer routes that prioritize safety over speed, since they're happy to spend a little more time if it means feeling safer. A total of about 61% of cyclists reported that they're frustrated by "lack of information on cycle lanes and safe infrastructure," while 50% of pedestrians wanted hazard warnings that current solutions fail to provide.

In Ireland alone from 2020 to 2024, 45 cyclists died and 1,278 were seriously injured. Many of these occurred at predictable dangerous spots such as junctions, poorly lit areas, and routes without cycling infrastructure. And these apps still direct users to these areas without warning them.

How SafePath solves this issue

SafePath uses crime statistics, lighting, road condition, real-time danger reports, and community feedback to point out where the routes in your area are safest. Rather than being able to find "fastest route," you're able to use "safest route" functionality, which may include factors like improved lighting, less crime, bike lanes, or avoiding a site where many accidents have been known to occur. The app also enables users to report any hazards directly with their descriptions and images. If a user notices a faulty streetlight or potholes, they can alert the community about it so that everyone knows what to expect or how to avoid it.

In addition, "Find Buddy" matches travellers going in similar directions because it was discovered that many individuals, particularly students and females, felt much safer if they were able to walk or ride a bike accompanied by someone else, particularly at night. SafePath doesn't only save time, it also enables you to arrive safely and with confidence

Market Analysis

SafePath covers a common gap we found in our competitive analysis with the help of this app comparison article(Portus, 2024). It says that most navigation tools, even those made for cycling, only focus on time or distance, with little thought given to personal safety (for example, they only have "quiet" or surface-type filters). This means that people who walk and ride bikes must figure out how to stay safe on their own instead of using evidence-based risk models. This disparity is essential since the environment and infrastructure have a clear effect on how likely it is that bikers and pedestrians may get hurt. For instance, having protected or separated facilities, well-designed streets, and enough light all make people less likely to get hurt and more likely to walk or cycle (Reynolds et al., 2009)(Teschke et al., 2012)(Vidal-Tortosa & Lovelace, 2024).

Our user research supports this reason. The interview guide focusses on the interface's readability, the user's perception of safety, and their expectations for features like "Find Buddy." These are exactly the areas where current services don't do enough for vulnerable road users (VRUs). The survey was given to students, engineers, healthcare workers, teachers, and other people in the community. This shows that there is a lot of interest in safe, everyday cycling beyond just sport riders.

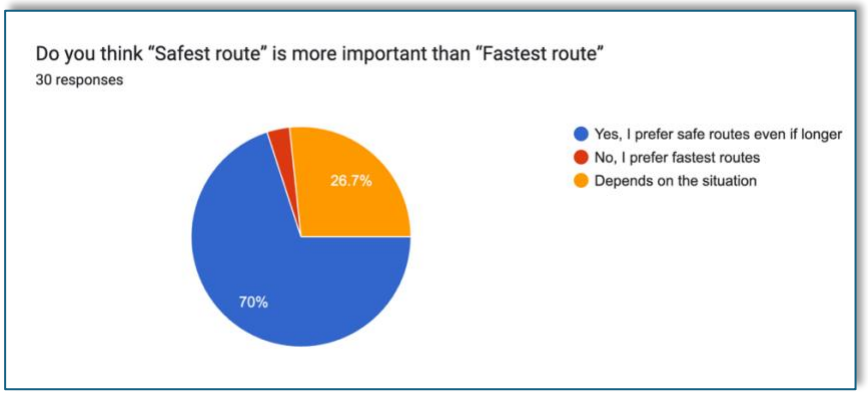

Figure 15- Safest vs Fastest

Research indicates that "near misses" are common and provide insights into risk, yet are never included in official datasets; crowdsourced reporting can address this gap(Nelson et al., 2015)(Branion-Calles et al., 2017). Adding community hazard reports to official feeds is not simply a nice-to-have feature; it is critical for safe routing.

Finally, safety situations change all the time. Patterns in the weather, visibility, and time of day change the level of danger; for example, rain, low visibility, and peaks all raise the risk of a crash, and changes in the weather change the number of cyclists and the chance of an accident(Myhrmann & Mabit, 2023)(Pazdan, 2020). A static "fastest route" model can't deal with these changing exposures; SafePath's purpose is to do so.

Technical Problem

Why Does System Exist

SafePath was developed to fill a clear gap in urban navigation technology: most mainstream routing systems model cities as weighted road-network graphs and optimise routes primarily for efficiency (travel time and distance), which can unintentionally direct pedestrians and cyclists through areas that feel unsafe or carry higher risk exposures (Delling, Goldberg, Pajor, & Werneck, 2017). This need is reinforced by recent UK road-safety statistics: in Great Britain (2024) there were 1,602 fatalities, and "vulnerable road users" accounted for about half of these deaths (Department for Transport, 2025). Research findings indicate that perceptions of safety and the absence or obsolescence of information can influence route selection and the adoption of avoidance behaviors. Consequently, individuals may persist in utilizing suboptimal or hazardous routes, as safer alternatives remain unacknowledged during the decision-making process (Foster, Giles-Corti, & Knuiman, 2014). SafePath addresses this by combining fast baseline routing via OSRM on OSM-derived road networks with safety-relevant context UK Police crime data and TomTom real-time traffic incidents then translating these inputs into map-aligned cues and composite safety scores so users can compare alternatives through clear presets and understandable trade-offs (Sohrabi, Weng, Das, & German Paal, 2022a)

Multiple independent reports point to the same problem SafePath is built to solve cyclist risk is not random, its clusters around predictable conditions that routing can help avoid. Ireland's RSA "Cyclist spotlight" analysis (2020–2024) shows most serious cyclist injuries happen in multi-vehicle collisions (70%), with cars and light goods vehicles most frequently involved, and it highlights common "failure to observe" type factors, exactly the kind of risk pattern that benefits from safer-route choices, hazard warnings, and junction-aware guidance. The RSA also reports a rising trend in single-cyclist collisions leading to hospitalisation (2014–2023),

with Dublin a large share, this is the gap SafePath's community hazard reporting + real-time alerts helps cover, because many "falls/slips/road defects" risks aren't solved by "fastest route" navigation. At a wider level, the EU's ERSO cyclist thematic report notes that roughly one fifth of cyclist fatalities occur at intersections/roundabouts and that cyclist fatalities are proportionally higher at intersections than road deaths overall, supporting SafePath's approach of penalising junction-heavy, higher-conflict routes and recommending safer alternatives, Figure – 20 shows statistical from RSA shows Cyclist Fatalities & Serious Injuries in Ireland (2020–2024) .

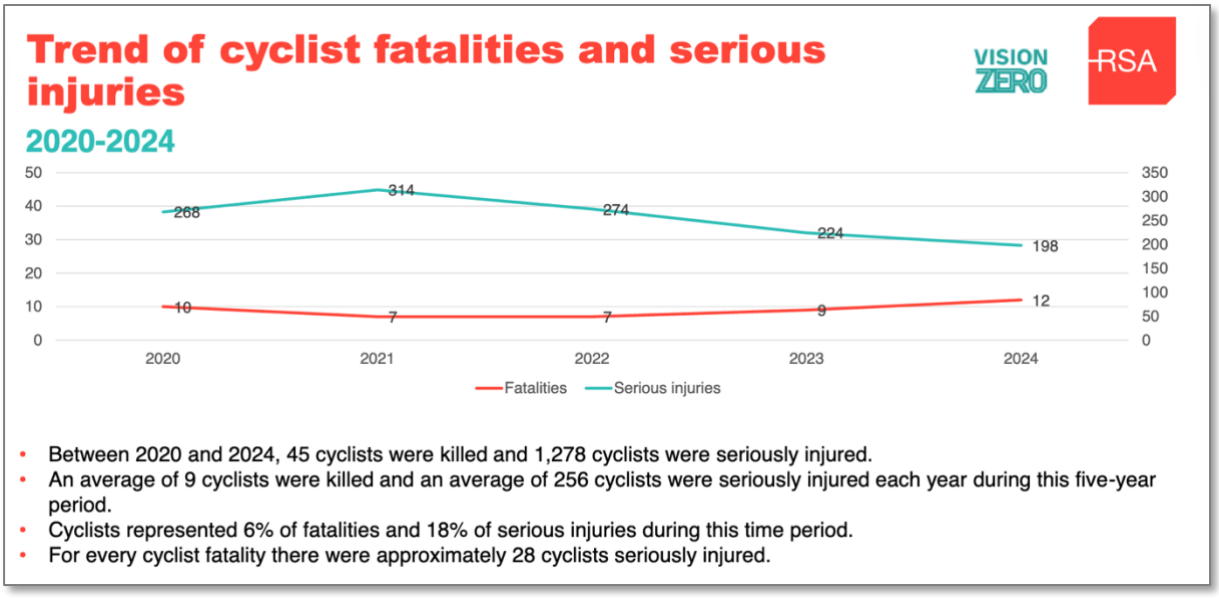

Figure 16 - Cyclist Fatalities & Serious Injuries in Ireland (2020–2024)

The Core Technical Problem

The core technical problem at the heart of SafePath is to generate and present *actionable safety information* for cyclists and pedestrians during route planning, despite safety-relevant data being fragmented, inconsistent, and often not designed for navigation use. Traditional routing engines can efficiently compute routes based on time and distance, but SafePath must additionally collate, normalize, and visualize heterogeneous risk signals, such as public crime reports, real-time traffic incidents, and community-submitted hazards, so that users can compare routes with meaningful safety trade-offs rather than relying on intuition alone.

This necessitates an interconnected pipeline that ingests multiple external data sources and transforms them into map-aligned attributes that can be applied to route geometries returned by OSRM. A major technical challenge is that these datasets differ in spatial precision, temporal update frequency, and format. For example, crime data may be published at an area-based resolution and updated periodically, real-time incidents may be point/segment-based and change minute-by-minute, and user hazards can vary greatly in accuracy depending on GPS quality and reporting context. These differences create both *geographic misalignment* (the same location represented differently across datasets) and time misalignment (static historical risk versus live disruptions), which must be reconciled to avoid misleading safety scores.

A further challenge is producing safety scoring that is fast enough for interactive use. SafePath relies on OSRM to compute baseline routes quickly from OSM-derived road networks, but the safety layer requires post-processing across dense route polylines: intersecting route segments with risk layers, aggregating scores consistently, and generating

understandable indicators without excessive latency. The system must also handle incomplete or noisy data without collapsing into “no result” behaviour; instead, it must degrade gracefully, communicate uncertainty where appropriate, and still provide users with usable route alternatives. Finally, because the output influences real-world decisions, SafePath must ensure that safety cues are not only computationally consistent but also *interpretable*, so users understand why a route is labelled safer and can trust the recommendation.

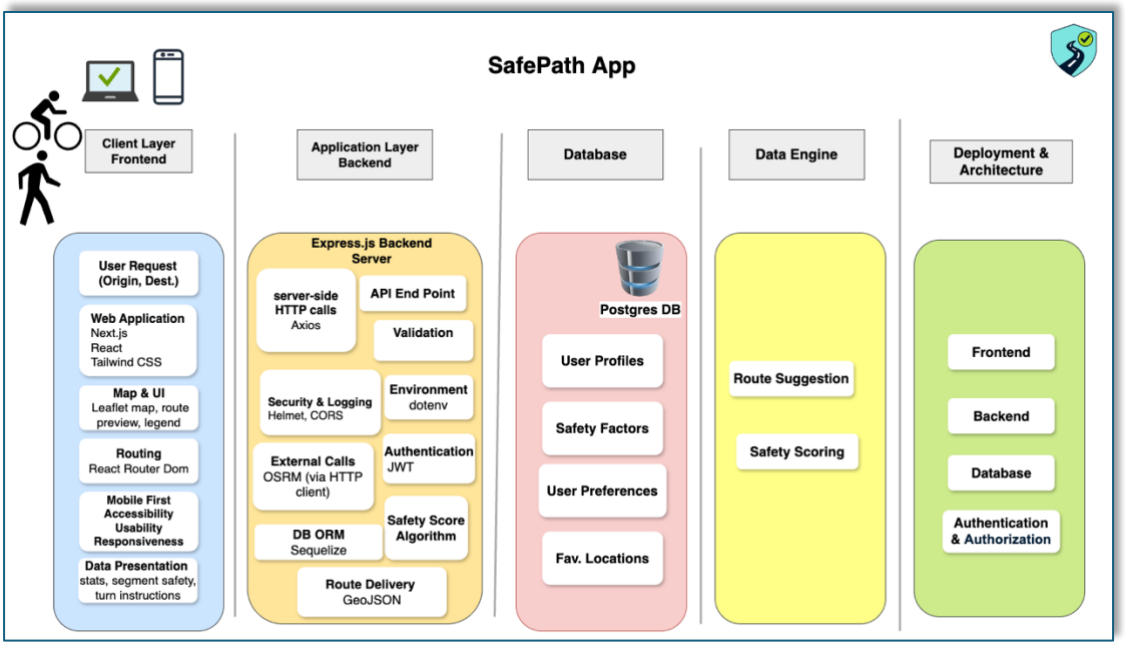

Figure 17 - Technical Component Diagram

SafePath is organized as a layered web system: a client/frontend layer built with Next.js (React) and Tailwind CSS provides the map-first UI (Leaflet), routing screens, accessibility/responsiveness, and safety-focused presentation (route preview, legend, segment safety, and turn-by-turn instructions); an application/backend layer implemented with Express.js exposes REST API endpoints with server-side validation, security middleware (e.g., CORS/Helmet), logging, environment configuration, and JWT-based authentication, and it calls external OSRM HTTP endpoints to compute baseline routes which are then post-processed by SafePath’s safety-score logic before being returned to the client as GeoJSON; a PostgreSQL database persists user profiles, preferences, saved locations, and safety-related records; and a data engine layer encapsulates the route suggestion and safety scoring workflows that connect routing outputs to safety overlays.

We chose this architecture because SafePath needs an interactive, map-first UX *and* a reliable server-side layer that can integrate multiple external data services without exposing keys or complex logic to the client. On the frontend, we used Next.js (React) to structure the application with a production-ready web framework and routing model that supports modern React features and scalable page composition. We paired it with Tailwind CSS to keep UI styling consistent and fast to iterate through utility-first classes, which helps maintain a coherent design system across many screens. For mapping, we used Leaflet because it is an established, lightweight library for mobile-friendly interactive maps matching SafePath’s need to visualize routes, hazards, and safety cues directly on the map.

On the backend, we used Express.js to build a straightforward HTTP API layer because it is intentionally minimal and well-suited for implementing REST endpoints and middleware-based request handling. We also added standard web-security practices such as Helmet for

security-related HTTP headers and controlled cross-origin access via CORS, which is essential when the frontend and backend are deployed separately. Authentication is implemented using JWT because it supports stateless session handling across services and is defined by an open standard, making it interoperable and widely supported (Bradner, 1997).

For routing, we used the OSRM HTTP API because SafePath's routing requirement is get high-quality baseline routes fast, then apply a safety layer on top; OSRM is designed as a high-performance routing engine for road networks derived from OpenStreetMap data and exposes a clean route service over HTTP. For real-time incident context, we integrated TomTom Traffic Incidents services because they are explicitly designed to provide up-to-date traffic incidents/closures via REST APIs. For persistence, we used PostgreSQL because it is a mature relational database suited to storing user accounts, preferences, saved places, and hazard records with consistency guarantees. Routes and overlays are returned using GeoJSON because it is a standardized geospatial interchange format (RFC 7946) that fits naturally with route geometry + properties, making it easy to render and annotate on the client map (Bradner, 1997; "CycleStreets Android app," -a).

We will discuss limitations of these choices (and the most realistic upgrades) explicitly in the Limitations and Future Enhancements sections, rather than mixing them into this rationale

The idea for this app went through multiple iterations before the current format was finalized. Starting from discovering suitable datasets for urban mobility, first for pedestrians, then for cyclists. We iterated on feasibility and coverage and ultimately combined both to serve mixed-mode users. The concept of a safety-first routing application, SafePath combining hazard-aware navigation with an optional buddy feature was chosen for a variety of reasons.

What does your system do

Business Overview

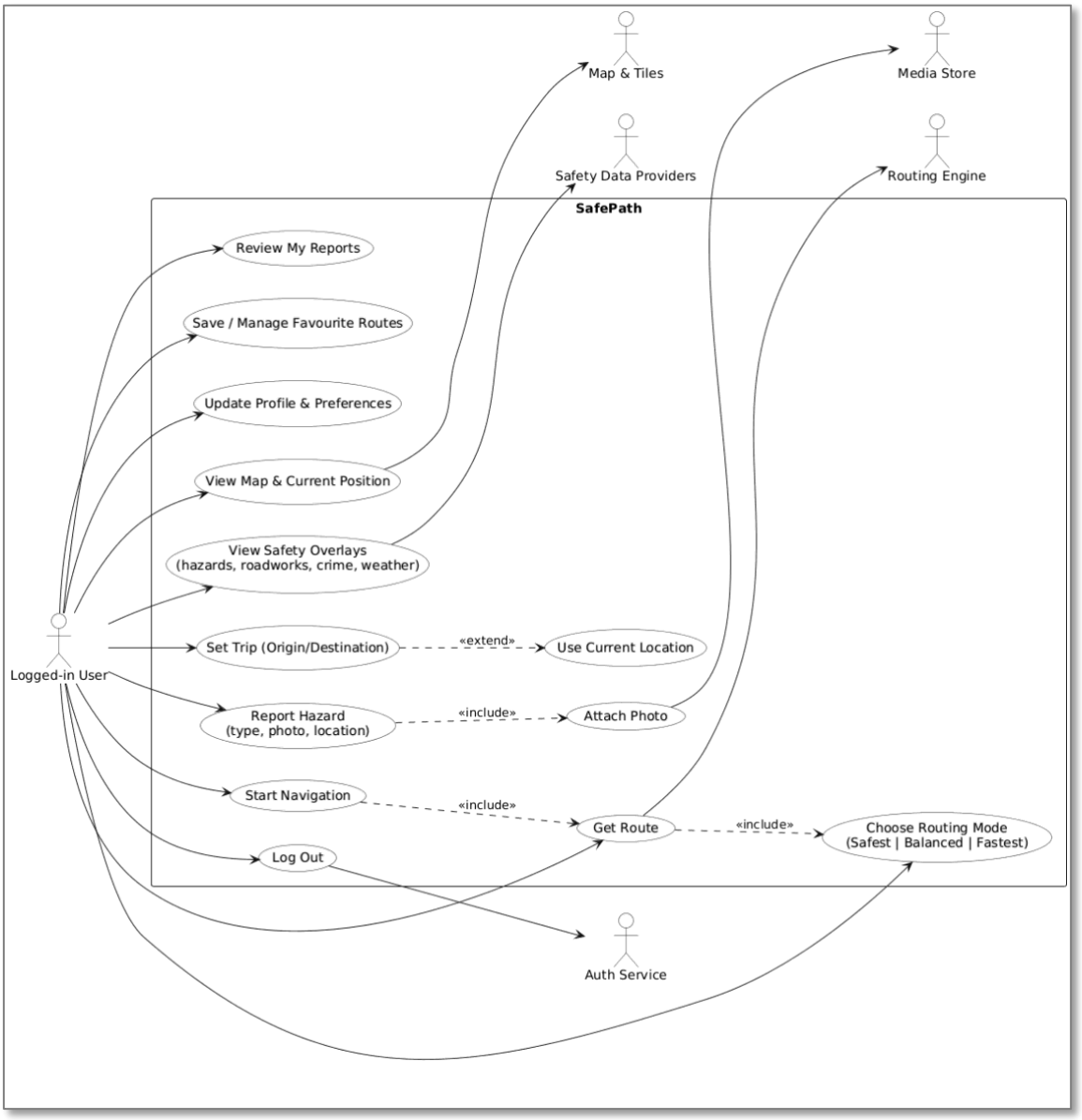

Figure 18 - SafePath Use Cases

SafePath is a navigation platform created specifically for people who walk or cycle in urban environments. Rather than focusing solely on the quickest journey from point A to B, SafePath prioritizes safety. It considers local crime patterns, reported hazards, lighting conditions, collision history and crowd-uploaded alerts, then generates safer alternative routes for the user to choose from. The intention is simple. A journey may take two extra minutes, but if that alternative stays on well-lit streets, avoids areas with frequent incidents and has fewer hazard alerts, it becomes far more reassuring for someone travelling alone or late at night. When a user opens the app, they are met with a clean map interface where they can select their start and end location. SafePath responds by displaying several routing options on the map. One will be the fastest route; another will be the safest. Each route appears in a different color, normally with the safest one in green and the fastest in purple, making visual comparison clear and intuitive. Instead of presenting safety as a mysterious

number, the route card explains why it is safer. For example, it might say that it avoids recent incidents or that the street lighting quality along the path is higher as seen in Figure - 10 . This approach encourages trust, as users can see the reasoning behind each suggestion rather than relying blindly on the system.

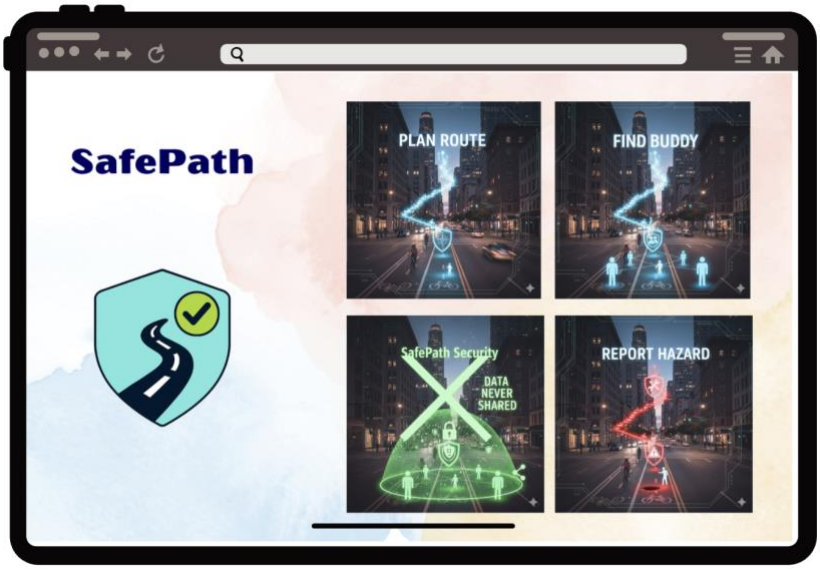

Figure 19 – SafePath App Idea

SafePath also enables users to submit live hazard reports directly on the map. Someone who encounters a pothole, broken streetlight or threatening area can open the hazard form and describe what they see. An optional photo can be attached, and the report becomes visible to other users instantly as seen in Figure -11. This material is integrated with official crime statistics, so the map continues to grow more accurate through community participation. A further feature known as Find Buddy allows travelers heading in the same direction to request walking or cycling together. This was introduced after research participants expressed that they would often feel safer with another person present. SafePath combines routing, live hazard awareness and social reassurance into one cohesive tool.

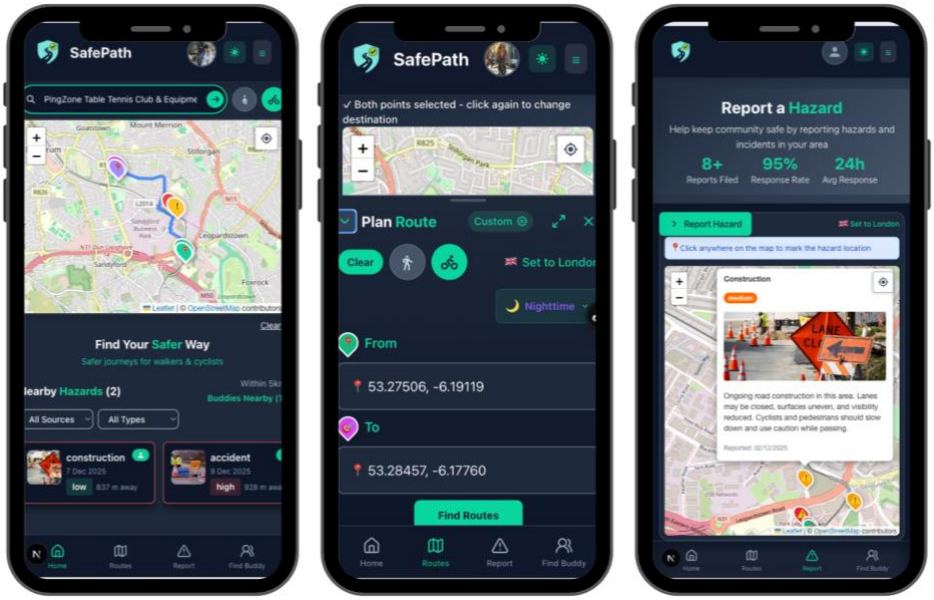

Figure 20 - Home - Routes - Hazard

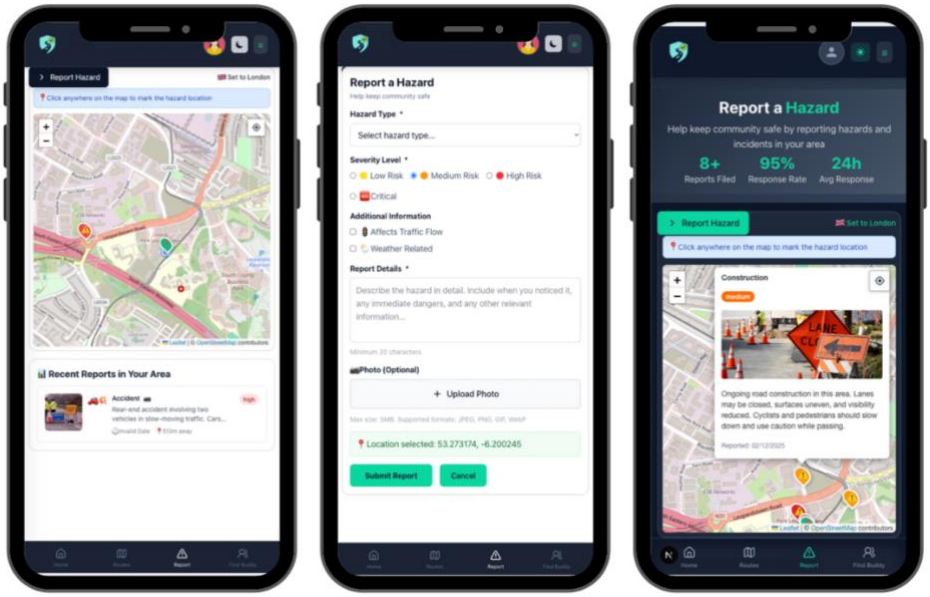

Figure 21 - Report Hazard

The FindBuddy feature is another key added value in SafePath. During our interviews and surveys, many participants specifically asked for a way to walk or cycle with others, and they were very interested in using this feature in the future. To use FindBuddy, the person must be a registered SafePath user and navigate to the FindBuddy page.

Once logged in, users can open FindBuddy from the top left-hand navbar (wide screen view), or from the bottom menu on mobile (and the top-right menu on smaller screens). SafePath then shows how many potential buddies are nearby, by default within a five-kilometre radius. If they want to narrow or expand that area, they can adjust the distance on the buddies card, and the list of nearby users updates automatically.

Each buddy card clearly shows whether that person is walking or cycling, so users can match with people travelling in the same way. Clicking on a buddy opens a small profile card

with the details they chose to share, such as their first name or nickname, their travel mode, and roughly where they are heading. If the user wants to join someone, they can send a request directly from that card; once the request is accepted, they become part of the same buddy group.

From there, SafePath lets them share their route with the group so everyone follows the same safe path together, instead of travelling alone. The idea behind FindBuddy is simple: combine safer routes with safety in numbers, so users feel more confident moving through the city, especially in the evening or at night as seen in Figures – 12 & 13 below.

Figure 22 - Search Buddy and Send Request

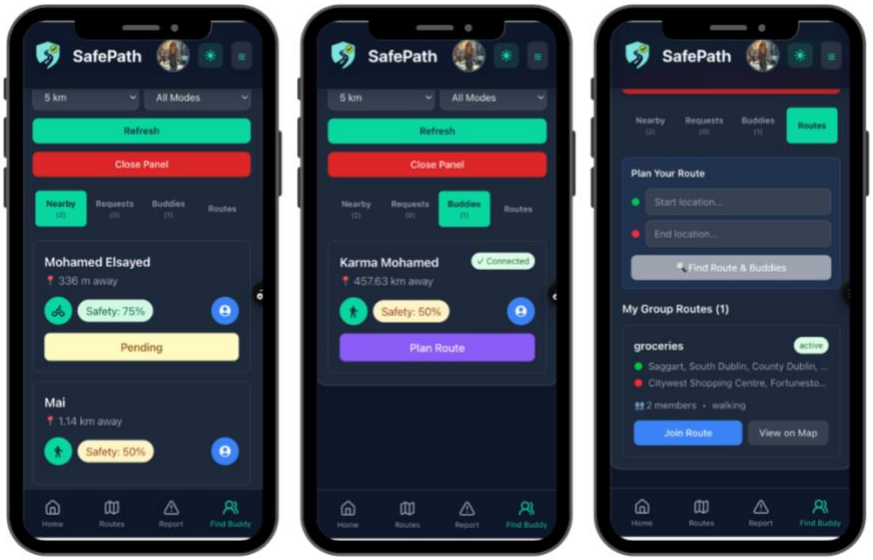

Figure 23 - View Buddy Request and Plan Route

Finally, SafePath treats security and privacy as core product features, not extras. A safety app only delivers value if people can trust it with their accounts and their location data, so we designed the platform around that principle.

For account security, SafePath never stores passwords in plain text. All passwords are hashed securely on the server before being written to the database, so even if the database

were accessed, the original passwords would not be visible. New accounts must be verified via email before full access is granted, which helps prevent fake or throwaway accounts. If a user forgets their password, the reset flow uses a one-time, time-limited link sent to the verified email address, rather than changing anything directly in the browser, reducing the risk of account takeover.

Because SafePath relies on location to deliver safer routes, we apply strict privacy controls. The first time a new user opens the app, they see a Location Privacy Notice and GDPR consent message that clearly explains how their location will be used. For normal usage, location is stored only in the browser's sessionStorage, is not sent to the server for live tracking, and is automatically deleted when the tab is closed. The only time precise location is persisted on the backend is when a user explicitly reports a hazard; in that case it is stored only with that specific report and only with their consent.

Location access is always under user control. SafePath uses the standard browser geolocation API, which means users must explicitly grant permission via the browser prompt, and they can revoke that permission at any time in their settings. This combination of hashed credentials, verified accounts, minimal server-side location storage, and explicit consent aims to balance two goals: giving users the benefits of safety-aware routing and community hazard reporting, while keeping them in full control of their personal data.

After analyzing research findings, developing key user personas representing cyclists/ pedestrians of different experience levels and comparing SafePath's concept with existing navigation apps such as CycleStreets, Waze, and others, we defined a set of core requirements. These outline the essential features and design principles SafePath needs for building an effective, user-focused navigation app.

Functional Requirements

- Users can search for walking or cycling routes.
- App provides fastest route suggestions.
- App provides safest route suggestions.
- Users can find a walking or cycling buddy going the same direction.
- Users can create an account (sign up) and log in.
- App can save user credentials securely for easier access.
- Users can report issues or incidents along routes.
- App can display secure bike parking locations near common destinations (markets, workplaces, etc.).
- Routes show nearby secure bike parking to help users plan safe parking options.
- Users can store preferences and bookmarks (e.g., avoid dark streets, prefer bike lanes).

Non-Functional Requirements

- The user interface must be readable and inclusive, using clear fonts and high contrast colours.
- App must be easy to navigate to help users move safely without confusion.
- User data must be stored securely, in line with GDPR compliance.
- The app must scale to other cities, supporting long-term growth and wider use.
- The system should be reliable and respond quickly to user actions.
- The app should support cross-platform compatibility (iOS and Android).

Req ID	Name of Req	Description	Priority	Who uses it?	Release
R 01	search fastest route	search for fastest route, speed factor only	A	All cyclists/ Pedestrians	Week 5
R02	search route	search for safest route according to safety factors	A	All cyclists/ Pedestrians	Week 5
	view suggested routes	suggest multiple routes (fastest, safest)	A	All cyclists/ Pedestrians	Week 8
R04	see recent hazard	warn user in real time about sudden hazards (roadworks, weather, accidents) so user can adjust route safely.	A	All cyclists/ Pedestrians/ city planners	Week 7
R05	report unsafe areas	wants to report unsafe areas or hazards to help others in the community stay safe.	A	All cyclists/ Pedestrians/ city planners	Week 8
R06	score routes	Add customizable safety factors to the profile page (crime, lighting, crashes, hazards, traffic) so each user can set their own safety preferences for route suggestions.	A	All cyclists/ Pedestrians/ city planners	Week 9
R07	find cycle/walk buddy	find cycle/ walker buddy to share ride with nearby	A	All cyclists/ Pedestrians	Week 9
R08	save user credentials	add login/signup pages	A	All cyclists/ Pedestrians	Week 6
R09	display secure bike parking	show routes with nearby secure bike parking (e.g., near markets, shopping centers, or workplaces) so I can plan where to park safely	C	All cyclists	Futur work
R10	store user preferences	store user preferences, bookmarks (e.g., avoid dark streets, prefer bike lanes)	B	All cyclists/ Pedestrians	Futur work
Non-Functional Requirments:		**secure user data:** want personal data handled securely and in line with GDPR.	A	All cyclists/ Pedestrians	
		scale to to other cities: want the app to scale to other cities so it has long-term impact and relevance.	B	All cyclists/ Pedestrians/ city planners	Futur work
		easy to navigate and inclusive UI: easy app to read and use (with clear fonts, colors, and simple reporting) so that user can navigate safely without confusion.	A	All cyclists/ Pedestrians	

Table 1 - Functional & Non-Functional Requirements

Business Requirements Research Mapping

Req. Description	User Survey Findings	Competitive Analysis Findings	Key Insight / Rationale
Provide safety-first route options alongside fastest routes	√	Fastest mainly	Majority of survey participants (78%) prioritized *safety over speed*; existing apps (Google Maps, Komoot) focus mainly on distance/time
Display visual safety indicators (crime, lighting, hazards) on map	√	-	Users requested visibility of unsafe areas; none of the reviewed apps integrate official crime data or lighting overlays
Enable real-time hazard reporting and alerts	√	√	Survey showed strong interest in community reporting; few competitors offer real-time user alerts (e.g., Waze app)
Allow users to personalize safety preferences (e.g., weighting safety vs speed)	√	-	Over 60% of respondents wanted control over “how safe is safe enough.” No direct equivalent found in competitors.
Simplify navigation UI with clear, uncluttered design	√	√	Feedback indicated early screens were cluttered; Waze and Citymapper demonstrate cleaner home layouts and contextual menus.

Req. Description	User Survey Findings	Competitive Analysis Findings	Key Insight / Rationale
Include community or buddy features for shared safety	√	√ (partial)	Survey participants expressed strong interest in being able to share my journey with a trusted person. While few navigation apps offer full peer-safety modes, some competitors provide limited or related community features, for example: • Google Maps: “Share Trip Progress” allows live location sharing with selected contacts. • Life360 and bSafe focus on personal safety through real-time location tracking and emergency alerts between friends or family. • Waze includes community hazard reporting and road incident sharing. These examples show that SafePath’s “Find Buddy” feature aligns with existing market interest but integrates it more directly into cycling and walking safety contexts.
Generate route safety scores using open data (crime statistics)	-	√ (research prototypes, not mainstream apps)	Academic review identified several prototype systems that compute “safer” routes by overlaying official crime datasets on the street network (e.g., SafeRoute(Levy et al., 2020)). However, major commercial apps (Google Maps, Apple Maps, Waze) do not expose crime-based safety scores, highlighting an opportunity for SafePath to differentiate using transparent, data-driven safety scoring.
Support city planning insights through anonymised data sharing	-	√ (research prototypes, not mainstream apps)	Found in academic prototype, SafeRoute (Levy et al., 2020) but absent in consumer navigation tools
Provide multi-device access (web + mobile)	√	√	Respondents reported switching between devices; all major competitors support both.
Integrate real-time communication channel (WebSocket alerts)	-	√	Competitors with real-time alerts / live sharing - Waze: real-time incident & hazard alerts from drivers and partners; live map updates. - Google Maps: real-time location sharing/trip progress to contacts (live updates while navigating). - Strava (Beacon): live location sharing during activities; updates every 15 seconds. - Life360: continuous, real-time family location sharing and safety alerts.

Table 2 - Business Requirements Research Mapping

Concept Map

Concept maps provide an effective way to display all essential development concepts for websites and applications. The tool enables you to visualize your goals and target audience while showing how different components relate to each other. The design process benefits from this cheat sheet because it helps you stay focused on essential elements. The process of creating the map are not structures, which will help generating innovative ideas which otherwise would not occur leading to improved user experiences for your product.

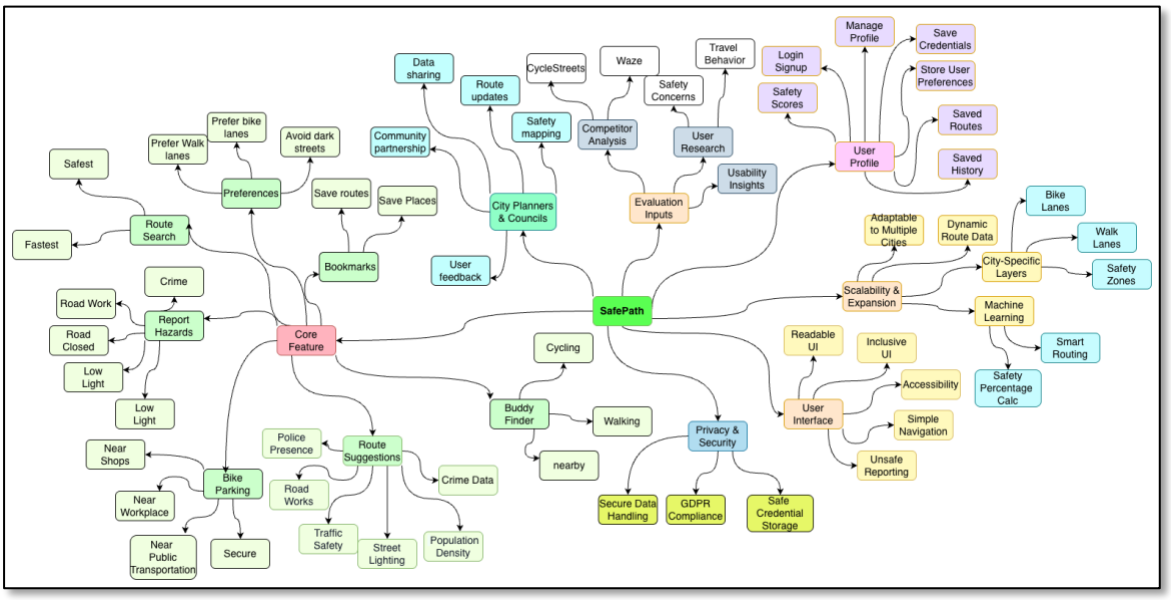

Figure 24 - SafePath Concept Map

Sample of Similar Apps

CycleStreets

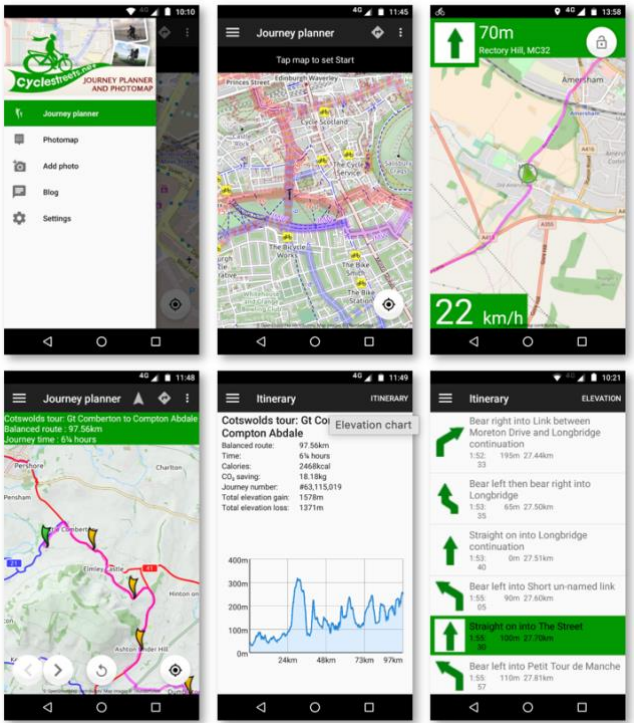

Figure 25 - Mobile App of CycleStreets

Feature	About	Primary Goal	Target Audience	Risk Assessment	Route Prioritization	Safety Awareness Level	Unique Selling Point (USP)	Free vs Paid Models
CycleStreets	UK-wide cycle journey planner for both Android and iOS devices that helps users plan routes for cyclists of all abilities, offering options for the fastest, quietest, or a balanced route	offers personalized, cycle-friendly routes fast, quiet, or balanced, while promoting better cycling infrastructure and community reporting.	includes cyclists of all abilities, from beginners to experienced commuters and campaigners	Examines data errors, bias, privacy issues, and unclear design. Using OSM and a subjective "quietness" score may cause outdated or unsafe routes and varying user perceptions.	Offers **fastest**, **balanced**, and **quietest** routes	Partial , "quietness" metric is subjective, not safety-scored	Cycling-specific routing with fastest, balanced, and quietest route options, built on open, community-edited OpenStreet Map data.	Free (open source), community funded

Table 3 - CycleStreets Comparison

App Features

- Plan cycle routes from A-B
- Add waypoints, e.g. A-B-C-D
- Turn-by-turn directions
- London Cycle Hire (Boris bike) points with live data
- Nearest points of interest
- Search for names, places, towns, postcodes, station codes
- Replan as quietest/fastest/balanced route
- Photomap - view photos
- Report cycling infrastructure problems
- Load an existing CycleStreets route.

All these information and Figure - 4 from (“CycleStreets Android app,” -b).

Waze App

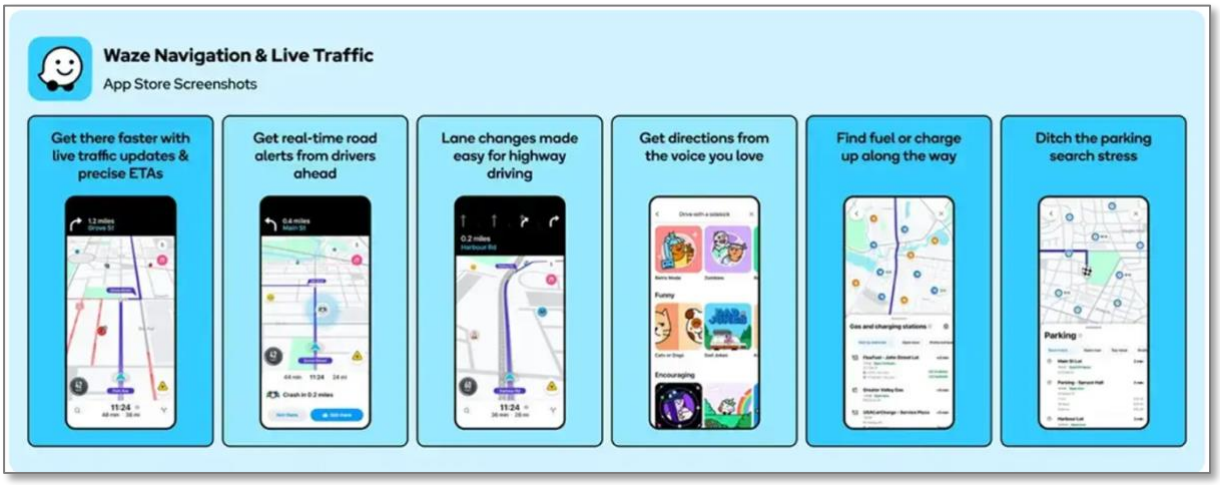

Figure 26 - Waze App

Feature	About	Primary Goal	Target Audience	Risk Assessment	Route Prioritization	Safety Awareness Level	Unique Selling Point (USP)	Free vs Paid Models	UI/UX
Waze	a community-powered, real-time navigation app that provides turn-by-turn directions, traffic updates, and safety alerts like police locations and road hazards. Owned by Google	improves driving by using real-time, crowdsourced data to find the fastest routes, alert drivers to hazards, and save time and fuel.	car drivers, particularly commuters, delivery personnel, and long-distance travelers	Driver-focused Crash History Alerts (uses historical data on crash-prone roads, traffic, elevation). No pedestrian/cyclist specific safety scoring.	Optimizes for shortest driving time, traffic avoidance	None, built for car navigation only	Crowdsourced, real-time driving updates (accidents, police, traffic jams) from millions of active users, focuses on driver collaboration and live alerts	Free, Full driving navigation, live alerts, crowdsourced reports, (ad-based model via map ads)	Bright, icon-heavy interface; engaging but sometimes cluttered.

Table 4 - Waze Comparison

App Features:

- **Crowdsourced traffic reports:** users share jams, hazards, police, and closures to update the map quickly.
- **Real-time rerouting:** routes automatically change when conditions worsen.
- **Turn-by-turn navigation:** voice guidance with text-to-speech street names.
- **Route personalization:** options by vehicle type and preferences (e.g., avoid tolls/HOV).
- **Planned drives:** schedule a trip and get "when to leave" alerts based on traffic.
- **One-tap incident reporting:** simple button to report accidents, hazards, and police.
- **Social map features:** see friends on the map and coordinate arrivals.
- **Community map editing:** users can update road and place details.
- **Lock-screen navigation:** directions and alerts while the phone is locked.
- **Integrations:** connects with music apps and voice assistants.
- **Nearby gas prices:** user-updated fuel prices at stations.
- **Minimal UI + gamification:** simple interface; points/rewards for contributing.

All these information and Figure - 5 from (Wang et al., 2016).

Walk-sharing (Bhowmick et al., 2021) — features and comparison to SafePath

Walk-sharing proposes an algorithm that matches pedestrians travelling in similar directions to support walking together. Key factors include expected waiting time for a partner, detour size when pairing, and the effect on safety and fear of crime. Compared to this, SafePath's FindBuddy follows the same core concept (do not travel alone when a suitable nearby match exists) but extends it into an operational product layer: a live map interface, safety-scored routes, a buddy discovery UI within a configurable radius (e.g., 5–10 km), and profile cards with request/accept flows. In UX terms, SafePath converts a mathematically described approach into a simple user journey: see nearby buddies → view basic information → invite them to share a safer route (Bhowmick et al., 2021).

Coolight (Kiyohara & Mondri, 2025) — features and comparison to SafePath

Coolight is presented as a research MVP/prototype for nighttime student commuting safety rather than a deployed public application. Its demonstrated features include an interactive safety map, real-time incident reporting, live location sharing, and a safety-optimised route planner, all evaluated within a research setting. SafePath aligns with these ideas through safety overlays (crime, lighting, hazards) and community-driven hazard reports, but extends the concept by adding FindBuddy for real-world shared journeys, supporting both walking and cycling, and targeting a practical operational product rather than a study prototype. From a UX perspective, SafePath builds on map-based safety interpretation by adding buddy markers, visible distances, and coordination flows on top of the safety map (Kiyohara & Mondri, 2025).

Pedestrians' perception of safety (Zhang & Bandara, 2024) — findings and how SafePath responds

This work highlights that perceived safety is shaped not only by crime statistics but also by context such as time of day, lighting, area familiarity, and previous experiences, alongside "safe mobility practices" like avoiding certain streets or selecting busier routes. SafePath's design directly reflects these insights by focusing on evening/night scenarios, displaying safety cues (e.g., lighting and hazards) clearly on the map, and offering FindBuddy to reduce the feeling of travelling alone. The work also supports several UX choices: using consistent icons/colours so users can interpret safety quickly, explaining why a route is considered

safer (instead of presenting only a single score), and allowing users to adjust safety–time preferences depending on the situation (Zhang & Bandara, 2024).

bSafe (bSafe, 2025) — features and comparison to SafePath

bSafe is a personal safety platform centred on emergency response and guardian-style monitoring. Its main features include “Follow Me” live tracking by selected contacts with arrival confirmation, voice/touch-activated SOS alerts that share location and can trigger live audio/video streaming and recording, and additional tools such as fake calls and timers. However, it does not provide safety-optimised route planning, avoidance of high-risk streets using crime/lighting/hazard context, or proximity-based matching with nearby travellers. SafePath differs by focusing on prevention and planning: it combines safety-scored routing with FindBuddy to connect nearby walkers/cyclists for shared travel, and the map is designed for route decision-making rather than one-way monitoring (bSafe, 2025).

Life360 (Life360, 2025) — features and comparison to SafePath

Life360 is a family-oriented safety and location-sharing platform built around persistent “circles” for continuous real-time location sharing. It also offers crash detection using phone sensors (with escalation depending on plan), plus features such as driving behaviour summaries, roadside assistance, place alerts, and item-tracker integrations. While Life360 is strong in background safety and long-term reassurance, it does not provide trip-level safety-first route planning based on safety context, does not support ad-hoc walk-sharing/buddy matching with nearby users, and does not focus specifically on walking/cycling micro-risks. SafePath instead is trip-centred: users choose origin/destination, compare safety-scored route options, view safety overlays, and optionally invite a buddy for that specific journey—then stop sharing once the trip ends (Life360, 2025).

Noonlight (Noonlight, 2025) — features and comparison to SafePath

Noonlight is an emergency response service designed for situations where risk is imminent. Its core interaction is a “hold button” panic flow: if the user releases without entering a PIN, Noonlight can contact emergency services using the user’s GPS location. It also provides professional monitoring/dispatch and offers APIs that other apps and IoT devices can integrate for emergency escalation. Noonlight does not calculate or visualise safer routes, does not support everyday map-based safety decision-making, and does not connect nearby travellers. In contrast, SafePath is prevention-oriented: it aims to reduce the likelihood of risky situations by helping users select safer routes, avoid higher-risk segments, and optionally travel with a buddy. UX-wise, Noonlight is intentionally minimal for stress conditions, whereas SafePath must balance a richer map interface (routes, overlays, buddy markers) with clarity so users can make multiple small safety decisions before and during travel (Noonlight, 2025).

This review shows that existing tools address only *parts* of the SafePath problem, but none deliver an integrated, prevention-focused experience for vulnerable road users. Most navigation and cycling apps still optimise primarily for time and distance, with only limited filters such as “quiet” routes, and they do not provide clear, evidence-based safety context that reflects how lighting, infrastructure quality, and environmental conditions affect risk for walkers and cyclists. In addition, official datasets often miss frequent “near misses” and small but important hazards, which makes community reporting a critical complement for safer route decisions. Research prototypes and academic studies confirm the value of safety maps, perceived-safety support, and travelling with others, but they are often not operational products that can be used in everyday commuting. Finally, safety conditions are dynamic changing with time of day, visibility, and weather so a static “fastest route” model cannot reflect real exposure patterns. These combined gaps justify SafePath as a unified system

that brings safety-aware route selection, community hazard reporting, and optional FindBuddy matching into one practical, map-first workflow, helping users make safer and more confident travel decisions instead of relying on guesswork.

Technical Solution: The Plot

System Architecture

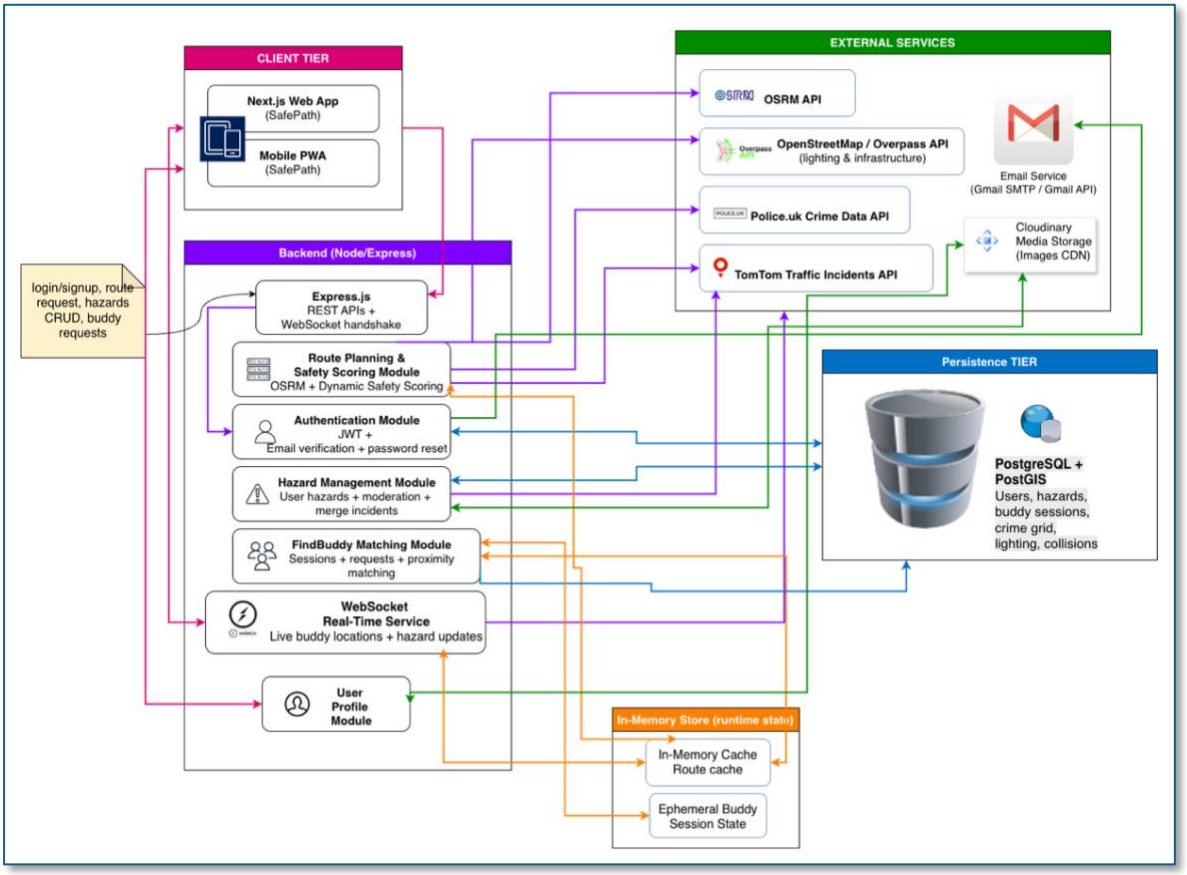

Figure 27 - SafePath System Architecture

- The Client-Tier portion of this diagram will be discussed in the Frontend section.
- The Backend-Tier portion of this diagram will be discussed in the Backend section.
- The External- Services & Database will be discussed in the Data Sources section.

SafePath implementation (MVP / proof of concept)

Right now, SafePath is set up as a single Node.js/Express backend that has REST APIs and a WebSocket (Socket.IO) channel for a Next.js web app and PWA client. The client sends requests for authentication, route planning, danger reporting, and FindBuddy to the backend. The backend handles all external integrations and data. SafePath calls the OSRM Route service to make baseline routes on a road network based on OpenStreetMap, and then it adds a post-processing safety scoring step that combines safety-context signals from the Police.uk Crime API, the TomTom Traffic Incidents API (for real-time disruptions), and—when needed, Overpass queries of OpenStreetMap tags for lighting/infrastructure indicators (“Introduction to PostGIS,” 2023; OSRM API Documentation. (n.d.), 2025; PostGIS Doc, 2023a). PostgreSQL + PostGIS stores long-lasting data like users, dangers, preferences,

and buddy requests/history. An in-memory store keeps short-lived runtime data like route cache and temporary friend session presence to keep interactive latency low. PostGIS is used to help with spatial searches and lookups that need to be fast (such finding dangers nearby) by using spatial indexing .

Only identifiers and URLs are kept in the database for media assets (hazard/profile photos) that are stored in Cloudinary (“Upload,” 2025).

Gmail SMTP sends account verification and password reset emails. This is done by the backend authentication module (Redis, 2025).

Future Enhancements (post-MVP research and scalability upgrades)

We plan to evolve SafePath beyond the single-server MVP by introducing shared infrastructure that supports higher concurrency, reliability, and faster user interactions. First, we will replace the single-instance in-memory store with Redis to centralise route caching and session/presence state**,** ensuring consistent behaviour across restarts and enabling horizontal scaling (Redis, 2025). If we deploy multiple Node/Express instances, we will follow Socket.IO’s recommended scaling approach by enabling sticky sessions and using a compatible adapter (e.g., the Redis adapter) so real-time events are delivered correctly across servers (Socket.io, 2025). We will also improve persistence performance by strengthening PostGIS query efficiency through GiST spatial indexes and index-aware spatial predicates for frequent operations such as “hazards within radius” and route–hazard intersection checks (Google Workspace Admin Help, 2025; PostGIS Doc, 2023a). For routing reliability and predictable latency under load, we aim to self-host OSRM and adopt its recommended MLD pipeline, reducing dependency on public endpoints and ensuring consistent routing profiles (Luxen & Vetter, 2011). Finally, we will refine safety-score accuracy by improving spatial alignment between route geometries and risk layers, applying time-aware weighting that separates historical signals from real-time incidents, and calibrating the scoring model using evaluation feedback, especially to reduce overconfidence where data coverage is sparse. In parallel, we will strengthen privacy and security by minimising retention of precise location data (ephemeral by default), hardening session and token handling in line with OWASP guidance (secure session management and robust JWT validation/expiry), and adopting signed Cloudinary uploads so privileged media credentials never reach the client while maintaining efficient uploads

System Technical Overview

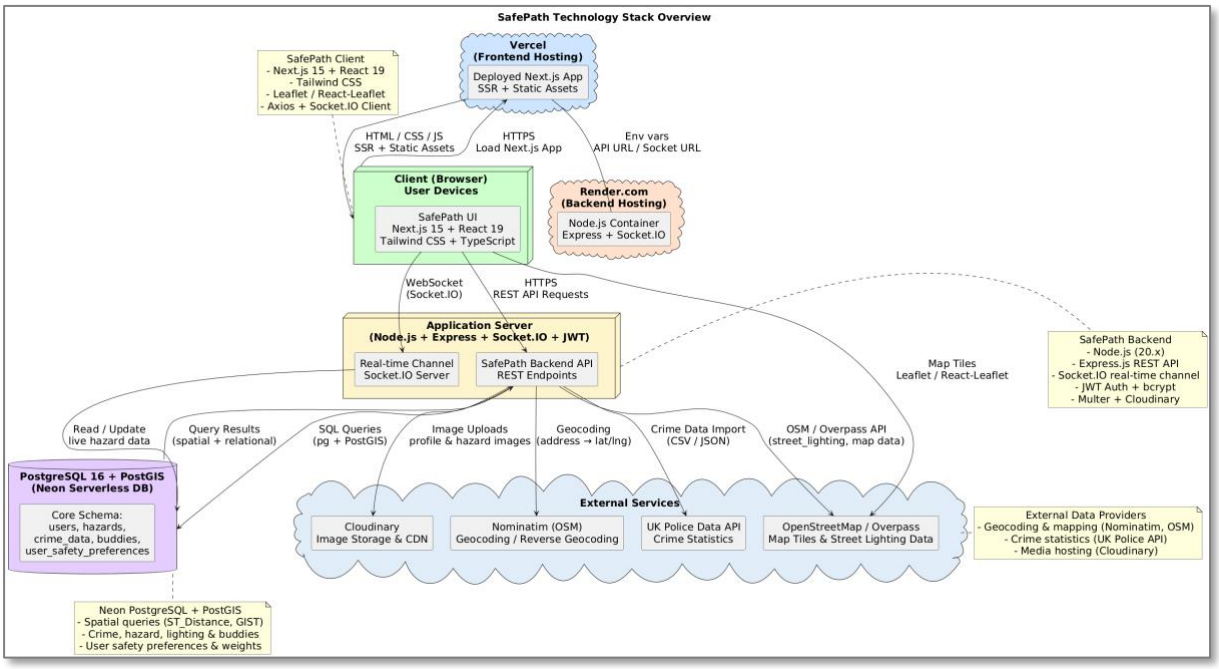

Figure 28 - System Technical Diagram

Although the user experience appears simple, the system processes a significant amount of data each time a route is requested. The architecture consists of a Next.js client application deployed on Vercel, a Node.js and Express backend hosted on Render.com and a PostgreSQL database enhanced with PostGIS for spatial analysis. The front-end communicates with the backend using HTTPS for regular requests, while WebSocket's through Socket.IO manage real-time updates such as hazard alerts. When a user asks for directions, the front-end sends the selected coordinates and travel mode to the backend. The server requests a preliminary route from the OSRM routing engine, which provides baseline geometry optimized for time efficiency. SafePath then evaluates each route step by step. Everyone hundred meters approximately, a coordinate sample is generated, and the system checks the corresponding geographical cell inside our spatial grid. Each grid cell holds pre-processed information including crime density, lighting quality, road characteristics, verified hazards and historical collision statistics. These values combine into a weighted Safety Score. The weighting can differ depending on whether users are cycling or walking and whether they prefer higher caution or faster movement. After calculating the overall safety value for all route options, SafePath ranks them and returns them to the interface. The safest and the fastest route become visible as coloured polylines. A user can tap one to view estimated travel time, distance and explanation for why the score is high or low. The interface updates without reloading thanks to client-side React state management. Hazard reporting flows in the opposite direction. A report sent from the app sends the image to Cloudinary and the data to the backend. Once validated, the record is stored in the database. Immediately after storing, a real-time event is triggered through Socket.IO which displays the new hazard to users nearby. This creates an environment where information spreads rapidly rather than relying on periodic refreshes. The entire hazard cycle is captured in the system diagram which outlines how user submissions propagate through the backend and return as new alerts. In early design plans, SafePath was structured as a microservices system with Redis caching and separate routing modules. Later, after feedback during interim stages, the architecture was simplified into a modular backend that maintained internal separation but ran within a single deployable service.

This proved easier to maintain, debug and scale within the time frame of the MSc project while preserving the future possibility to expand into microservices later if required.

Frontend

Front-End Architecture and Technology Stack

We chose React, Next.js, and Tailwind CSS because they are well-suited to working with the map application in SafePath. The main screens in SafePath rely on a coordinated state of the user interface, which involves selecting a start location and an end location, route options, safety filters, and threat and buddy overlay switches. Such a programming approach can be very well paired with React's components and declarative rendering because they allow you to update the interface based on state ("Quick Start – React"). They not only make it simpler to update the interface but make it simpler to see which functions have added new capabilities when they change. Such an approach to developing makes it simpler to see which functions have added new capabilities when they change because in this case everything is based on state, such as in React. Through such a way of developing, it is simpler to see which functions have added new capabilities when they change because everything is based on state, such as in React. Therefore, this approach to developing will make it simpler to see which functions have added new capabilities when they change because everything will be based on state, such as in React. Furthermore, such a way of developing is suitable for creating a mapping interface in a React environment because React Focuses on front-end performance and interface updates because React makes front-end programming simpler, faster (React Leaflet n.d.).

A major factor in our choice of Next.js is to construct a web application which is functional and gives a better service than an existing application in use. Next.js can handle basic routing and layouts and can render in a variety of manners such as Server-side rendering and Static site generation. Such methods can aid in faster loading times and efficient responsive navigation paths. For a safety application, this is a critical function because a slower application will make consumers less trusting of and less likely to employ it in their everyday lives. Next.js can work in coordination with existing methods used to deploy apps such as setting up environments and creating preview builds. Such methods allow programmers a chance to make modifications and test them prior to completing work on them (Next.js n.d.).

As a reason to pick Tailwind CSS in this project, it is a tool with which one can easily incorporate graphic elements quickly and easily. Moreover, in map overlying, making it easy to use is where the legibility, distance, and contrast of elements come into play. As a matter of fact, in creating map overlays, using a utility-first approach, Tailwind CSS ensures all elements share common space, font, colour tokens, and responsive breakpoints. Furthermore, this is ideal for mobile-first ergonomics since maps can be better accessed using the thumb. Moreover, you can easily customize the final navy and green colour schemes used in this project to show different sorts of routes and dangerous situations (Yu, 2022).

User Experience Design Decisions

SafePath implements an interaction style based on a map-first interaction model, which puts the map front and center for both web and mobile interaction. To achieve an interactive map layer, React-Leaflet is used, which is a library providing a React interface for Leaflet. With this, basic map functionality such as rendering a base tile layer, creating a route polyline, or showing an interactive marker or overlay can be achieved (React Leaflet n.d.). Users input

origin and destination either via searching or by granting geographic location access on devices. After which, when they are submitted, origin/destination suggestions appear in a scrollable card format, where clicking on a card shows a specific path highlighted on a map, thus making decisions based on geographical context rather than text.

The mapping experience underwent a major overhaul based on learning more about the actual use context of the app. The early designs were very Desktop Computer-oriented and tended to have a static home page with lots of function buttons presented all at once. Feedback from both end-users and expert reviews suggested this approach tended to be very busy and not very efficient for a safety app used when on-the-move. Therefore, this led to an overhaul to a mobile-first focus where speed and progressive disclosure were prioritized. The latest design for the home page differentiates functions for guests and authenticated users: guests can immediately view dangers, see dangers in the area within a given radius (5 km, for example), and view a summary of travel buddies in an area; authenticated users can display routing right from the home page and access information on travel buddies in detail. Several other modifications were carried out to improve interaction with the home page: a hazard card display so a user can pick a hazard from a list and go right to where it is and what it is all about on a hazard map.

The routing interface was also improved in this way. Early Figma prototypes were built and tested with a medium-fidelity prototype, where users pointed out usability flaws and a need for better organization and quicker access to important controls. As a function of this, SafePath optimized map front and centre presence by incorporating a side panel design pattern for organizing route selection, safety preference entry, and auxiliary controls without hiding important map information. Such an organization took place on a variety of pages, with further work being considered for improving navigation through a persistent map page with side panel tabs.

Visual Design, Accessibility, and Mobile Ergonomics

SafePath's visual design evolved through iterative testing rather than being fixed from the start. In the early prototypes, we used a light, safety-themed palette: an off-white base with green to represent safe choices and amber to differentiate hazards and warnings. Multiple high-fidelity screens were produced in Figma using this theme, because it matched the app's safety purpose and looked clean in isolation. However, once we presented these designs to users and mentors/experts, the feedback was consistent: the palette felt too flat and not modern enough, and some UI elements did not stand out strongly when placed on top of detailed map tiles. Participants also highlighted that the experience needed to be mobile-first, since SafePath is designed for use while moving, where glanceability and thumb-reach matter more than desktop density.

In response, we ran a focused round of design research and experimentation, using a combination of peer and expert input, quick competitive scanning, and AI-assisted suggestions for accessible contrast and map-overlay readability. This led to the final direction: a darker navy + green palette that improved contrast against map backgrounds and gave the interface a more modern, confident look. Within this system, we deliberately established a "visual language" so users can understand meaning quickly: we distinguish safe vs. fast routes using consistent, high-contrast route styling, and we keep hazard severity visually separate from route quality. We also differentiate cycling vs. walking through consistent iconography and UI accents so the mode is always clear without requiring extra reading.

Accessibility and mobile ergonomics were treated as first-class design constraints. Controls were made larger and placed in thumb-reachable zones to support one-handed use, spacing was increased to reduce mis-taps, and key map interactions were kept minimal to avoid cognitive overload. The result is a clearer, more legible interface especially outdoors while still staying aligned with the app's safety-first goal.

Below are selected UI screenshots showing SafePath's design evolution from early to current iterations. Older versions are highlighted with a light-blue frame to distinguish them from the latest UI.

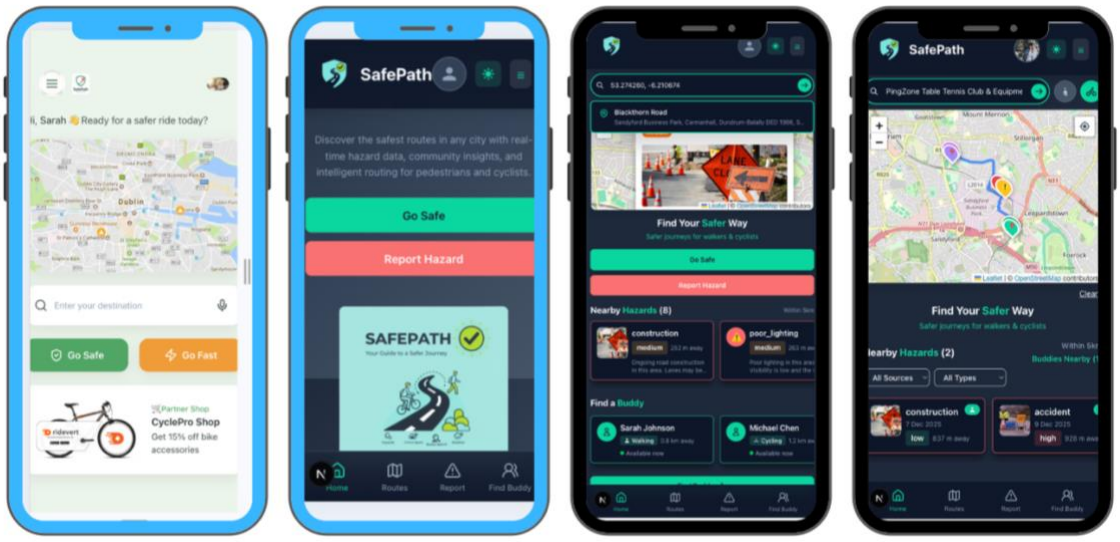

Figure 29 - Home Page Design History

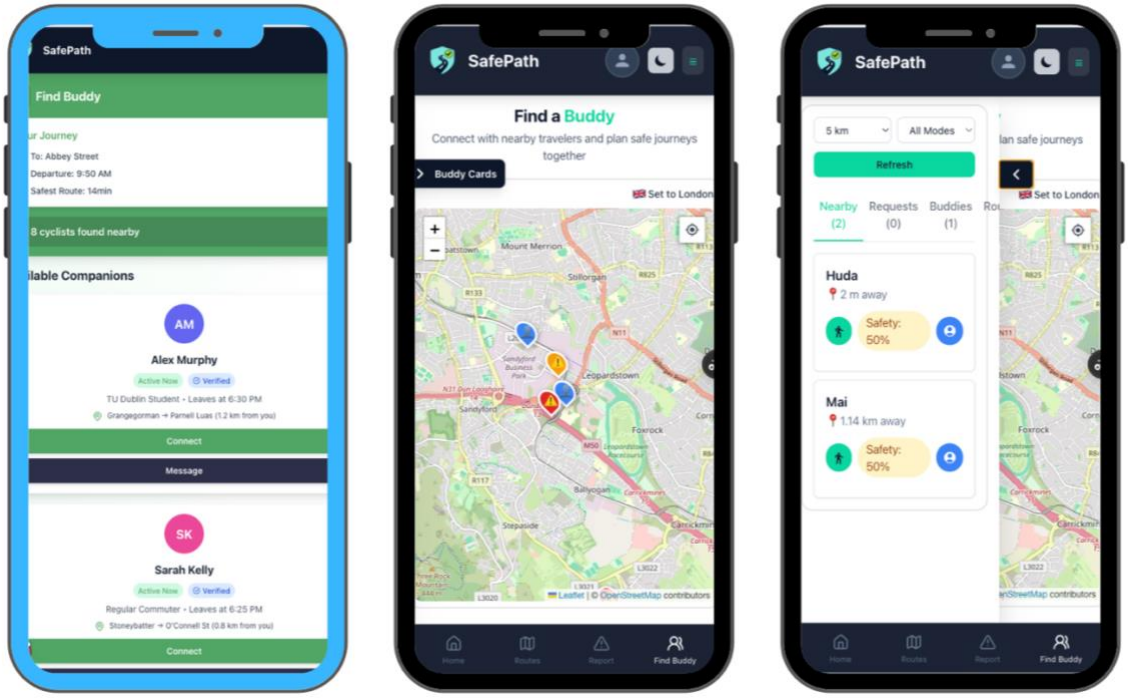

Figure 30 - Findbuddy Design History

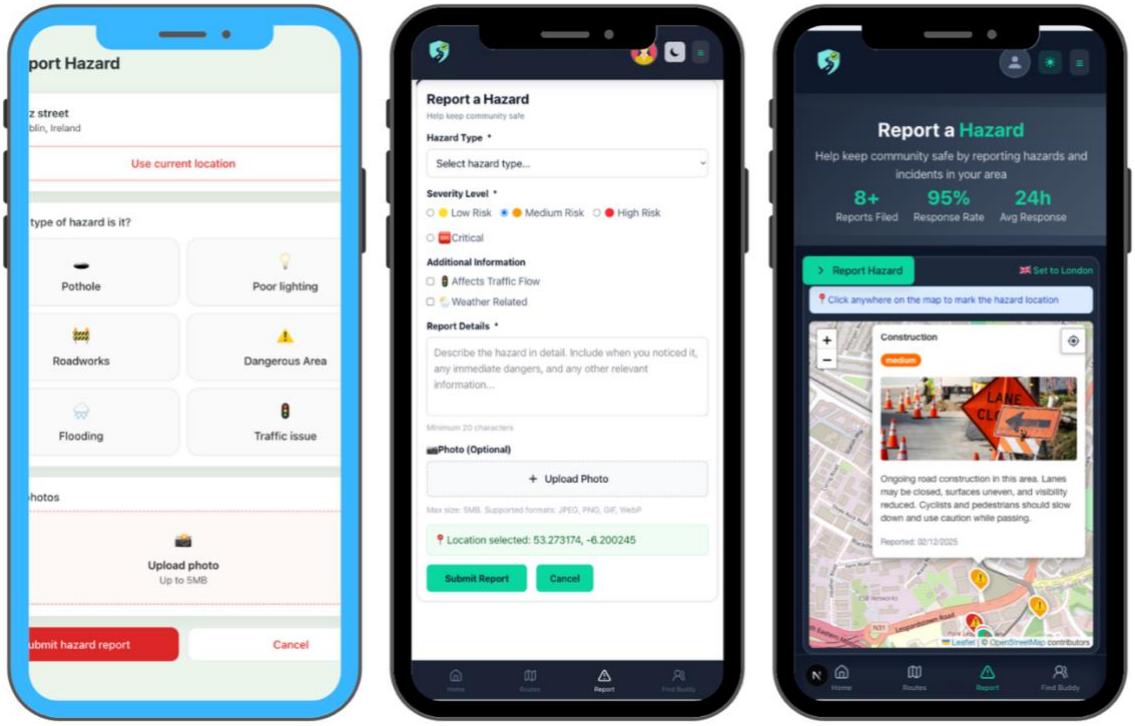

Figure 31 - Hazard Design History

Sitemap

Based on the user journeys, a sitemap can be developed to outline the overall structure of the web app. It provides a high-level view of how different elements are organized and connected, offering a clear overview of the user interface. This serves as a quick reference tool during the design process, helping to ensure consistency and clarity.

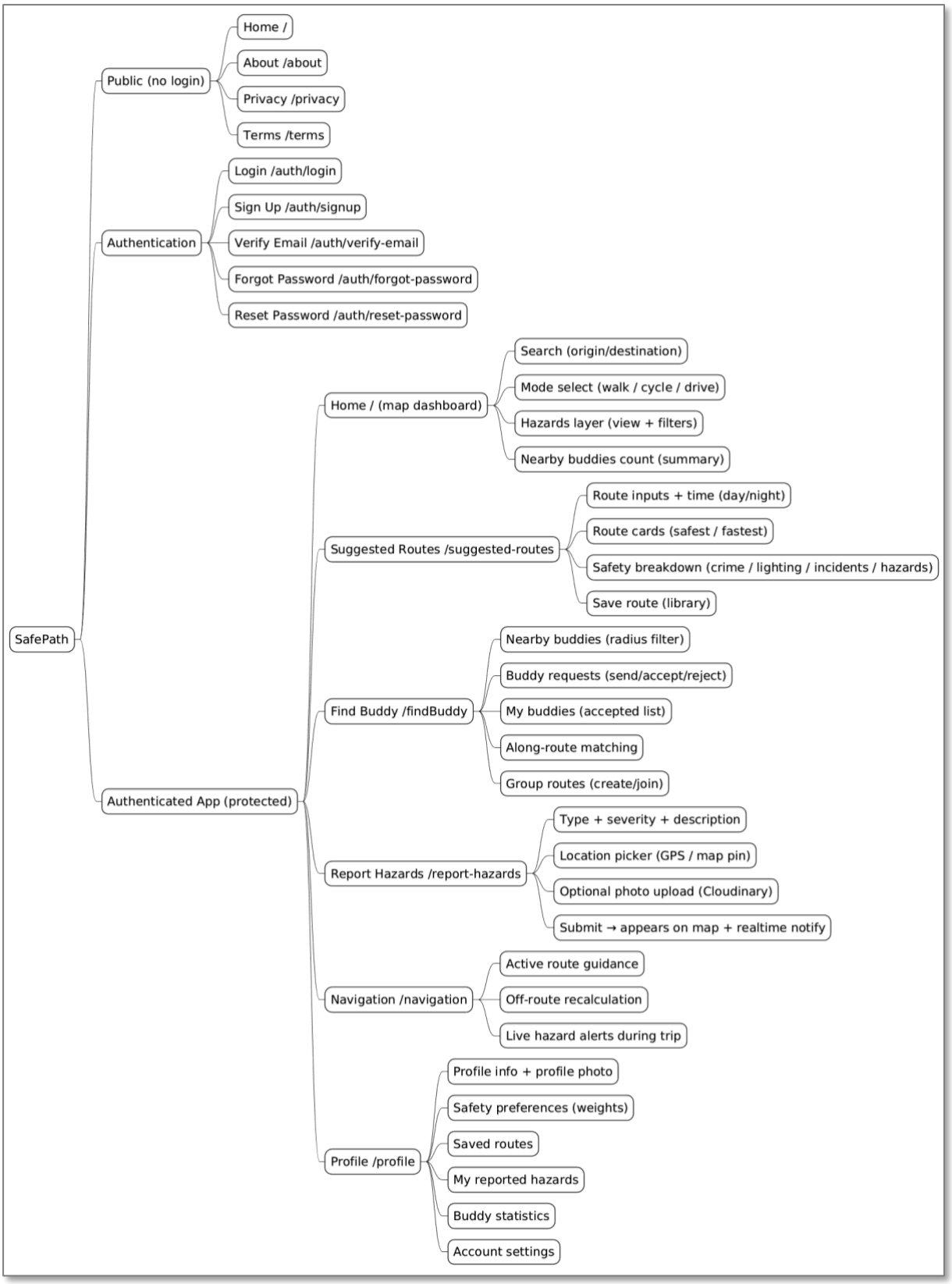

Figure 32 - SafePath Site Map

User Interface Components

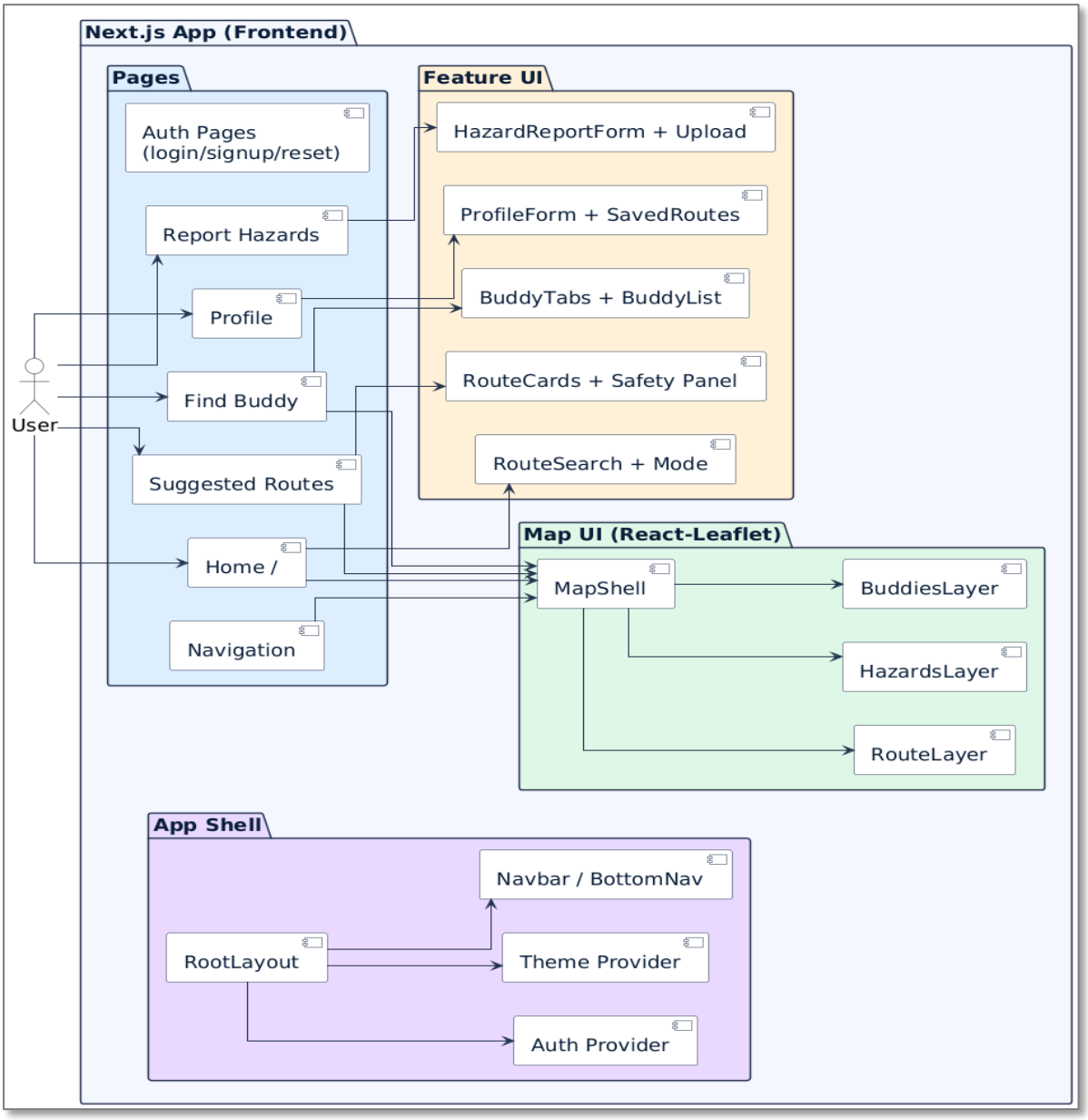

Figure 33 - Frontend Interface Component Diagram

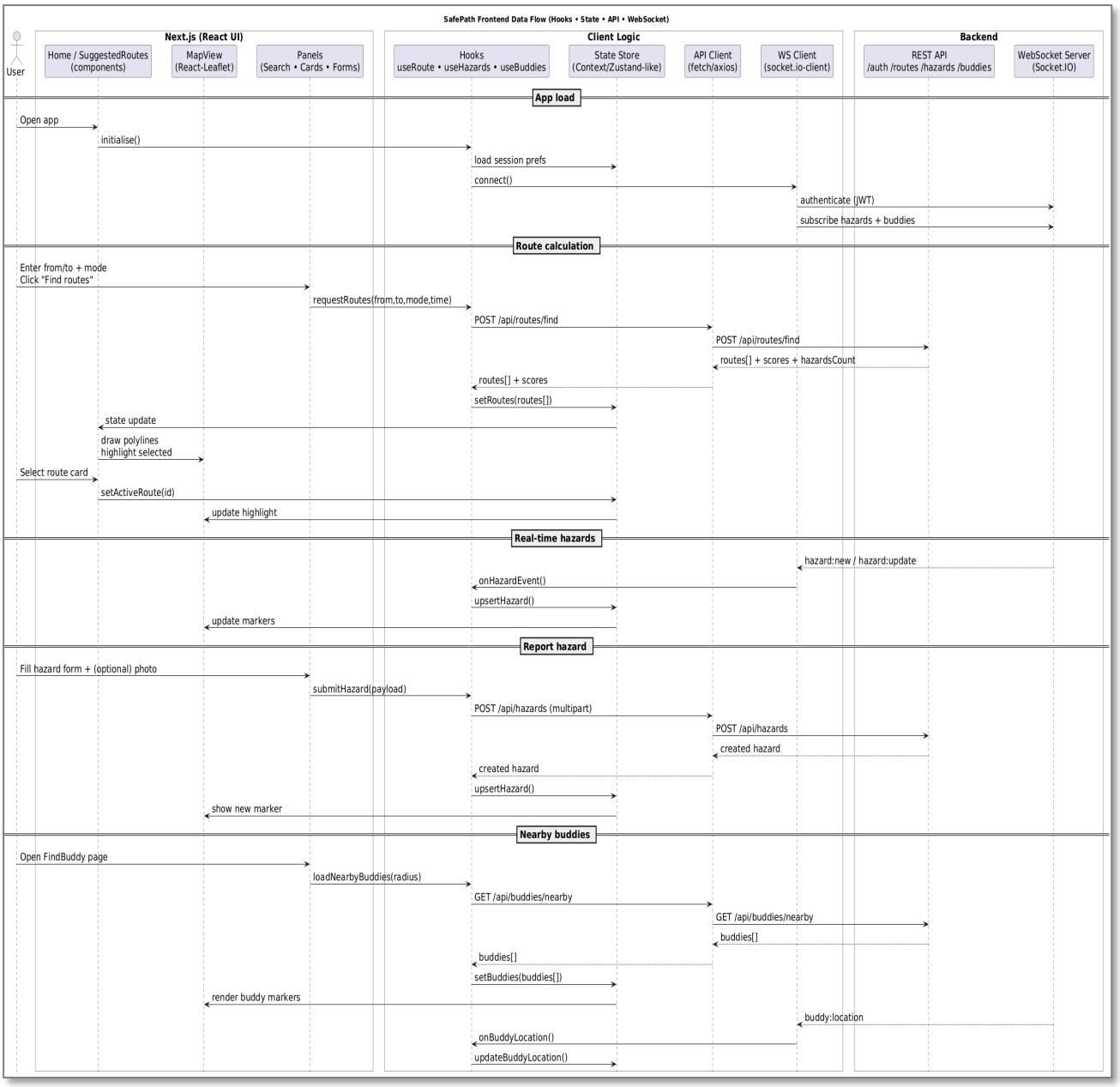

Figure 34 - Frontend Data Flow Diagram

Backend

Backend Architecture Overview

The backend of SafePath forms the core engine that connects all moving parts of the system. It acts as the bridge between raw spatial data, routing logic, user requests and the live hazard stream. The entire backend is built using Node.js twenty. Express is used to structure RESTful endpoints in a clean and modular manner, while Socket.IO supports real time communication. Everything runs on Render.com which manages HTTPS, deployment pipelines and scaling, allowing us to focus on logic rather than server maintenance. All sensitive configuration such as JWT secrets, Cloudinary keys and database URLs are stored as environment variables rather than being hard coded which keeps the system secure and portable.

Safety-First Routing Engine

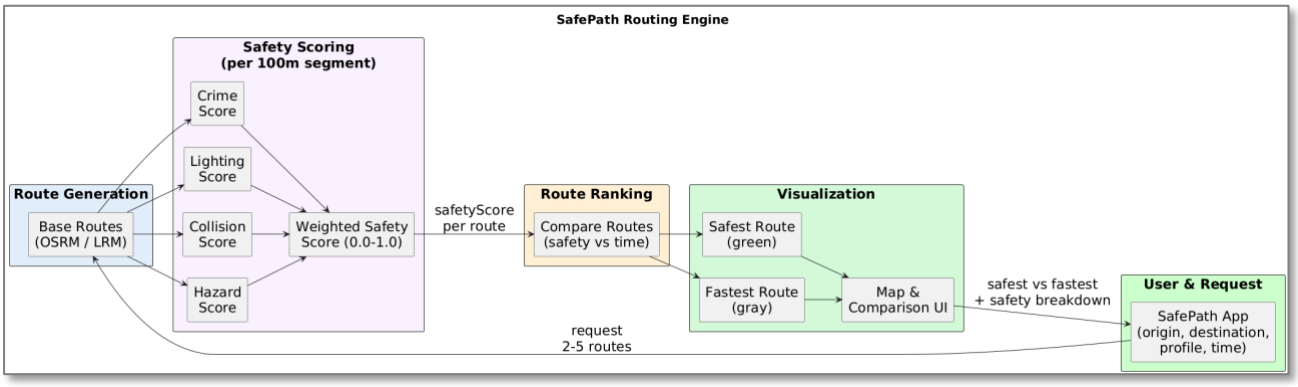

Figure 35 - Safety Routing Engine

Engine Purpose

The SafePath routing engine is designed as a safety-first extension of traditional shortest route navigation. Instead of optimising only for distance and travel time, the engine evaluates how risky each route is by combining multiple evidence sources: recent crime statistics, community-reported hazards, live traffic incidents, street-lighting conditions and historical collision data.

Conceptually, SafePath follows the line of work on safe or risk-aware routing, where paths are evaluated not only by length but also by exposure to crime or accidents. However, SafePath generalises these ideas into a multi-factor safety score and exposes the decision logic through an explicit rule-based engine.

For each user request (origin, destination, transport mode, safety preset and time of day), the engine:

1. Computes a conventional fastest route and a small set of alternatives using OSRM (Open Source Routing Machine).

2. Calculates a composite safety score for each candidate route.

3. Classifies the fastest route as SAFE, MODERATE or HIGH_RISK.

4. If necessary, synthesises safer alternatives by routing around dangerous segments.

5. Returns a concise comparison between the fastest, safest including the safety–time trade-off.

Figure 36 - Plan Route Step #1

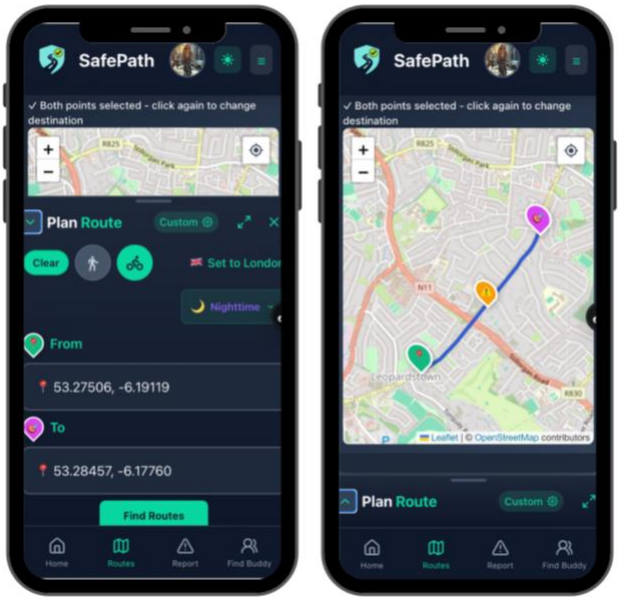

Figure 37 - Figure 25 - Plan Route Step #2

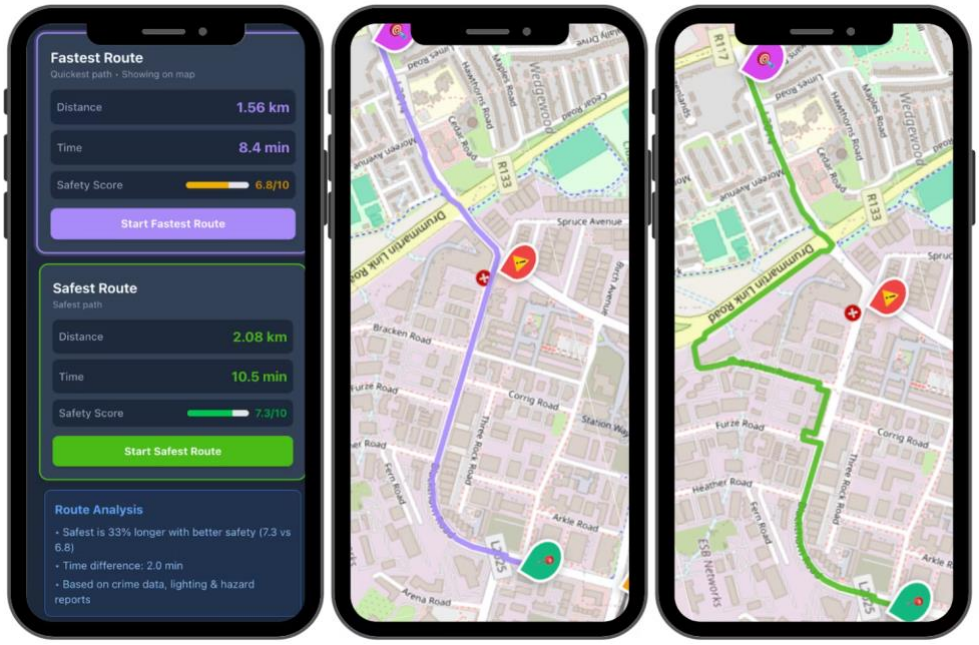

Figure 38 - Safest vs Fastest

Figures (36, 37 & 38) illustrates this flow from user input, through route generation and safety scoring, to the map visualisation in the Routes interface.

Inputs, Data Sources and Route Representation

The engine operates as a post-processing layer on top of OSRM using OpenStreetMap (OSM) as the base road network. For each origin–destination pair, OSRM provides:

- a fastest route polyline (sequence of coordinates),
- optional alternative routes, and
- basic distance/time estimates.

SafePath then enriches these routes with four safety dimensions:

1. **Crime rate and severity (UK Police API)**
    - Three months of crime data are imported as CSV and aggregated into a grid of 0.01° cells over Greater London.
    - Each crime type is mapped to a severity weight (e.g. violent and sexual offences ≈ 1.0, robbery and weapons ≈ 0.9, burglary ≈ 0.8, anti-social behaviour ≈ 0.3).
    - Cell scores are percentile-normalised to distinguish genuinely high-risk areas.

2. **Street-lighting quality (OpenStreetMap)**
    - OSM lighting tags and proxy features (e.g. distance from central London) are used to derive a **lighting index** (0 = well-lit, 1 = dark).
    - Lighting is treated as a key factor for night-time walking and cycling, reflecting empirical evidence that illumination affects both perceived and actual safety.

3. **Community-reported hazards and live incidents**
    - Hazards reported in the SafePath app (e.g. accidents, construction, flooding, poor lighting, unsafe crossings) are stored in PostgreSQL/PostGIS with type, severity, timestamp and geometry.
    - A PostGIS ST_DWithin query counts active hazards within a 500 m radius of each sampled point to estimate hazard density.

- TomTom Traffic Incident API events (accidents, closures, broken-down vehicles, severe jams) are mapped into the same hazard taxonomy and merged with community reports.

4. **Traffic collisions (historical road safety data)**
   - Where available, collision records are spatially aggregated into a collision density index.
   - This factor is particularly important during rush-hour periods, following prior work that combines crime and accident data for safe routing (Kim, Joo, & Park, 2011).

To make computation tractable, SafePath does *not* score every vertex of a path. Instead, each candidate route is:

- wrapped in a route corridor (typically 300–500 m on each side), which defines which hazards and crime points are "close enough to matter"; and
- split along its length into short route segments of approximately 100–200 m.

Within each corridor, only points that fall near a given segment contribute to that segment's local safety score. Figure 30 describe this distinction: the corridor controls *which* events are considered, while the segments control *where along the route* risk is measured and visualised.

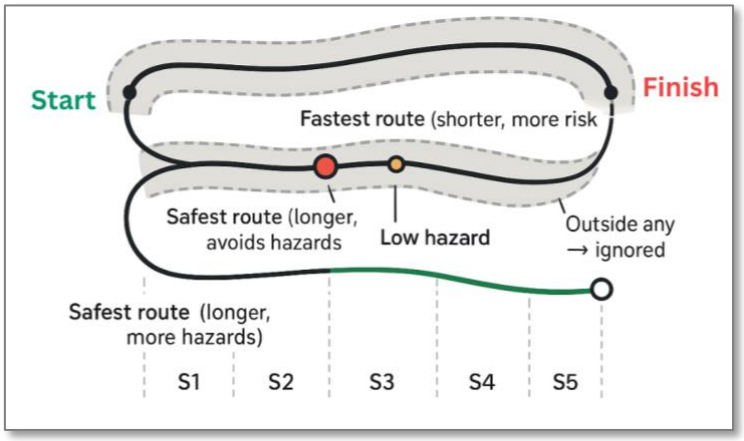

Figure 39 - Route corridor and segments illustration

Rationale for a Rule-Based System

Most recent studies explore learning-based or multi-objective optimisation approaches for safe guidance, including reinforced learning and multi-criteria shortest path algorithms (Kim et al., 2011). Although learning-based and multi-objective optimisation methods can achieve strong performance, they frequently operate as black boxes, which makes it difficult to justify individual routing decisions to end-users or examiners. In SafePath, we therefore made a deliberate design choice to implement the routing engine as a transparent, rule-based system. This decision is motivated by four considerations.

1. **Interpretability:** Every recommendation (for example, why Route B is preferred over Route A) can be traced back to explicit thresholds and IF–THEN rules. This level of traceability is important in a safety-critical context and for an MSc project that must be defensible at the level of individual assumptions and parameters.

2. **Deterministic behaviour.** A rule-based design guarantees that identical inputs produce identical outputs. This determinism simplifies systematic testing, debugging and the kind of mathematical verification we report later in this chapter.

3. **Practical constraints.** Within the limited time frame of an MSc project, developing, training, and testing a machine learning model for such purposes as crime or collision prediction would involve additional work with additional data.

4. **Future extensibility.** The current architecture already mirrors structures used in risk-aware routing research such as crime grids, hazard density fields and time-dependent weight profiles, so machine-learning components can be introduced later as additional risk layers feeding into the same rule base, rather than replacing it entirely.

In summary, the engine treats multi-factor safety scoring and rule-based classification as a baseline "safety layer" that can be audited in the present work and progressively extended with learning-based models in future iterations of SafePath.

Safety Scoring Setup and Two-Tier Weighting

At each sampled point $\boldsymbol{pi}$ along a route, the engine computes four normalised factors in [0,1]:

- $\boldsymbol{ci}$**:** crime rate and severity
- $\boldsymbol{coli}$: collision density
- $\boldsymbol{li}$**:** lighting index (0 = bright, 1 = dark)
- $\boldsymbol{hi}$: hazard density (community + TomTom).

These are combined into a composite point-level safety score using a weight vector $\boldsymbol{w}$ that always sums to 1.0:

$$\boldsymbol{safetyScore(pi) = ciwc + coliwcol + liwl + hiwh}$$

the $\boldsymbol{w}$ terms are the weights for each factor:

- $\boldsymbol{wc}$ = crime weight, how strongly crime risk influences the score.
- $\boldsymbol{wcol}$ = collision weight, how strongly traffic collisions / accidents influence the score.
- $\boldsymbol{wl}$ = lighting weight, how strongly street lighting quality influences the score.
- $\boldsymbol{wh}$ = hazard weight, how strongly live hazards (accidents, roadworks, flooding, user reports, etc.) influence the score.

And they satisfy: $\boldsymbol{wc + wcol + wl + wh = 1}$

The default weights are:

- crime: 0.40
- collisions: 0.25
- lighting: 0.20
- hazards: 0.15

Users can override these via the Safety Settings component, choosing presets (Balanced, Cyclist Safety, Night Walker, Crime Aware) as quick presets or specifying custom sliders as seen below in Figure - 40.

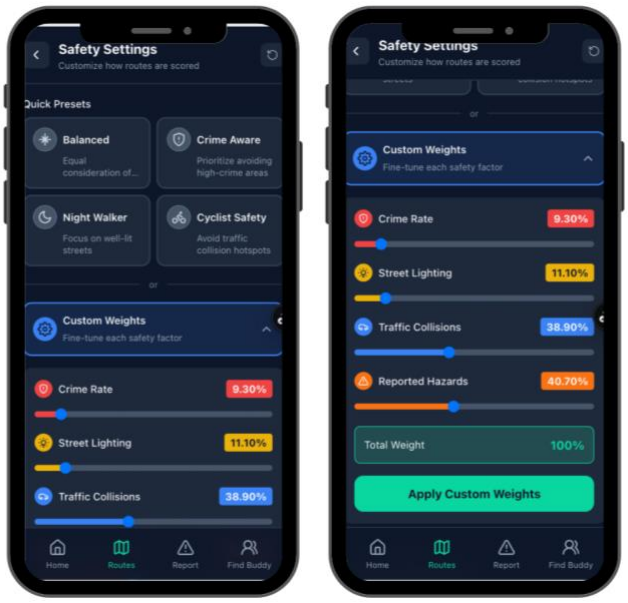

Figure 40 - User Safety Scoring Preferences

To prevent users from accidentally turning off critical dangers, like making some of factors pointing to zero, SafePath uses a two-tier weighting scheme:

1. **Backend severity multipliers (hard safety rules).**
    - Each hazard is assigned a fixed severity multiplier (e.g. critical ≈ 3.0×, high ≈ 2.5×, medium ≈ 1.2×, low ≈ 0.5×).
    - These multipliers are *not* user-configurable and guarantee that, for example, an accident or serious incident always dominates local risk, even if the user reduces the overall hazard weight.
2. **Frontend factor weights (personalisation layer).**
    - The user distributes 100% importance across the four dimensions (crime, lighting, collisions, hazards).
    - This changes the relative contribution of each dimension to the final safety score, but never cancels the backend severity multipliers.

For each candidate route, sampled points are aggregated into a route-level safety score $\boldsymbol{S}route$. Instead of a pure mean, the implementation blends the average and the worst segment:

$$\boldsymbol{S}route = 0.7\, s^{-} + 0.3\, \boldsymbol{S}max \ ,$$

where $\boldsymbol{s}^{-}$ is the mean of all point-level scores and $\boldsymbol{S}max$ is the maximum observed along the route. This intentionally penalises routes that contain short but extremely dangerous sections, which is consistent with safety-first routing approaches that avoid diluting local peaks of risk .

For user display, the route score is finally mapped to a 0–10 rating:

$$\boldsymbol{R}route = (\mathbf{1} - \boldsymbol{S}route) \times \mathbf{10},$$

so that higher values indicate safer routes. Time-of-day adaptation in SafePath is handled by the backend functions `detectTimeOfDay()` and `adjustWeightsForTimeOfDay()`. Four time contexts are distinguished (day, night, morning-rush, evening-rush), and each context adapts the base factor weights. In practice, lighting is given more influence at night and during the evening rush, while collision risk is given more influence during morning rush hours when traffic is heaviest.

This design is consistent with recent evidence that both objective risk and perceived safety vary with time and context. Research about crime, fear, and mental health related to the night city indicates that the night city is perceived as a more dangerous place and has a clear impact on the well-being of citizens, especially concerning the lack of illumination (Li et al., 2023). Recent research on safe pathfinding algorithms shows the interest in considering safety not just as a distance issue but incorporating various factors such as crime factors and illumination (Sohrabi, Weng, Das, & German Paal, 2022b). Complementary research on multi-criteria car navigation demonstrates how it is possible to combine safety-related attributes with traditional efficiency metrics in support of driver-oriented, safety-aware routing decisions (Solé, Sammarco, Detyniecki, & Miguel, 2022).

SafePath's time-dependent weight profiles adopt these insights in a pragmatic way: the engine places greater emphasis on lighting in night-time and evening-rush scenarios, and relatively more emphasis on collision risk during high-traffic morning rush periods.

Algorithm Pipeline and Core Implementation Functions

Algorithmically, the routing engine can be described as a five-phase pipeline:

1. **Data collection and context detection**
    - Input: origin, destination, transport mode and user safety preset from the frontend.
    - Detect time-of-day context and construct context-specific factor weights.
    - Load relevant crime, lighting, hazard and collision data along a corridor around the route(s).

2. **Generation of base routes**
    - Call OSRM to obtain a fastest route and up to 2–3 built-in alternatives.
    - For each route, build a corridor (e.g. 300–500 m) and split the polyline into ~100–200 m segments for local scoring.

3. **Per-segment safety scoring**
    - Sample points along each route segment.
    - For each point, compute crime, lighting, collision and hazard scores using the current weights and backend severity multipliers.
    - Aggregate over segments to obtain $\boldsymbol{S}route$ and user-visible rating $\boldsymbol{R}route$.

4. **Classification and alternative generation**
    - Classify each route as SAFE, MODERATE or HIGH_RISK based on $\boldsymbol{S}route$.
    - Identify dangerous segments where local scores exceed a danger-zone threshold (0.12 after calibration).
    - If the fastest route is MODERATE or HIGH_RISK, or contains dangerous segments, generate alternative routes using:
        - **OSRM alternatives** with different geometries,
        - **waypoint-based avoidance** around dangerous segments, and
        - **lateral offset routes** along parallel streets.

- When live accidents or critical hazards are detected (within ~500 m), apply aggressive hazard boosts and relax the maximum detour factor (up to 3×) to prioritise safety over travel time.

5. **Evaluation, selection and response**
   - For each candidate route, compute:
     - safety score $\boldsymbol{S}route$,
     - classification (SAFE / MODERATE / HIGH_RISK),
     - distance and travel time.
   - Compare the safest candidate against the fastest route using safety improvement and distance ratio rules:
     - accept if >15% safer and <25% longer,
     - or if >25% safer and <40% longer;
     - reject if not safer or >50% longer (unless critical hazards apply).
   - Return the fastest route and, if accepted, the safest route to the frontend with GeoJSON geometries, breakdown of safety components, and an explanation of the safety–time trade-off (e.g. “12% longer but 40% safer”).

These phases are implemented in a set of backend functions, including:

- `calculateFastestRoute()`: OSRM wrapper.
- `calculateRouteSafetyScore()`: per-point scoring and route-level aggregation.
- `classifyRouteSafety()`: SAFE/MODERATE/HIGH_RISK classification.
- `identifyDangerousSegments`(): detection of segments above the 0.12 threshold.
- `generateSafeWaypoints()` and `getOSRMRouteWithWaypoints()`: waypoint-based avoidance.
- `generateOffsetRoutes()`: lateral offset routing.
- `evaluateAlternative()`:rule-based acceptance/rejection of candidate routes.
- `findRoutes()`: orchestration entry point exposed via `/api/routes/find`.

On the client side, the Routes page coordinates user input, calls the backend API, and visualises fastest and safest routes with distinct colours and textual summaries. The Safety Settings modal manages presets and custom weights, and a time-of-day indicator makes the adapted behaviour explicit to users.

This architecture is broadly consistent with multi-criteria safety routing approaches that overlay risk scores on a conventional road network and then derive safer alternatives through constrained or weighted path search (Kim et al., 2011).

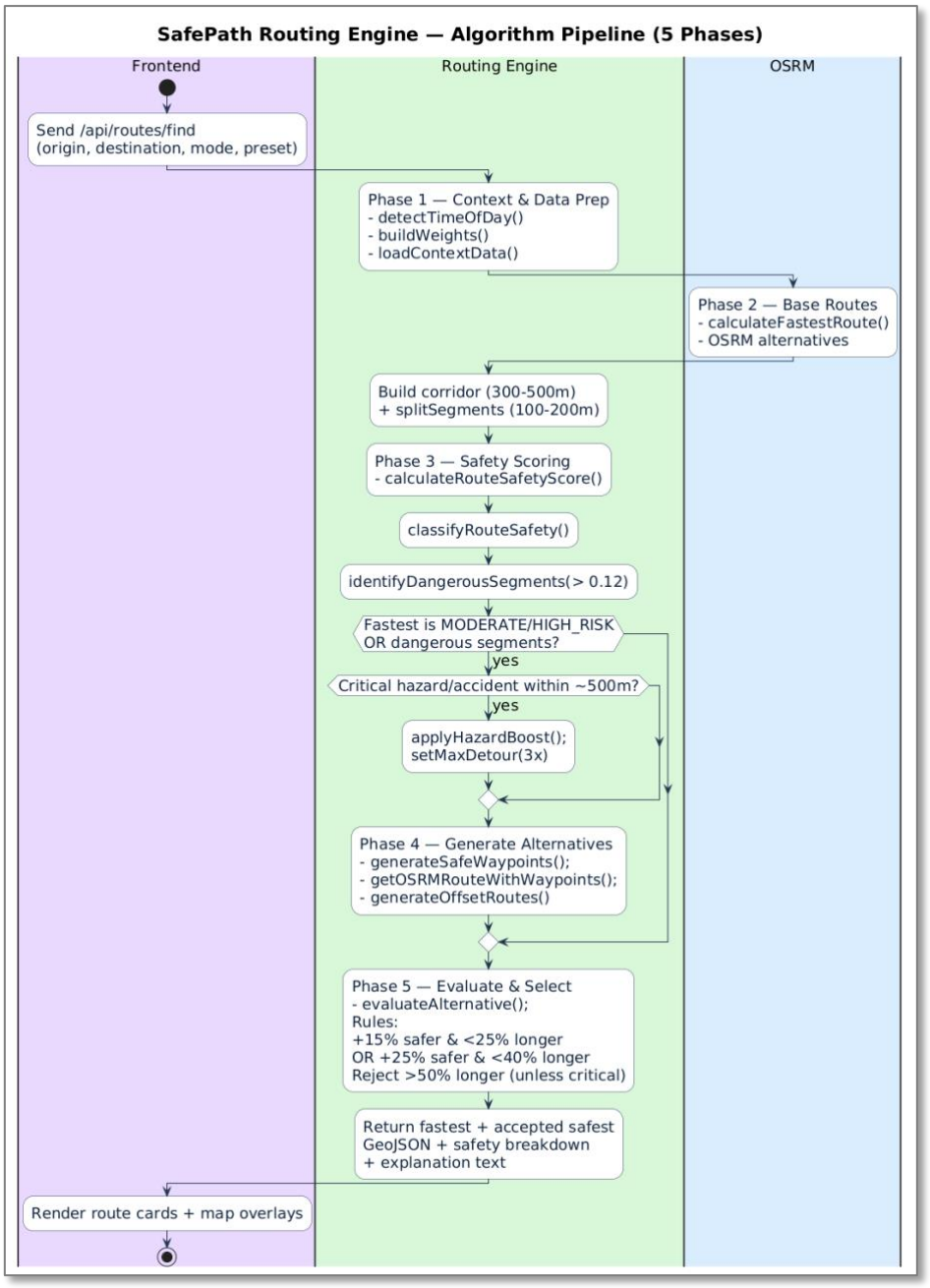

Figure 41 - Routing Algorithm

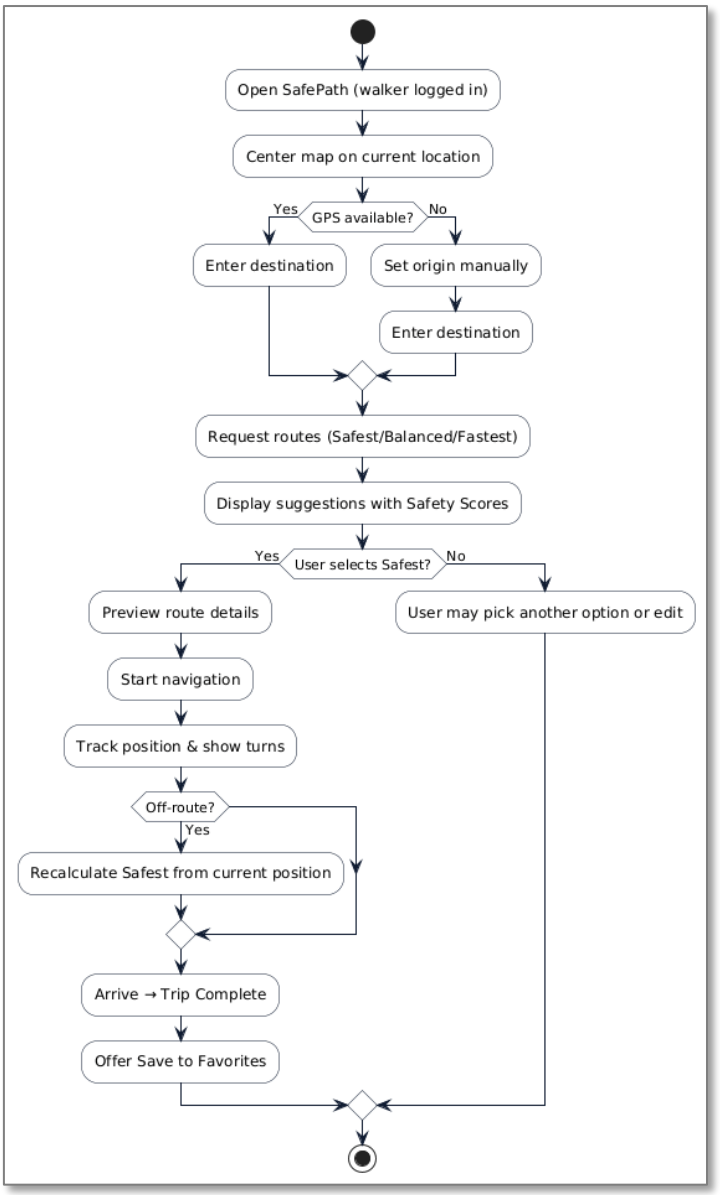

Figure 42 – Route Flowchart

Limitations and Design Trade-offs

Although it is quite complex, the current SafePath routing engine has several limitations:

- **Geographical coverage.** Crime and collision data are currently limited to the London Metropolitan area. Routes outside this region fall back to default or partial safety estimates. Similar constraints are reported in prior crime-aware routing work that focuses on a single city.
- **Data freshness.** Police crime datasets typically lag by 1–2 months; hence, sudden changes in crime patterns may not be captured immediately.
- **Simplified risk model.** Safety is represented as a single scalar score, rather than modelling full Pareto fronts of distance versus risk as in some multi-objective routing formulations (Chen et al., 2024). This is a deliberate design trade-off in favour of clarity and runtime simplicity.
- **No elevation or micro-infrastructure features.** The current model does not incorporate gradient, surface quality or detailed cycling infrastructure attributes (e.g. protected lanes).

- **Heuristic thresholds and multipliers.** Values such as the danger threshold (0.12), safety classification cut-offs (0.30, 0.60) and detour limits (25%, 40%, 50%) are chosen through a combination of literature-informed ranges and internal calibration. Different cities or user groups may prefer different parameters.
- **Limited empirical validation.** While the engine aligns with several published models of safe routing, a full validation against real-world incident outcomes or large-scale user studies remains future work.

Testing and Mathematical Verification

To ensure that the implemented routing engine matches the documented model, a dedicated mathematical verification and testing process was carried out on the backend module `routeCalculator.js`. The verification procedure included:

1. **Specification-to-code review.**
   - Default factor weights, scoring formulas, thresholds, and rule conditions were treated as the standard specifications.
2. **Formula and threshold checks.**
   - The composite safety score formula, the 70/30 average–maximum aggregation, the rating conversion $\boldsymbol{Rroute} = (\mathbf{1} - \boldsymbol{Sroute}) \times \mathbf{10}$, and the SAFE/MODERATE/HIGH_RISK thresholds were all checked against the code constants and arithmetic.
   - The danger-zone threshold of 0.12 was confirmed, including comments documenting its reduction from 0.15 during testing phase.
3. **Function-level traceability.**
   - All key functions described in the design (`calculateRouteSafetyScore`, `classifyRouteSafety`, `evaluateAlternative`, `detectTimeOfDay`, `identifyDangerousSegments`, `generateSafeWaypoints`, `generateOffsetRoutes`, `selectBestSafeRoute`, `findRoutes`) were located in the code and checked for behavioural consistency with the design description.
4. **Unit and integration tests.**
   - Unit tests assert that representative scores (e.g. 0.15, 0.45, 0.75) are classified into the expected safety categories.
   - Alternative-evaluation tests verify that specific combinations of safety improvement and distance ratio trigger the correct acceptance or rejection rule.
   - Integration tests run end-to-end route calculations between known locations and confirm that the “safest” route is strictly safer than the fastest baseline under the current weights.
5. **Load and robustness tests.**
   - Simple load tests (e.g. 100 concurrent route requests) were used to assess performance under moderate pressure and to confirm that external API failures (e.g. OSRM endpoint downtime) are handled via fallback endpoints rather than crashing the service.

The outcome of this process is a confirmed one-to-one correspondence between the documented safety-aware routing model and the actual backend implementation. Minor enhancements, such as the inclusion of the maximum segment score alongside the mean and the reduction of the danger threshold from 0.15 to 0.12—have been documented as calibrated improvements rather than undocumented deviations. This traceability supports the validity of the experimental findings reported elsewhere in the thesis and strengthens the argument that observed user behaviour can be attributed directly to the formally specified routing logic.

How Community Hazards are Reported

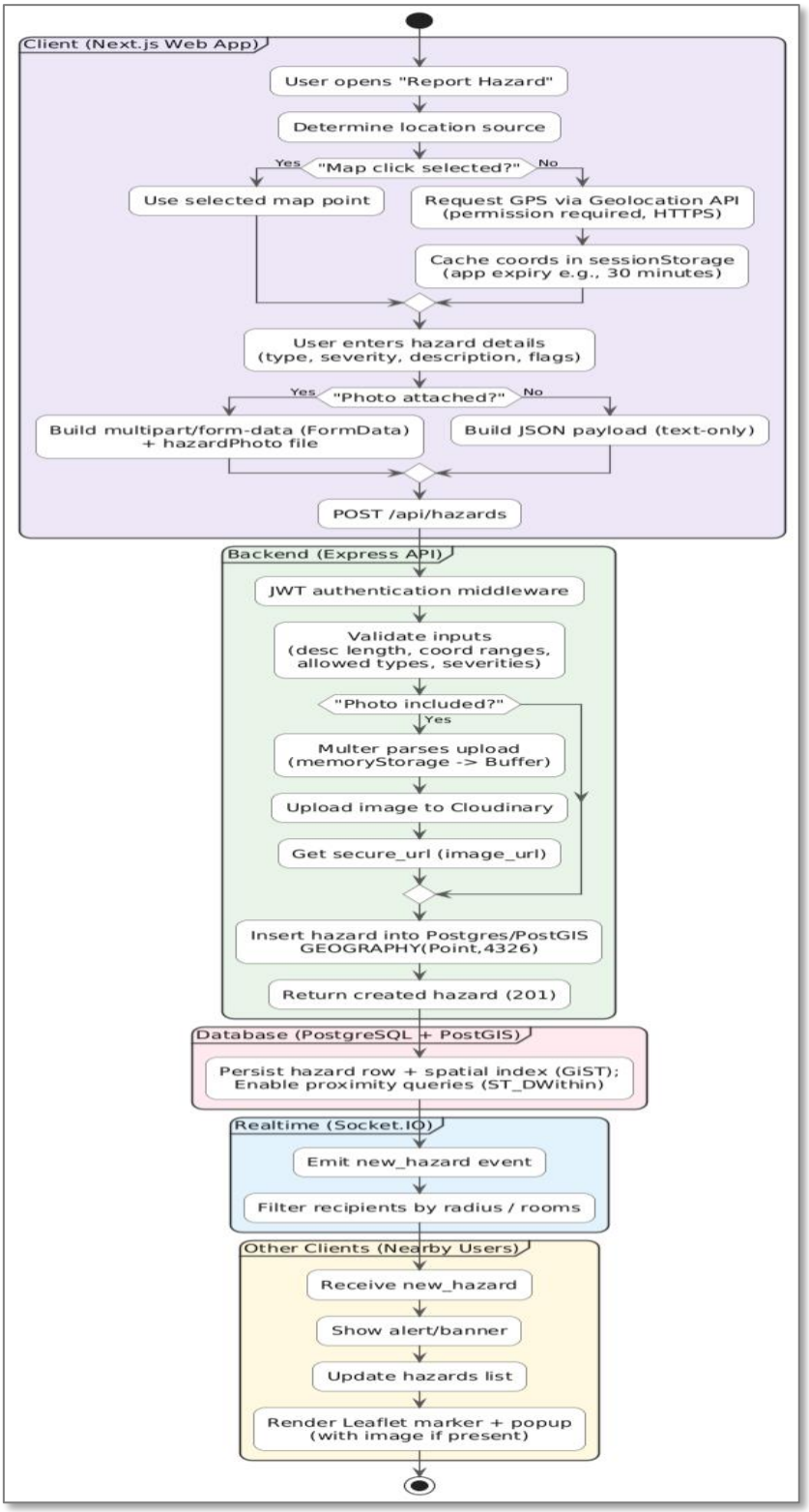

Figure 43 - Report Hazard Flow Diagram

SafePath allows users to report real-time road hazards (such as pot holes, road work, accidents, flooding, lighting) and post these hazards to nearby users in real time. The pipeline integrates permissioned GPS data gathering, temporary client-side caching,

PostgreSQL/PostGIS Geographic storage, and Socket.IO push notifications for live map updates .

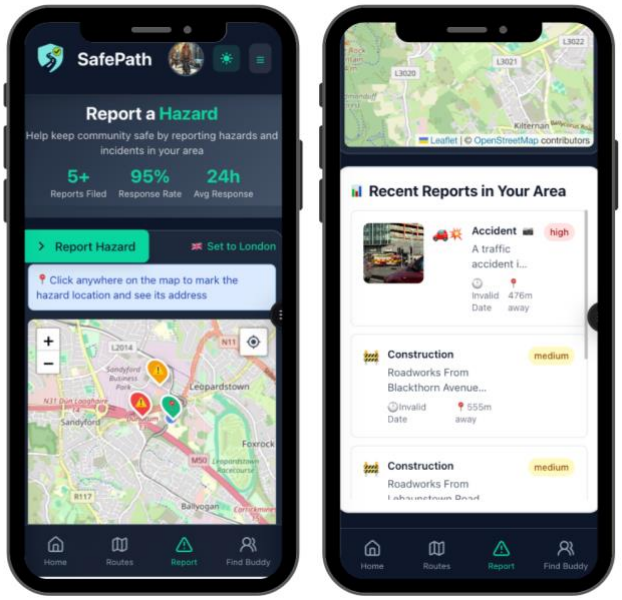

Figure 44 - Report Hazard Main Page

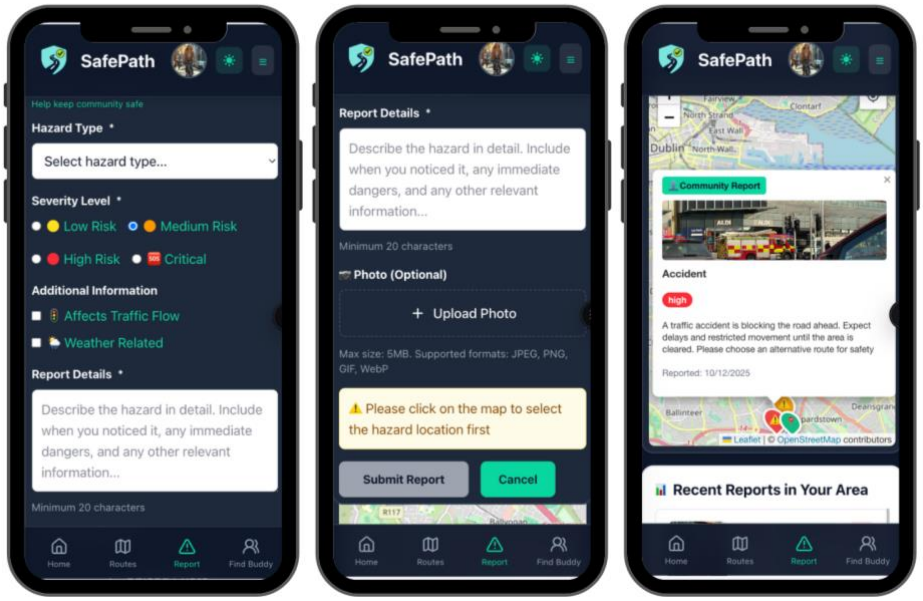

Figure 45 - How to Report Hazard

Below simple story, end-to-end, of how a real-time user hazard “spot” is created and shown on the map:

1. **User chooses the hazard location**
   - The user either clicks a point on the map (best when they’re reporting something they can see ahead), or the app uses their current GPS location (browser Geolocation).
   - Now you have the hazard coordinates: latitude + longitude.

2. **Frontend sends the hazard report to the backend**
   The frontend sends:
   - hazard details (type, severity, description, flags)
   - the coordinates (lat, lon)
   - optional photo (if there is a photo, it uses multipart/form-data, otherwise JSON)

3. **Backend validates and prepares the "hazard spot"**
   On the server you:
   - check the user is logged in (JWT)
   - validate description and coordinate ranges
   - convert the coordinates into the PostGIS format:

   Important rule: PostGIS point uses (longitude, latitude) order.
   So, the stored "spot" is built like:
   - `ST_Point(lon, lat)` - makes the point
   - `ST_SetSRID(..., 4326)` - marks it as GPS/WGS84
   - `::geography` - so distance calculations work in meters

4. **Save it in the database**
   insert a new row into hazards with:
   - type, severity, description
   - `location` = the PostGIS geography point
   - `latitude` and `longitude` (also stored for quick access)
   - image_url if uploaded to Cloudinary

   Now the hazard persisted (it won't disappear if someone refreshes).

5. **Broadcast it in real time (Socket.IO)**
   Right after saving, the backend emits a Socket.IO event like:
   - `new_hazard` with the hazard payload (id, lat, lon, type, severity, etc.)

   You don't need to wait for polling; this is a push update.

6. **Nearby clients receive it and update the map instantly**
   Any connected client listening to `new_hazard` will:
   - add the hazard to the hazards list
   - render a Leaflet marker at (lat, lon) and attach a popup (description + photo if present)

   So, the hazard appears on the map immediately.

7. **If someone refreshes, they still see it**
   When the page reloads:
   - the client calls "get nearby hazards"
   - the backend runs a query like `ST_DWithin(location, user_point, radius_m)`
   - returns hazards within the radius
   - map renders them again

Big idea:
- **"Spot calculation"** = turn (lat, lon) into a PostGIS geography point with SRID 4326 (meters-friendly).
- **"Real-time display"** = save to DB + emit `new_hazard` over Socket.IO + Leaflet renders a marker.

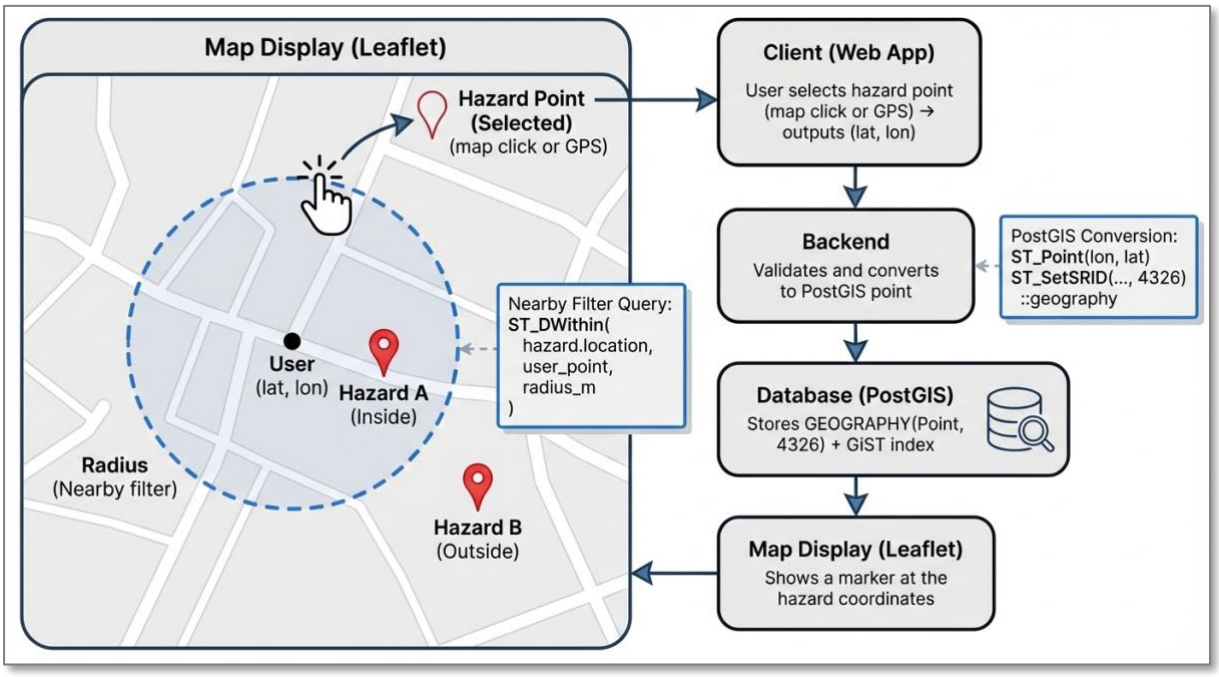

Figure 46 - Hazard real-time spot

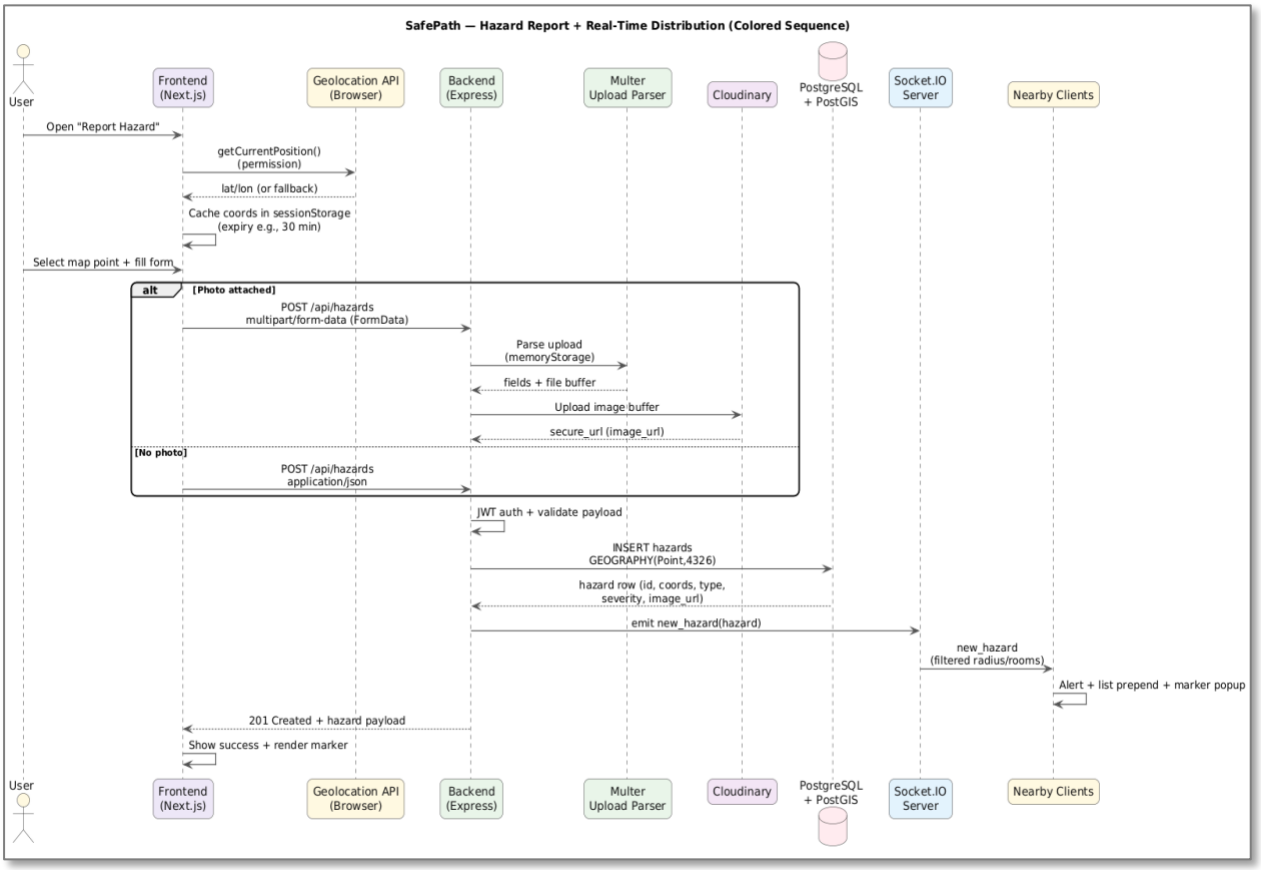

Figure 47 - Report Hazard Sequence Diagram

Hazard Reporting System - Current Limitations

SafePath allows users to report road hazards such as potholes, broken streetlights, accidents, and dangerous road conditions. When a user submits a hazard report, it appears immediately on the map for all nearby users to see. While this real-time sharing works well technically, the system has some important limitations.

First, there is no way to check if hazard reports are accurate. Any user can submit a report, and it appears instantly without any review or validation. This means false reports, spam, or incorrect information could appear on the map and potentially mislead other users. Although the system stores information about who reported each hazard and when it was reported, there is currently no admin dashboard or moderation tools to review and manage these reports.

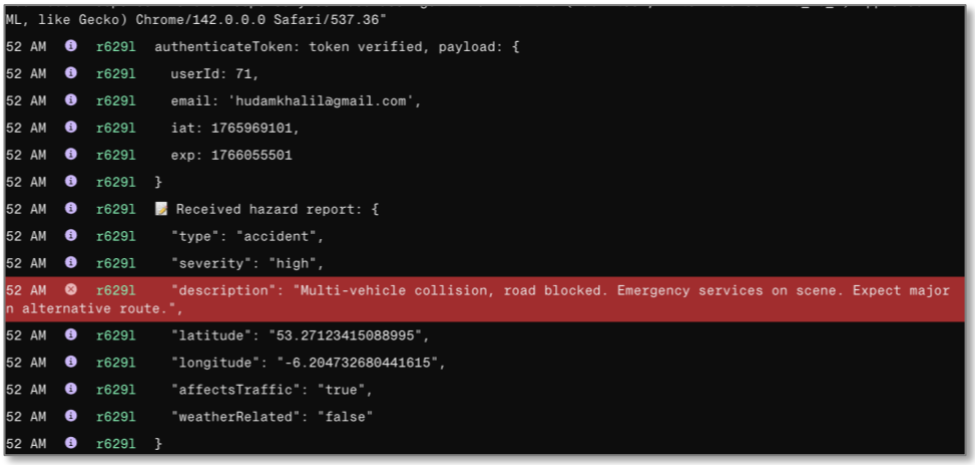

Figure 48 - Report Hazard Tracking Data

Second, the system cannot track the quality of reports over time. The database includes a field designed for community voting on hazards, where users could confirm if a reported hazard is real, but this feature was not completed within the project timeframe. Without this verification system, there is no way to tell which reports are reliable and which might be questionable.

Comparison with Waze

To understand these limitations better, it helps to compare SafePath with Waze, a well-established navigation app that also uses community-reported hazards. Following guidance from project mentors, we examined Waze as a benchmark for mature hazard reporting systems. Waze allows millions of users to report road hazards including construction, potholes, accidents, objects on the road, and broken traffic lights (Diaz, 2024). Waze has developed sophisticated systems to check report quality, including moderation tools, user reputation scores, and automated detection of suspicious patterns.
SafePath uses a similar concept of community reporting, where users share real-time information about road conditions to help others stay safe. SafePath already includes turn-by-turn navigation with real-time hazard alerts displayed during active navigation sessions, showing users warnings about accidents, poor lighting, and construction ahead with distance indicators and safety scores. However, Waze has been operating for many years with millions of users, which has allowed them to build and refine validation systems for report quality. SafePath successfully demonstrates that community hazard reporting and navigation work technically, but reaching the same level of reliability and community trust would require implementing similar quality control features.

Hazard Future Enhancement

Integration with Waze

Based on mentor recommendations and our comparative analysis, we plan to integrate SafePath with Waze's hazard data to enhance coverage and reliability. Waze provides developer tools that allow other applications to access crowdsourced hazard information from their extensive user base (Waze for Developers, n.d.). This integration would significantly strengthen SafePath's hazard detection capabilities.
The primary goal of this integration is to access hazard information from Waze's large community of users through the Waze Data Feeds API (Waze for Developers, n.d.). This would be especially helpful when SafePath is first launched in a new area, because the number of SafePath users reporting hazards would initially be small. By combining hazard reports from both SafePath users and Waze users, the system could provide more comprehensive safety information from the start.

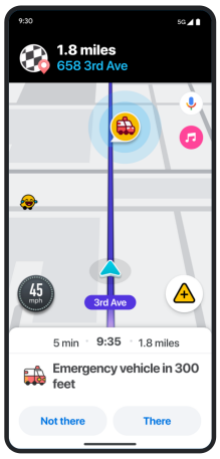

Figure 49 - Waze Hazard Expiry Check
Note: This picture from Waze help page (*Spot Emergency Vehicles in Your Area - Waze Help*, n.d.)

Waze's hazard data includes real-time reports of accidents, construction zones, road closures, potholes, and other road hazards contributed by millions of users globally (Diaz, 2024). These reports would be integrated directly into SafePath's existing navigation interface, enriching the real-time alerts already shown during active navigation sessions.

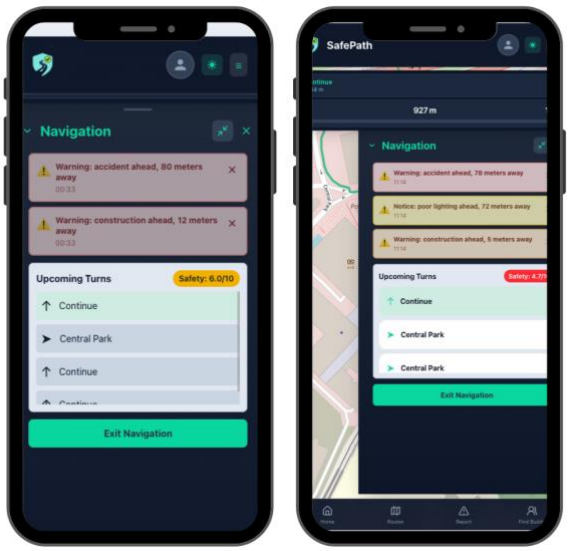

Figure 50 - Real Time Hazard - Navigation View

Users would continue to use SafePath's turn-by-turn navigation with safety scores and hazard warnings, but with significantly more comprehensive hazard coverage thanks to Waze's community contributions.
This approach solves a critical challenge for new safety applications and building sufficient hazard coverage during the initial deployment phase. Instead of waiting months or years to grow a large reporting community, SafePath could immediately benefit from Waze's established network while building its own user base. The integration would be transparent to users, who would simply see more complete and reliable hazard information within SafePath's safety-focused navigation experience.

Hazard Hotspot Heatmaps Using Grid Segmentation

SafePath currently displays community risks as individual points “markers” in real time, because users need accurate locations and specific details about reports (type, severity, image). As a future enhancement, SafePath can add an optional “Hazard Hotspots” heatmap layer that aggregates recent hazard points into spatial “hot areas” to make patterns easier to see and reduce marker clutter. This idea is motivated by Castells-Graells, Salahub, and Pournaras’ work on cycling safety mapping, which demonstrates how geolocated safety events can be transformed into a city-wide risk representation (they estimate a continuous spatial risk surface using kernel density estimation and use it to support safer route recommendations)(Castells-Graells, Salahub, & Pournaras, 2020).

How it would work in SafePath

- **Aggregate hazards into grid cells:** Snap hazard points into consistent cells (e.g., ~111m) using `ST_SnapToGrid()`, then compute per-cell metrics (hazard_count, severity-weighted intensity, most_recent). This produces hotspot “cells” rather than individual reports.
- **Fast delivery via materialized view:** Create a `hazard_density_grid` materialized view and refresh it on a schedule (e.g., every 10–15 minutes). This keeps hotspot queries fast and avoids expensive aggregation on every insert.
- **Frontend heatmap overlay:** Add a toggle in the map UI: Pins (live hazards) vs Heatmap (hotspots)**.** The heatmap reads `(lat, lon, intensity)` per cell and renders a smooth overlay layer.

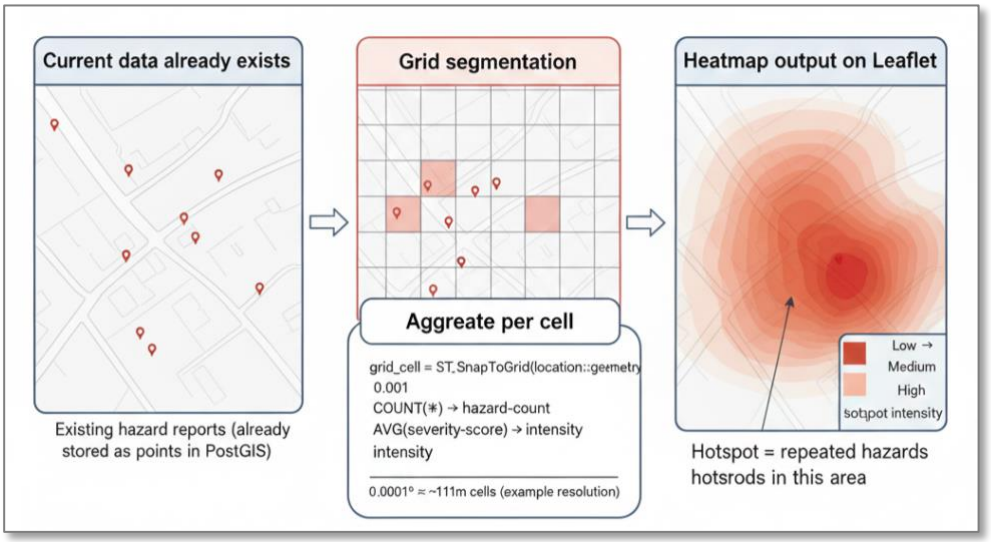

Figure 51 - Hazard Hotspot Heatmap (Grid Density Layer)

Findbuddy Feature

Purpose and user value

FindBuddy is a safety-by-companionship feature: when a user is travelling (walking/cycling), the system helps them discover nearby users, connect via a request, and (if connected) coordinate routes/group routes. The business value is straightforward: people feel less vulnerable when they are not travelling alone, especially at night or in unfamiliar areas, so FindBuddy supports SafePath's mission beyond "safer routes" by enabling safer journeys through social support.

How FindBuddy works in SafePath

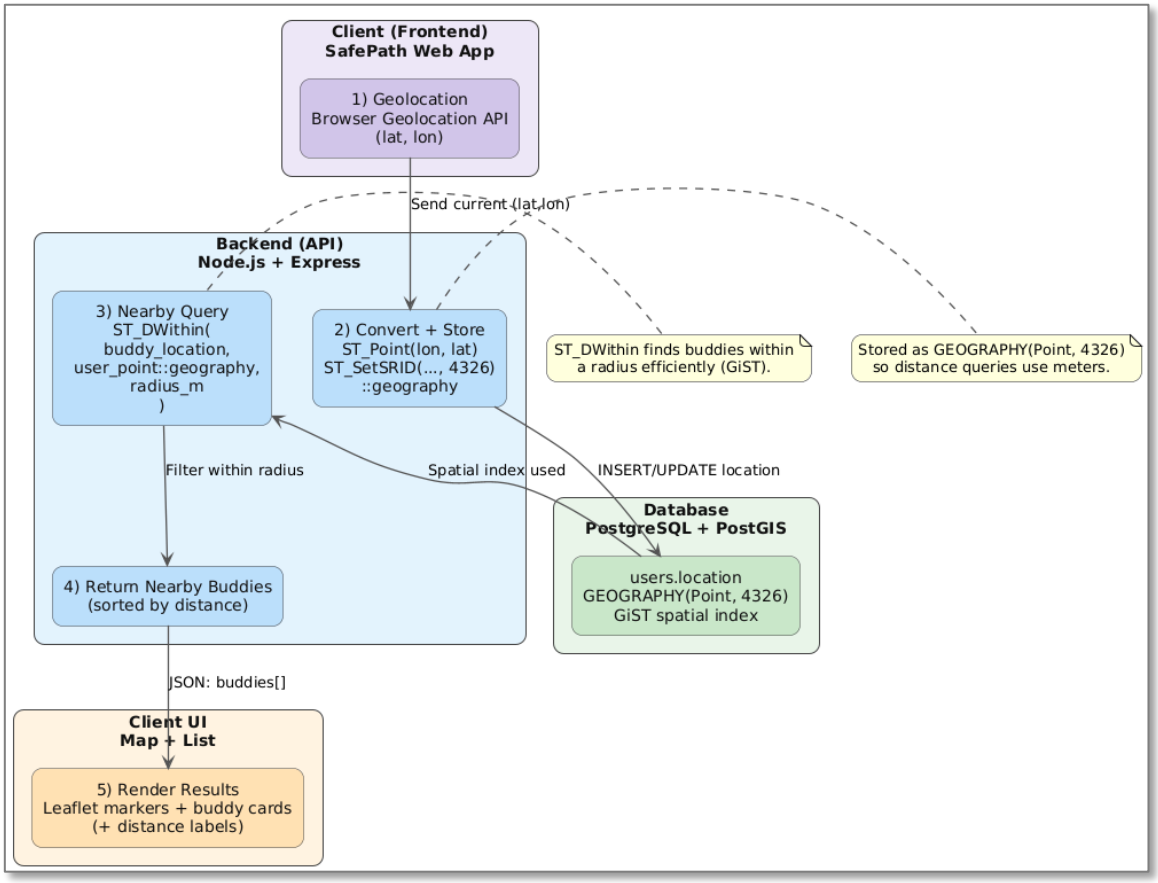

Figure 52 - Findbuddy End-to-End Flow

FindBuddy is implemented as a JWT-protected, location-based discovery + consent workflow in the Next.js client and an Express/PostGIS backend. When a logged-in user opens the FindBuddy page, the client retrieves a current GPS point (preferably from session-scoped storage, otherwise via the Browser Geolocation API), then persists that point to the backend so spatial queries can be performed reliably. The backend stores the user's location in `users.location` as a PostGIS GEOGRAPHY(Point,4326) and uses ST_DWithin to filter nearby users within a radius, and ST_Distance to return a distance value for sorting and UI display. The discovery response includes derived request state by left-joining the `buddy_requests` table, allowing the UI to show Pending / Connected / None without additional calls.

On the client, nearby users are shown in two synchronized views:
(1) a Leaflet map with a green “You” marker and blue buddy markers, and
(2) **a** Buddy Cards panel that displays name, distance, transport mode, and safety priority.

Figure 53 - Findbuddy - Request

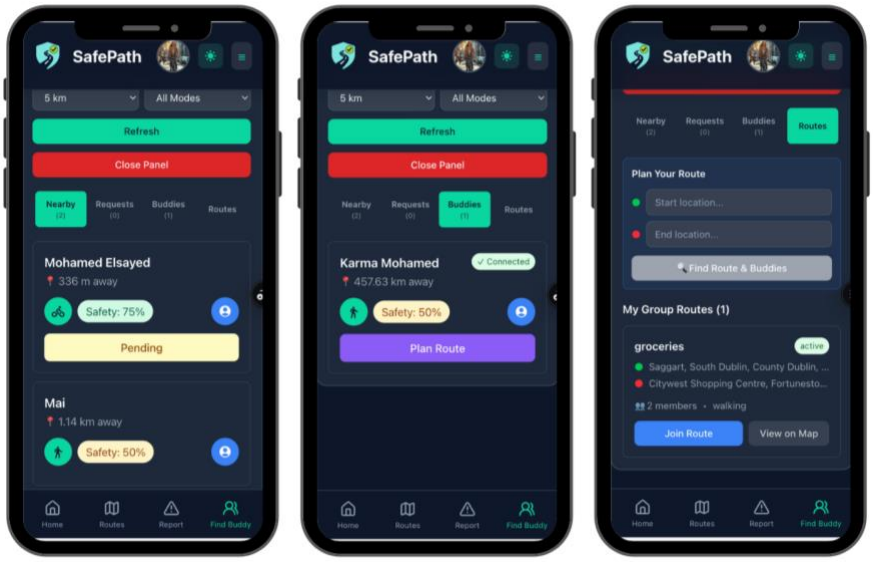

Figure 54 - Findbuddy Accept Request

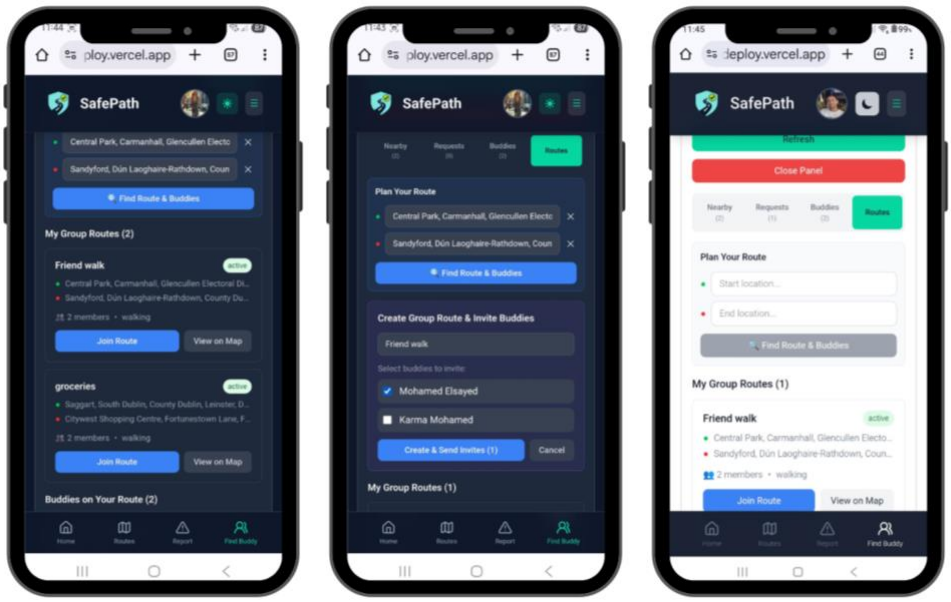

Figure 55 - Findbuddy Groups

Users can send a buddy request to a nearby user via `POST/buddies/requests`, which inserts a `pending` row into `buddy_requests`. Received pending requests are listed via `GET /buddies/requests?status=pending`, and the receiver can accept/reject via `PUT /buddies/requests/:id` which updates status to `accepted` or `rejected`. Accepted connections are retrieved via `GET /buddies/accepted`, optionally calculating distance from the user's current point (if lat/lon are provided) to order connected buddies by proximity.

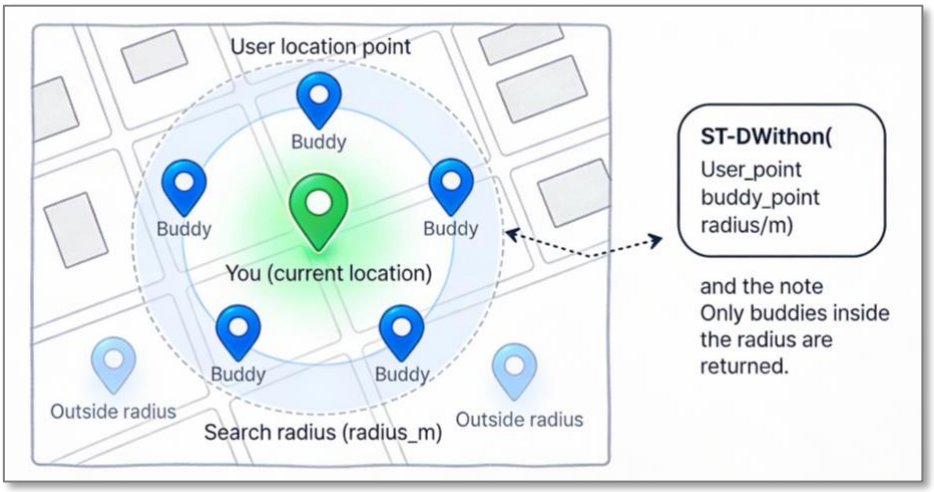

Figure 56 - Findbuddy Spatial Query Concept

FindBuddy's spatial matching is based on a simple "users within radius" concept: the client provides the current user's latitude/longitude, the backend builds a user_point (WGS84 / SRID 4326) and runs a PostGIS proximity filter using ST_DWithin(user_point, buddy_point, radius_m) to return only buddies whose stored `users.location` falls inside the search circle. Because SafePath stores locations as GEOGRAPHY(Point,4326), the radius is interpreted in meters, making it suitable for real-world distance checks; the results are then ordered using ST_Distance so the UI can show nearest buddies first, while any users outside the radius are excluded by the query (and therefore never returned to the client). For

scale, this pattern is designed to work efficiently with a spatial index on the location column (e.g., GiST)(*Chapter 4. Data Management*, n.d.; *ST_Distance*, n.d.; *ST_DWithin*, n.d.).

Finally, the client can create and manage lightweight "group routes" using `POST /buddies/group-routes`, `GET /buddies/group-routes`, and `POST /buddies/group-routes/:id/join`, inviting only users who already have an accepted buddy relationship.

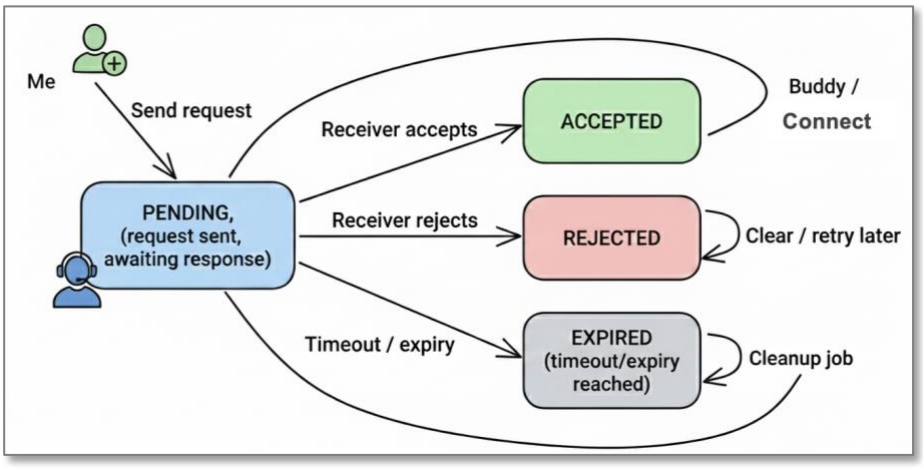

Figure 57 - FindBuddy Request State Machine

SafePath's FindBuddy radius-based discovery (returning users whose stored location falls within a distance threshold, like `ST_DWithin(user_point, buddy_point, radius_m)`) was inspired by research "Find Me If You Can: Improving Geographical Prediction with Social and Spatial Proximity" (Backstrom et al., 2010). It was also shaped by location-proximity service literature that formalizes "within-radius" proximity checks and discusses privacy-aware proximity mechanisms (which aligns with our consent-based buddy requests on top of proximity filtering) (Location Privacy, n.d.).

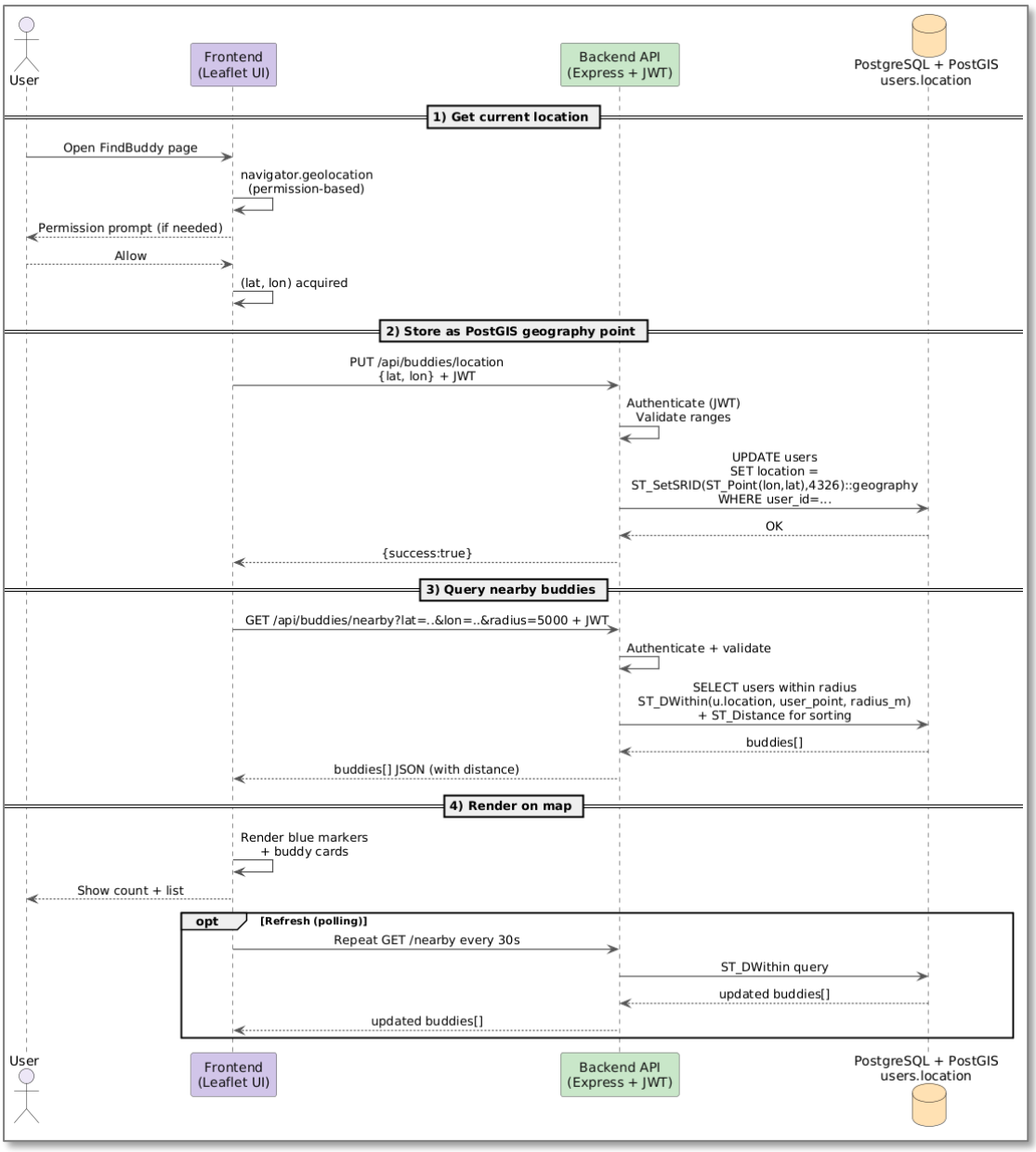

Figure 58 - FindBuddy Architecture Sequence

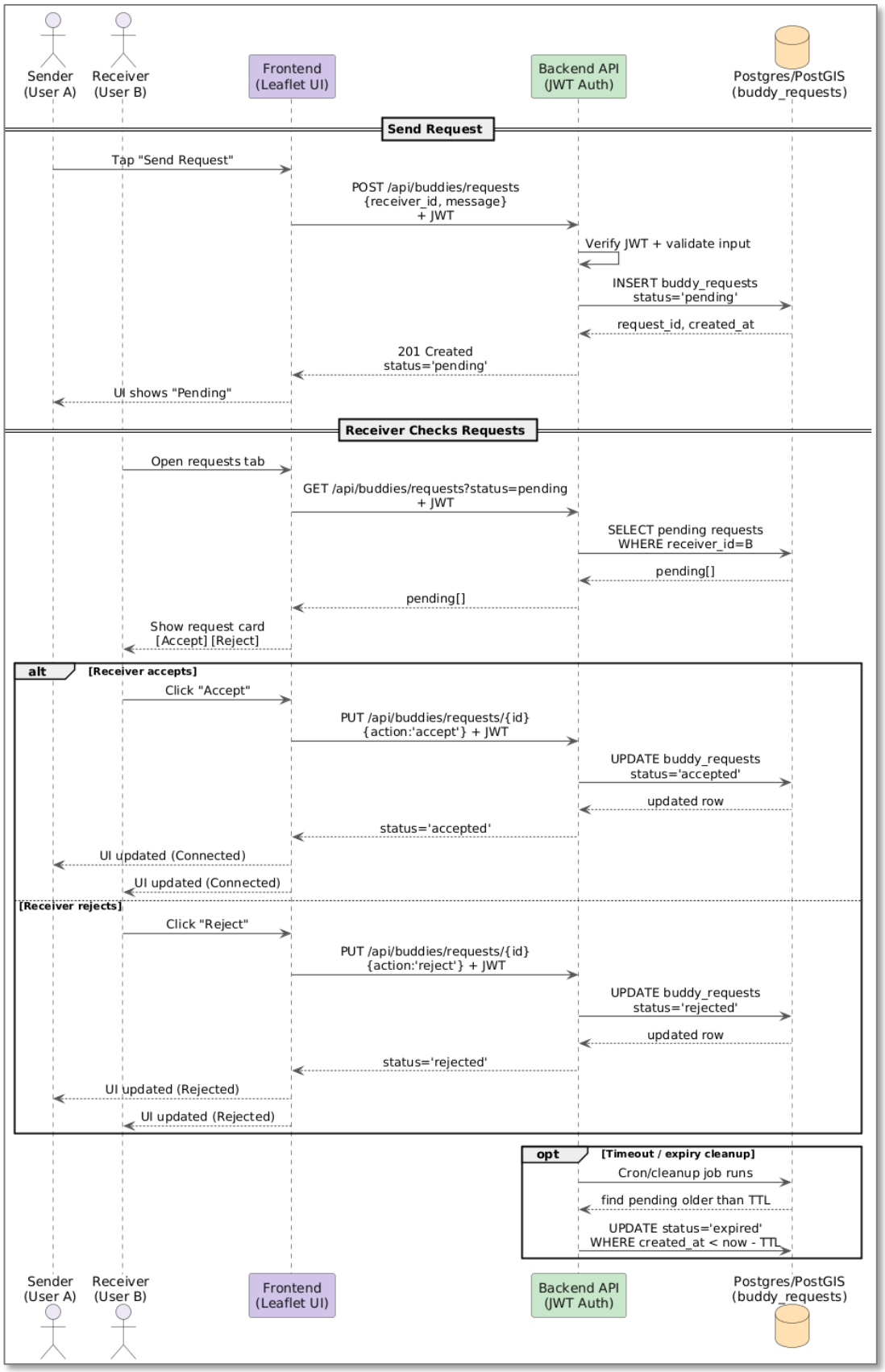

Figure 59 - Buddy Request - Life Cycle

Current Limitations

The Find Buddy feature helps users connect with other people traveling in similar directions so they can walk or cycle together for safety. While the core functionality works, several limitations affect the system's reliability and user accountability.
The most significant limitation is that the system does not keep proper records of buddy connections and shared journeys. While the system tracks when users send and accept buddy requests, it does not maintain detailed logs of actual shared trips. If a safety incident occurred during a matched journey, there would be no way to review what happened, which routes were shared, or even confirm which users were traveling together. This represents a significant gap for a safety-focused application.

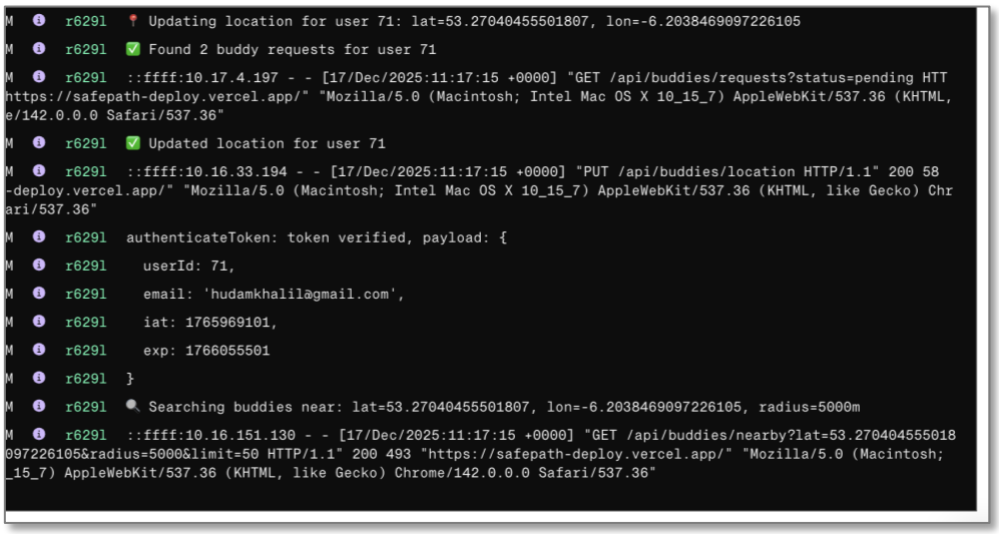

Figure 60 - Tacking Findbuddy Request

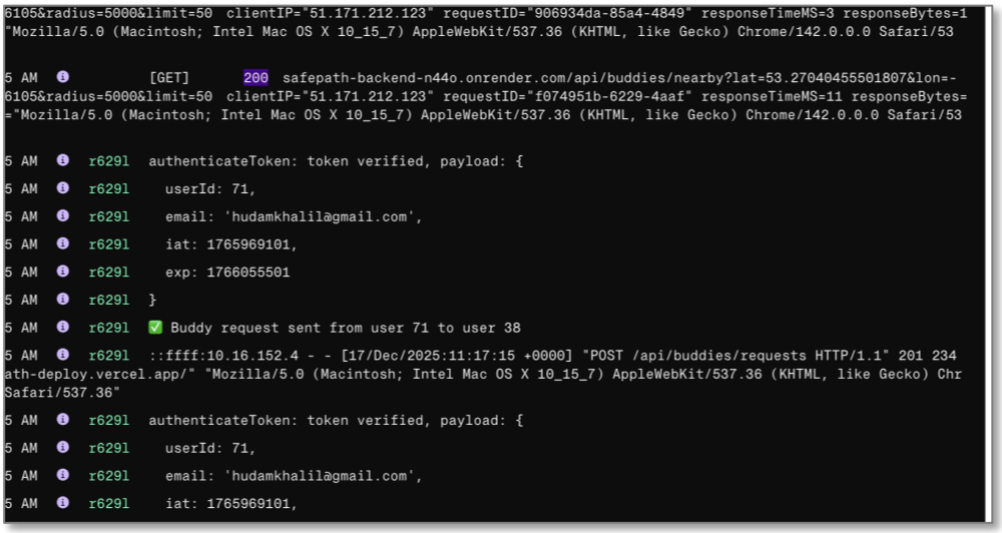

Figure 61 - Tracking Findbuddy Accept

Future Enhancement

Several enhancements would strengthen the Find Buddy feature's safety and usability. First, implementing comprehensive audit trails would track all buddy interactions including when requests were sent and accepted, which routes were shared, journey start and completion times, and user profiles involved in each match. These logs would be essential for

investigating any safety incidents while maintaining appropriate privacy protections through access controls and data retention policies.
Second, enforcing privacy preferences in the proximity matching system would ensure users who opt out of discovery are properly excluded from nearby buddy searches. This would integrate the existing privacy controls more deeply into the matching logic.
Third, adding advanced matching options would improve the quality of connections. Users could filter potential buddies by experience level (beginner, intermediate, advanced), preferred travel pace, or specific safety priorities. This would help ensure better compatibility between matched travellers.
Finally, implementing in-app messaging or coordination features would allow matched buddies to communicate about meeting points, timing adjustments, or route changes without leaving the SafePath platform. This would make the coordination process smoother and keep all safety-related communication in one place.

User Authentication & Privacy

It uses a secure and privacy-friendly account system with bcrypt for password storage and JWT-managed sessions with mandatory email confirmation and GDPR-compliant management of location data. Location data gets stored with short-lived client-side cache storage and gets updated on the server side only when the user accesses location-based services such as FindBuddy functionality.

Authentication And Account Lifecycle

Sign up (account creation with verification)

User signup validates user input (name, email, password, and, if present, coordinates), searches for existing email addresses, and uses bcrypt (10 rounds of salt hashing) on the password. Simultaneously, a secure email verification code is generated on the server side using crypto.randomBytes(32) that is then sent via email using Nodemailer (Gmail SMTP) for the user. A user account will be inactive (isVerified=false) prior to email verification.

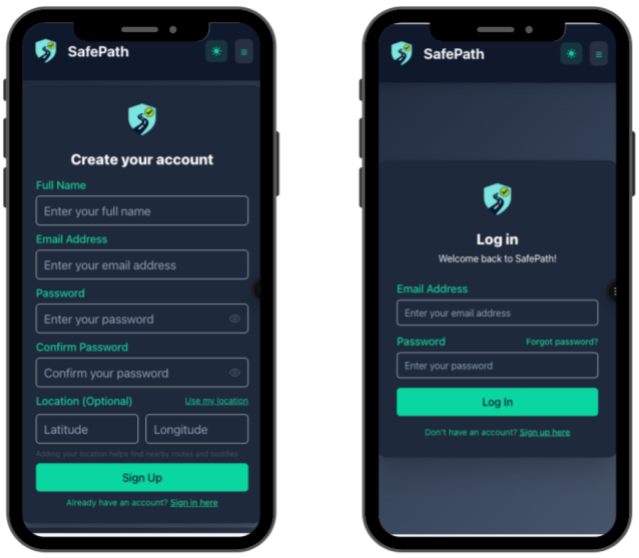

Figure 62 - Sign Up / Login

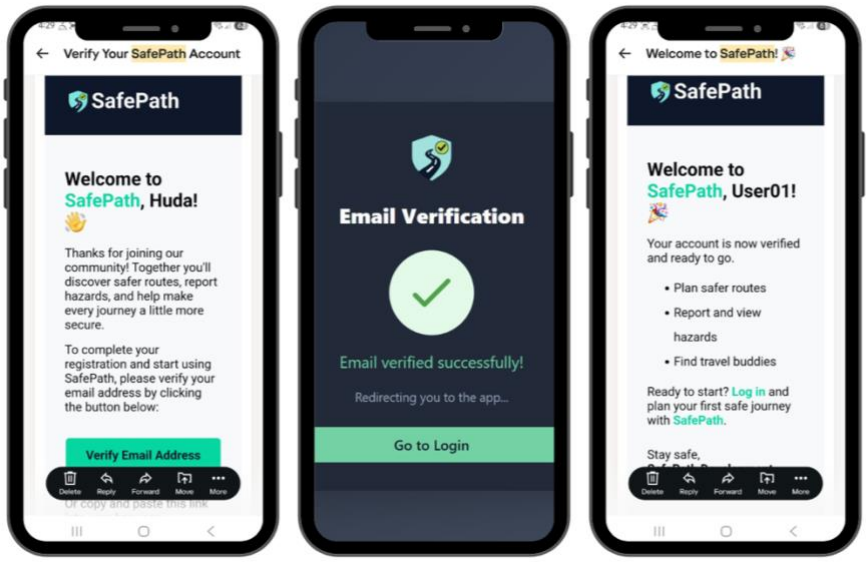

Figure 63 - Signup Email Verification

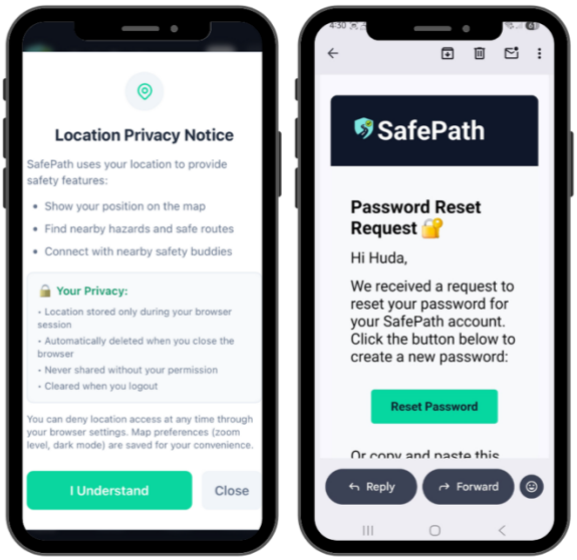

Figure 64 - Privacy Notice & Password Reset Notification

Email verification (account activation)

When a user engages with a verification link, SafePath checks the following: token exists; token is valid; token has not expired; the token belongs to a user that is already verified. When all the above are true, the is_verified flag will be set to True, and the token information will be emptied to avoid token reuse.

Login (only for authenticated users)

On login, SafePath looks up the user by email address and checks his password using bcrypt.compare() (safe comparison). Login will fail if isVerified === false. For verified users, it generates a JWT that expires in 24 hours, emitting claims that are minimal for identity (e.g., userId, email).

Protected Routes (JWT Required)

Sensitive endpoints (FindBuddy, location updates, buddy requests) are protected by an authenticateToken middleware. authenticateToken extracts a bearer token, confirms it using a shared secret, then injects req.user (e.g., { userId, email }) into the request.

Location Privacy Model (Client and Server)

The sensitive data treated by SafePath include the GPS location, while non-sensitive UI preferences are handled separately.

Client side (GDPR-aware)

The current position is stored in sessionStorage with a timestamp and automatic expiry (for example, 30 minutes). The fact that sessionStorage is cleared when the browser is closed reduces the risk of storage. Map view state that is not sensitive (for example, center, zoom, and others) could persist in localStorage.

Server-side (Location usage tied to feature activity)

When the user interacts with FindBuddy, the frontend saves the current GPS location into the server as a PostGIS GEOGRAPHY(Point, 4326) field (meter-friendly distance calculations), in the correct order of coordinates ST_Point(lon, lat). The backend then carries out a search using a radius on ST_DWithin(.) and sorts the resulting objects on ST_Distance(.).

Security Measures:

- No passwords are ever stored in plaintext; only bcrypt hashes.
- The payloads in JWTs are small and do not include passwords and sensitive profile data.
- The email verification links are random and temporary, thus resisting account takeover through stale and leaky links.
- Validation of inputs is handled by express-validator, and SQL queries are parameterized to prevent injections.
- Data storage is time bounded on the client side, and the server is dependent on feature engagement. Corrections and Gaps to Report (critical observations)
- The FindBuddy service does not currently enforce privacy preferences in the query.
- The "/buddies/nearby" endpoint filters on location IS NOT NULL and distance only and does not currently exclude those who are privacy opted out, which is a considered as an app limitations problem.
- The "Location never sent to server unless user explicitly shares" wording should be precise.

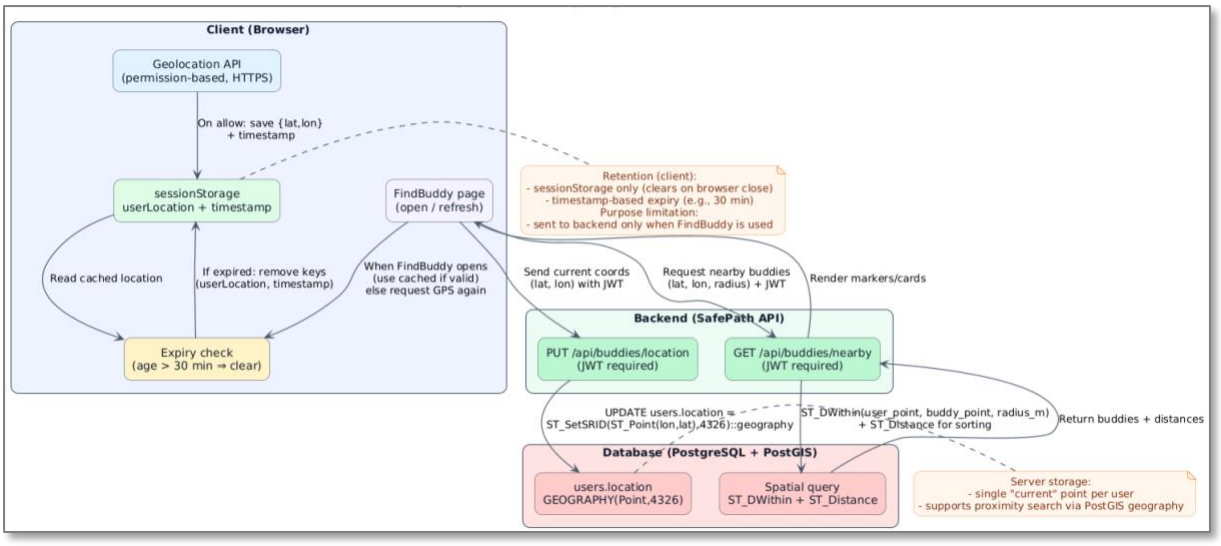

Figure 65 - Location privacy data flow & retention

In SafePath implementation, FindBuddy usage implies the presence of a persisted location feature. However, this should not have a background sharing option. A precise statement for this is: “Location is only persisted server-side while using location-dependent features (FindBuddy), not continuously in the background.” - Naming conventions in coordination: Naming conventions for the backend are PUT /api/buddies/location with parameters of lat, lon, while elsewhere it might refer to parameters of latitude, longitude. While presenting, maintain similar naming conventions or what seems appropriate for mapping lat = latitude, lon = longitude.

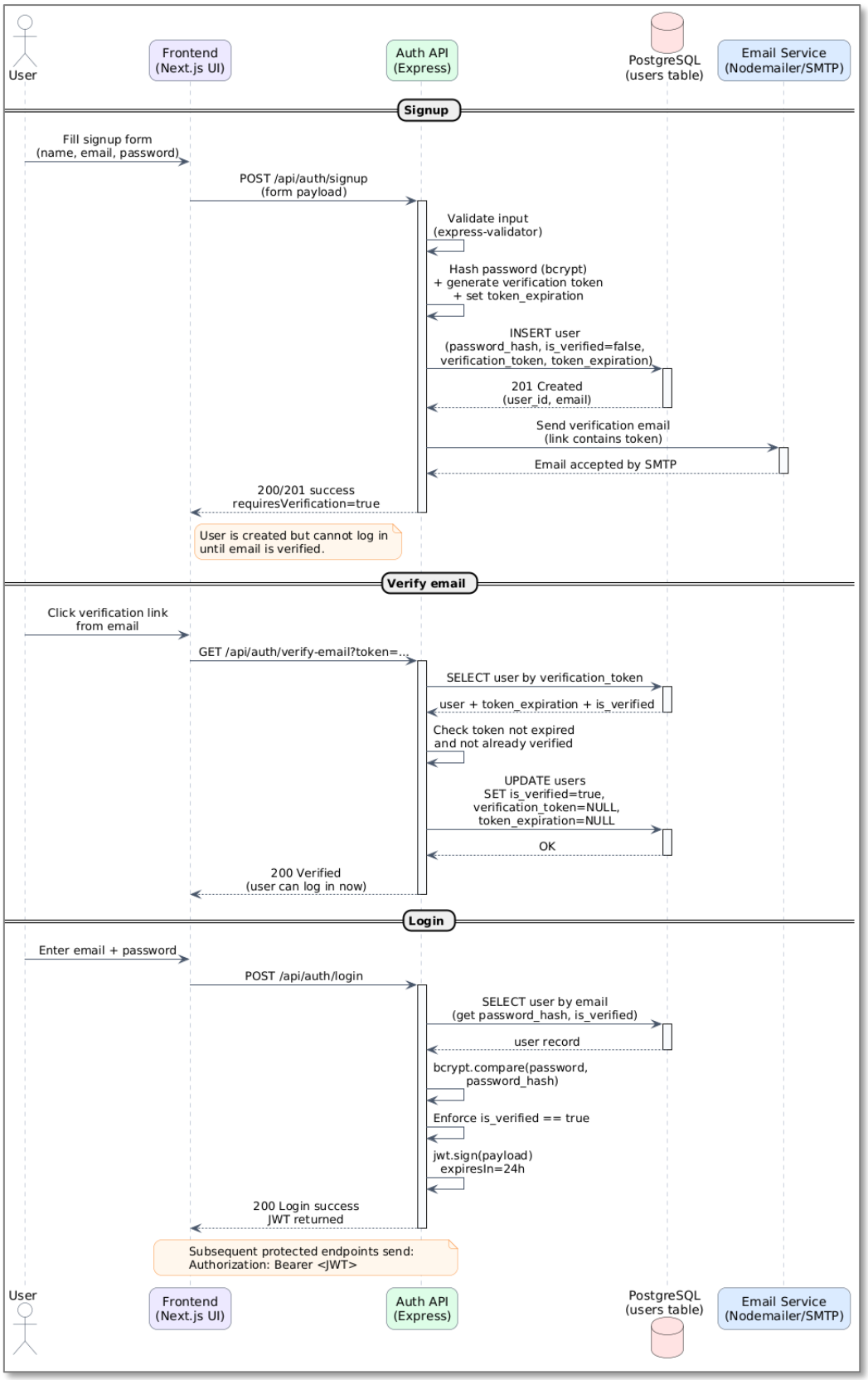

Figure 66 - Signup/ Verify Email/Login (end-to-end sequence)

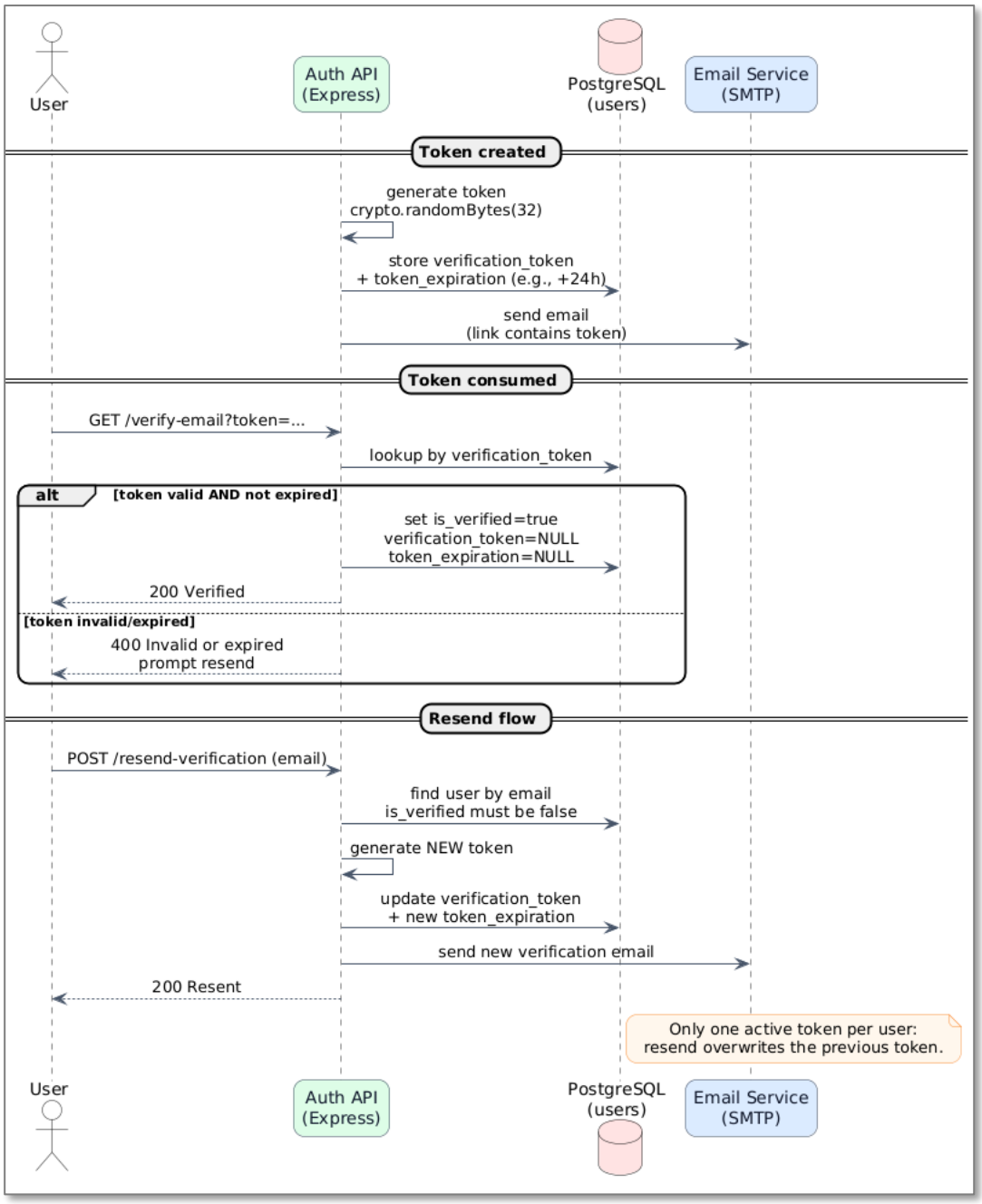

Figure 67 - Email verification token lifecycle

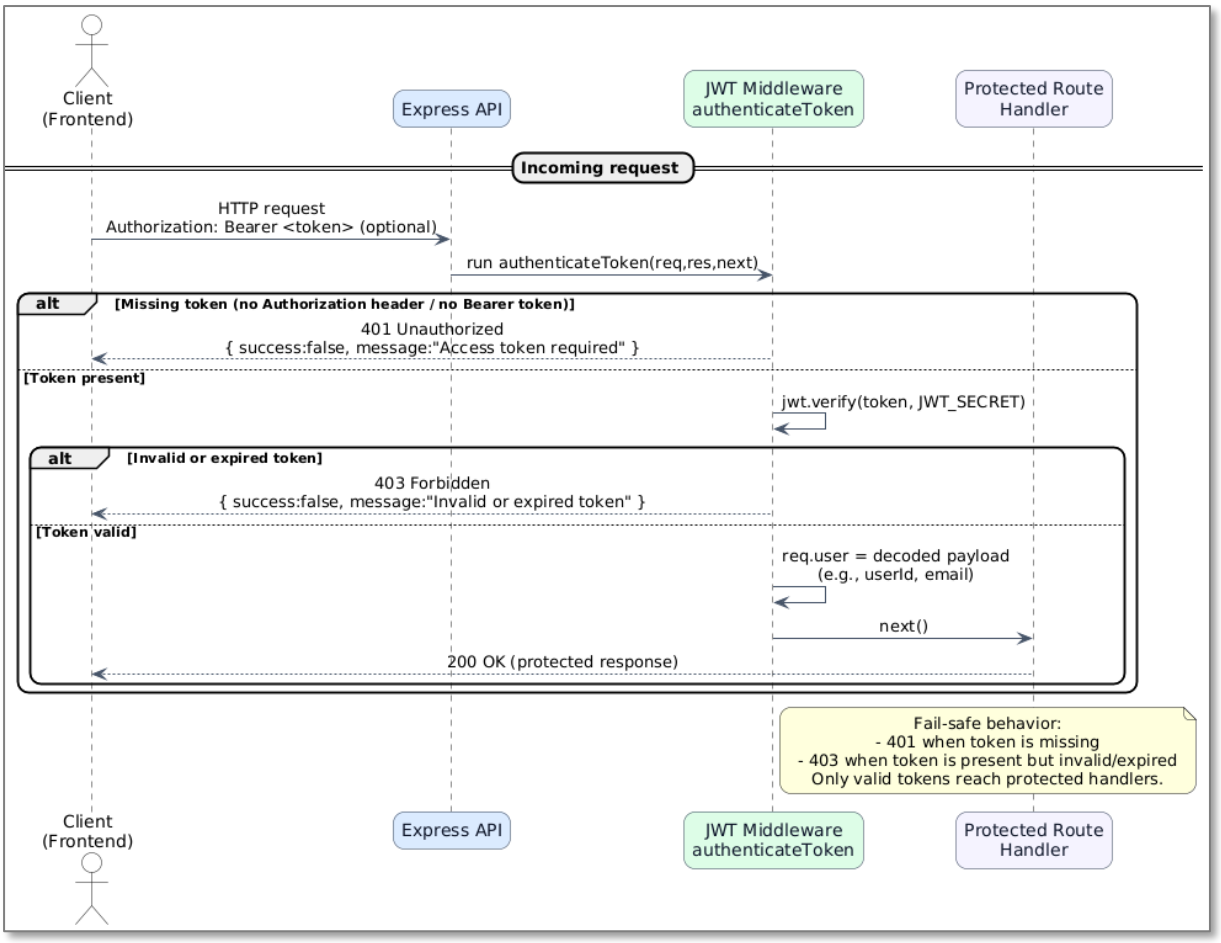

Figure 68 - JWT-protected request flow (middleware gate)

Backend Summary

SafePath's backend is a Node.js + Express API that combines secure auth, spatial data processing, and safety-related routing + hazard services behind JWT-protected endpoints. It uses PostgreSQL + PostGIS as the main datastore so the server can run real geographic queries (meters-based) for features like FindBuddy and location-aware results. Authentication is implemented with bcrypt password hashing, email verification tokens (generated with cryptographically secure randomness, stored with an expiry), and JWT sessions enforced via an authenticateToken middleware that gates protected routes (fail-safe 401 for missing token, 403 for invalid/expired token). For FindBuddy specifically, the backend stores the user's current point in users.location as GEOGRAPHY(Point,4326) and returns nearby users using ST_DWithin (radius filter) + ST_Distance (sort by nearest), while buddy connections are managed through a buddy_requests table supporting pending/accepted/rejected transitions plus group-route helpers (create/list/join) and "along-route" discovery using a simple line corridor with ST_MakeLine + ST_DWithin. In the wider system, the backend also acts as an integration layer for external safety inputs (e.g., incidents/hazards and map-derived context) and can broadcast live updates through WebSockets (Socket.IO) where applicable, while keeping client privacy controls central by only persisting location when the user actively uses features that require it.

Data Sources

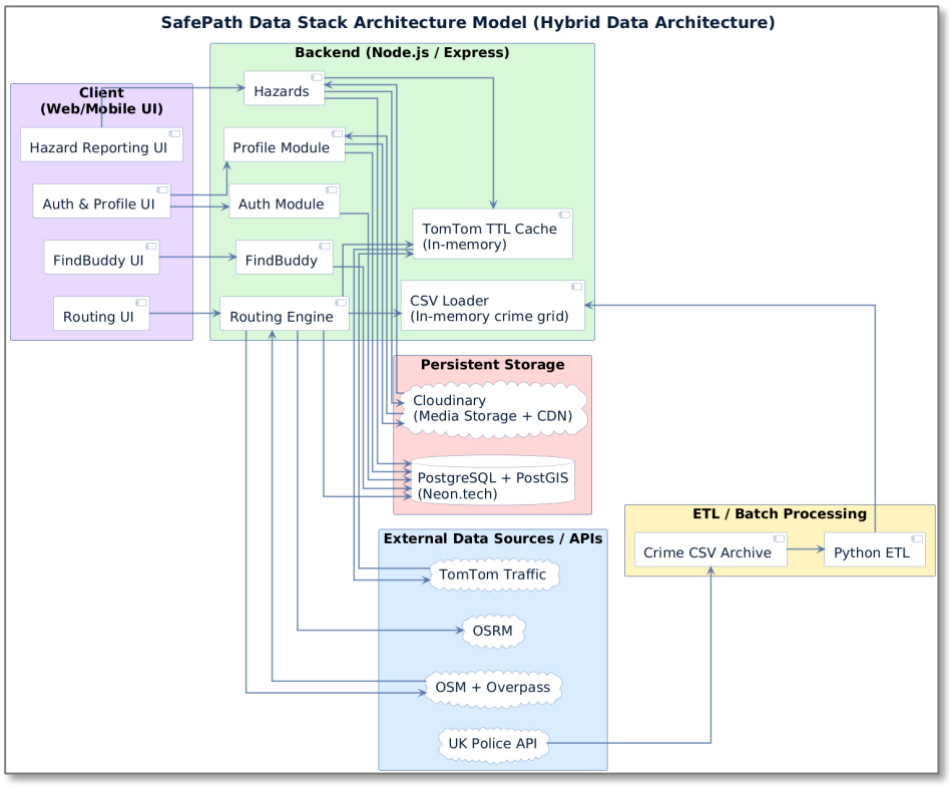

Figure 69 - SafePath Data Stack

Data Sourcing and Collection

User & Profile Data

SafePath stores user accounts and profile fields in PostgreSQL because these records are transactional and must remain consistent for login, email verification, and password reset flows. User location is stored using PostGIS geography types to support accurate “nearby” queries, and user safety preferences persisted in a dedicated 1:1 table so routing can reliably apply the user’s chosen weights across sessions. Users also provide profile details (username, email, etc.) and may upload a profile photo; when a profile image is uploaded, the backend sends the file to the Cloudinary Upload API and receives a secure hosted URL, then stores only this URL reference in the user profile record (e.g., users.preferences JSONB). This allows the UI to render images via Cloudinary’s CDN without storing image files in PostgreSQL. Key bottlenecks include security/correctness (duplicates, token handling) and proximity lookups at scale, addressed through unique constraints, server-side validation, and GiST spatial indexing.

Routing data

SafePath’s routing decisions are driven by a mix of historical and real-time sources, and we intentionally use a hybrid storage strategy to keep route scoring fast. Crime data is collected from the UK Police Data API as monthly bulk CSV downloads, that’s one of the app limitations that we collect crime data only for London city currently , then cleaned and normalized through a Python ETL workflow including de-duplication, coordinate normalization, category grouping, severity weighting, and time-decay so recent incidents influence scoring more strongly. Because scoring samples many points along each route,

storing raw crime points in PostgreSQL would cause heavy read amplification; instead, the cleaned crime CSVs are loaded into server memory and aggregated into grid cells with pre-computed risk values, enabling O(1) lookups during scoring . Route geometry (polyline/legs) is requested on demand from OSRM and does not persist by default. Road network context and lighting signals come from OpenStreetMap, with targeted enrichment queries via the Overpass API (e.g., streetlamps and “lit” tags); since Overpass can be slow and rate-limited, derived lighting indicators are cached in PostgreSQL (e.g., streetlighting) with an expiry window (TTL) to reduce repeated API calls. Finally, collision/traffic incident risk is collected as a volatile real-time signal from TomTom Traffic API during route calculation, because traffic conditions change minute-by-minute, and is typically handled with short-term caching to control latency and API cost.

Hazards Data

SafePath collects community hazards through an in-app reporting flow where users submit hazard type, severity, description, location and timestamp, with an optional photo. Hazard metadata is stored in PostgreSQL and indexed with PostGIS so the system can efficiently run “within radius” queries to show nearby hazards and compute hazard density during routing. Images are **not** stored in the database; instead, uploaded photos are sent to the Cloudinary Upload API, and SafePath stores only the returned secure URL (and optionally an id/public_id) in the hazard record. This keeps the database lightweight while Cloudinary delivers optimized media via CDN. In parallel, external real-time incidents (e.g., accidents/jams) are collected from the TomTom Traffic API (TOMTOM API Doc, 2025) and treated as short-lived signals because freshness matters; these incidents may be cached/expired to prevent stale hazards from influencing routing.

FindBuddy

FindBuddy data is collected through user actions such as sending, accepting, or rejecting buddy requests, and creating or joining group routes. The core buddy logic is first party (no external social API): it is implemented using a lightweight relational model where buddy_requests stores sender/receiver pairs and a status state machine (pending, accepted, rejected). Accepted requests act as the effective “buddy connection,” avoiding the need for separate connection tables. Proximity discovery is computed from users’ stored locations (users.location as a PostGIS geography type), enabling “nearby users” queries and route-based matching. The key bottlenecks are (1) spatial proximity searches and (2) relationship filtering at scale; these are addressed by using GiST spatial indexes on location for fast radius lookups and by indexing buddy_requests fields (e.g., sender_id, receiver_id, status) to keep joins and status filters efficient (PostGIS Doc, 2023b).

Data Storage

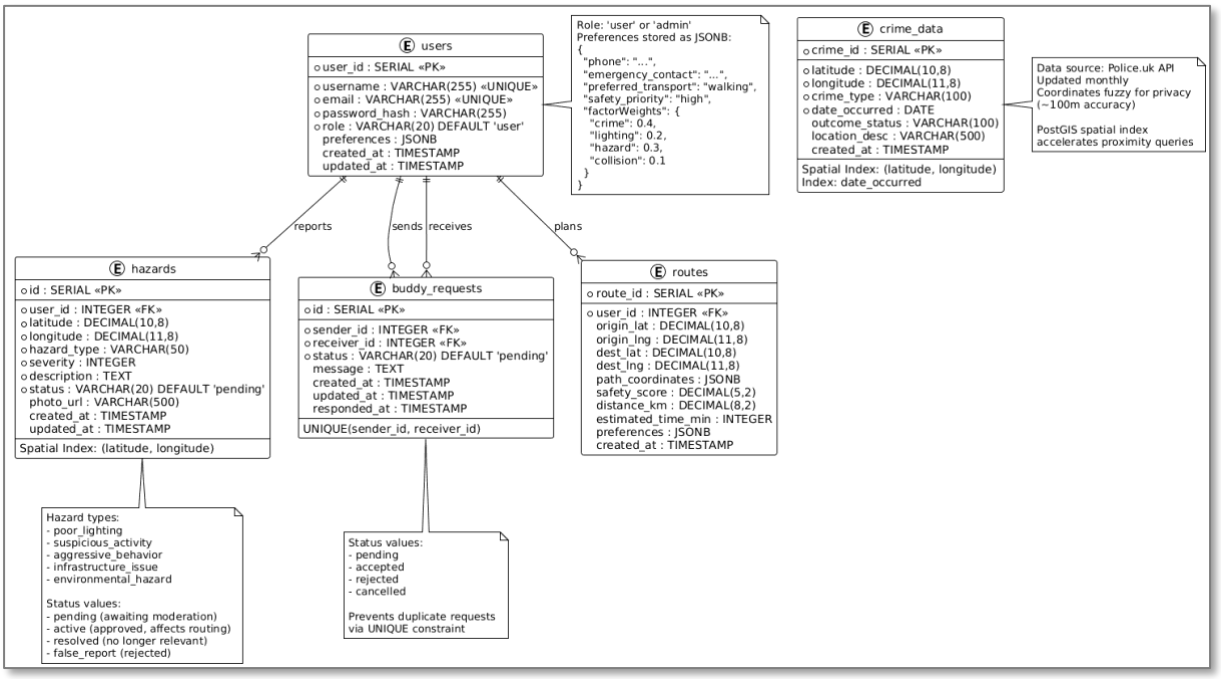

Figure 70 - SafePath ERD Diagram

User Profile Data Storage

SafePath stores account and profile records in PostgreSQL because they are transactional and must remain consistent across authentication workflows (registration, login, verification, and password reset). The users table stores identity fields (username/email), hashed credentials, token fields, and timestamps. For location-aware features, users.location is stored as GEOGRAPHY(POINT, 4326) and indexed with a GiST index to keep proximity searches efficient. Profile images are not stored in PostgreSQL; instead, images are uploaded to Cloudinary and SafePath persists only the secure URL reference (e.g., within users.preferences JSONB). This avoids database bloat and leverages Cloudinary's CDN for optimised delivery.

Routing Data Storage

Routing storage follows a "store what the user owns, compute what is heavy" approach. In PostgreSQL, SafePath persists (1) user preferences in user_safety_preferences (1:1 with users) to apply the same weights across sessions, and (2) saved routes in routes as the user's route library, including start/end locations and optional geometry (GEOMETRY(LINESTRING, 4326)) when saved. Raw crime points are intentionally **not** stored in PostgreSQL because the route-scoring query pattern would cause heavy read amplification (many sampled points per route × many routes). Instead, UK Police crime CSVs are loaded at server startup and aggregated into a ~1km grid stored in memory (JavaScript Map) for constant-time lookups during scoring. Lighting data is also not repeatedly fetched live: Overpass-derived lighting indicators are cached in PostgreSQL (street_lighting) with a 30-day TTL, turning slow external calls into fast local reads. TomTom incidents are treated as volatile; they are protected with short TTL caching (e.g., 15 minutes) and stored/expired as needed rather than treated as long-term historical truth.

Hazards Data Storage

Hazard metadata is stored in PostgreSQL because it must be persistent (for community reports) and efficiently available for enquiry by distance for both map display and route scoring. The hazards table stores hazard type, severity, description, timestamps, status, source, and location as PostGIS geography; GiST indexing supports fast “within radius” queries. Images are stored externally: hazard photos are uploaded to Cloudinary, and only the secure URL is stored in hazards.image_url, keeping the database lightweight while enabling CDN delivery. External incidents collected from TomTom are stored as short-lived hazards (source='tomtom') with expiry logic to prevent stale incidents from influencing routing. The MVP includes verified_count as a placeholder for future community verification, but does not yet implement voting endpoints or UI, so verification effects are explicitly documented as post-MVP work.

FindBuddy Data Storage

FindBuddy is stored in PostgreSQL because buddy relationships and group travel are relational workflows that require consistent state transitions. Buddy requests are stored in buddy_requests as sender/receiver pairs with a status state machine (pending/accepted/rejected), and accepted requests represent the effective buddy connection (simplifying schema by avoiding separate connection tables). Group travel uses group_routes plus the group_route_members bridge table to represent the many-to-many relationship between users and group routes. Proximity discovery relies on users.location stored as PostGIS geography and accelerated by GiST indexing, while relationship filtering is kept fast through indexes on buddy_requests fields (sender_id, receiver_id, status). This design supports scalable discovery and consistent social state while keeping the MVP database small and maintainable.

Data Stack Conclusion

SafePath’s MVP implemented as a justifiable hybrid data architecture focused on safety-first routing on the road network (walking/cycling). Routing geometry is obtained via OSRM endpoints backed by OpenStreetMap street data, while safety scoring combines three implemented data channels: (1) UK Police crime data downloaded monthly as CSV and loaded at server startup into an in-memory, grid-aggregated risk model for constant-time lookups during route sampling; (2) real-time incident/collision signals from the TomTom Traffic API, protected by a 15-minute TTL cache with background refresh to reduce cost and latency; and (3) lighting indicators derived from OSM/Overpass queries, mitigated by a PostgreSQL street_lighting cache table with a 30-day TTL to avoid Overpass rate limits and multi-second response times. User-generated hazards are fully implemented, including photo uploads to Cloudinary and storage of only the returned URL in PostgreSQL, with PostGIS GiST indexes supporting fast proximity queries. FindBuddy is implemented using a lightweight relational model (buddy_requests as a state machine) plus group travel (group_routes, group_route_members) and proximity discovery using users’ PostGIS location field. This scope prioritises performance, correctness, and demonstrable integrations under MVP time constraints, with storage optimisations that are concrete and measurable (crime grid in RAM, Overpass TTL cache, TomTom short TTL cache, and media offloaded to Cloudinary).

Several items were intentionally excluded or simplified to maintain academic integrity and deliver a working MVP within the limited timeframe. SafePath does not implement public transport/transit routing (no GTFS/GTFS-RT feeds or transit APIs), so “public transport maps” is removed from claims and replaced with “road network routing.” Likewise, STATS19 historical collision ingestion is not implemented (no download/ETL/storage pipeline), so collision risk is derived only from TomTom’s live incidents; STATS19 is documented as a

Phase-2 enhancement for long-term collision hot-spot modelling. Hazard “community voting/verification” is only partially supported: the schema includes a verified_count field, but there is no voting endpoint or UI in the current implementation, so it is described as a future enhancement rather than an active scoring mechanism. These planned upgrades are clearly scoped as post-MVP work: adding a formal hazard verification workflow (endpoints + audit trail), integrating historical collision datasets (STATS19) with ETL and spatial indexing, and exploring multimodal routing only if transit integration becomes a project requirement.

Conclusion - Technical Solution

SafePath delivers several meaningful contributions from a technical standpoint. First, it demonstrates how safety can be quantified and integrated directly into routing instead of being visualized merely as points on a map. Most navigation systems treat safety as an afterthought. SafePath proves that risk indicators can be processed, harmonized and transformed into a Safety Score that influences route generation itself. This alone is a shift in the way routing algorithms can work. A significant technical contribution lies in unifying heterogeneous datasets into one scoring framework. Crime incidents, OpenStreetMap roads, collision statistics and real time user hazards originate in entirely different formats. Some come as CSV files, others as API responses or geometric network edges. SafePath transforms them into a common grid model that boosts routing performance. The grid-based approach dramatically reduces computational cost compared to evaluating individual incidents. It balances accuracy with real time responsiveness which is essential for real world use. Real time hazard broadcasting is another impactful contribution. Instead of expecting users to refresh the interface repeatedly, SafePath pushes hazard updates instantly through Socket.IO. This means that as soon as one person reports a broken streetlight, a cyclist approaching that segment could be warned before arriving. This infrastructure also enables future possibilities like live crowd movement, emergency alerts or time-based safety adjustments. Transparency and user trust form another core innovation. Often applications compute internal scores silently without allowing users to question the reasoning. SafePath takes the opposite approach by showing safety breakdowns and explaining the decision making behind route generation. During evaluation users expressed higher trust when understanding why a route was suggested. This demonstrates how data explainability can change behaviour in safety focused design. The platform is also built with forward scalability in mind. Although currently operating for London, the entire system can adapt to a different city by rerunning the ETL process and recreating grid layers. New safety attributes such as air quality risk, flood zones or school areas could be added without rewriting fundamental architecture. A future version might incorporate machine learning to generate predictive scores for routes where live data is unavailable. The existing rule-based engine provides a foundation for training such models. SafePath therefore stands not only as a working product but as a framework for city aware routing. It shows how open datasets can be converted into safety knowledge and how user interaction can strengthen that knowledge over time. It moves routing beyond the fastest path paradigm into a space where navigation feels protective, contextual and human centred.

User Evaluation

Introduction and Purpose

This evaluation phase evaluates whether SafePath's safety-first routing concept is communicated clearly through the interface, and whether users can successfully use safety information during typical walking/cycling journeys. Usability is a quality attribute *that* assesses how easy user interfaces are to use *(Nielsen, 1993)*.

The study is a moderated, task-based usability test using the current SafePath web app prototype. In moderated usability testing, a facilitator asks a participant to complete tasks while observing behaviour and collecting feedback (Moran, 2019). To ensure comparability, all participants used the same SafePath build and completed the tasks using a standardised UK (London) crime dataset, while participants were recruited from both London and Dublin. This design keeps the crime-risk layer consistent for all sessions (controlling map content for the key safety dataset), while still capturing usability feedback from users in different cities; all other SafePath features and workflows remain city-agnostic and are expected to behave similarly across locations.

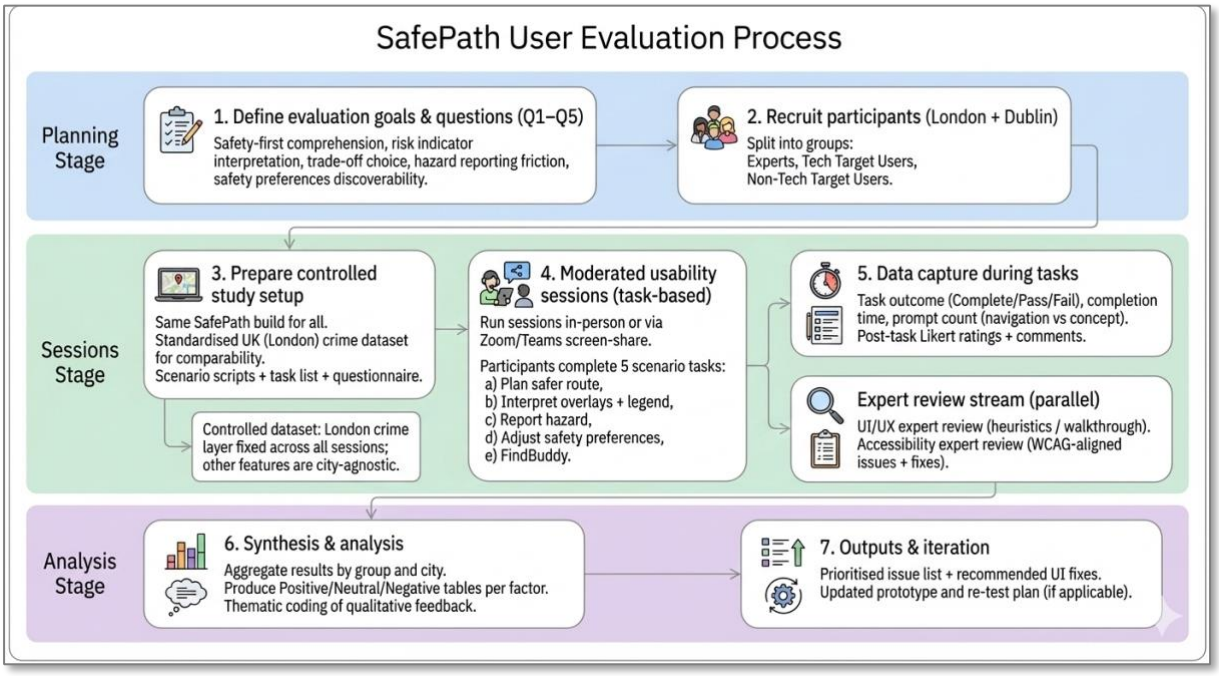

Figure 71 - User Evaluation Process

Components Evaluated and Rationale

This evaluation focuses on the SafePath components that most strongly determine whether users can learn the interface quickly, complete tasks efficiently, avoid critical errors, and trust the safety-first outputs, all core aspects of usability (Nielsen, 1993, 2001).

C1. Route planning "Safest vs Fastest" selection and explanations
SafePath's main value depends on users understanding why a safer route may differ from the fastest route and being able to select and start navigation confidently.

C2. Safety communication layer (map overlays, hazard markers/cards, legend, route colour meanings)
Safety-first routing fails in practice if users cannot interpret risk indicators or do not notice them during route choice. This component is evaluated for learnability and error prevention (misinterpretation) (Nielsen, 1993).

C3. Hazard reporting workflow (capture, pin, details, submit, confirmation)
Because SafePath relies on user reports as a safety signal, the reporting flow must be efficient and low-friction; otherwise contribution drops even if the feature exists (Nielsen, 2012).

C4. FindBuddy workflow and privacy controls (discoverability, consent, "nearby" accuracy)
This is a safety-sensitive, location-based feature; usability and trust depend on clear consent, minimal steps, and accurate "nearby" behaviour (GOV UK, 2017).

C5. Onboarding and authentication (signup, email verification, password recovery expectations)
Account flows are common failure points; evaluating them is necessary because usability applies to the full journey and affects adoption and completion of safety features that require login(Understanding WCAG 2.2, 2024; W3 Org, 2024).

C6. Mobile layout and reachability (map-first flow, route list accessibility, button clarity)
Most routing interactions happen on mobile; layout reachability issues can dominate time-on-task and prevent successful completion even when backend functionality is correct.

C7. Accessibility (readability, spacing, non-colour cues, WCAG-aligned issues)
Accessibility was included to ensure the interface remains usable across different abilities and device conditions, guided by WCAG 2.2 principles and success criteria.

Evaluation Questions

The user evaluation is designed to answer a focused set of questions that test whether SafePath's safety-first intent is understood, actionable, and trustworthy during realistic mobility tasks. These questions are assessed through moderated task completion and follow-up ratings/comments, which is a standard approach in usability testing (Nielsen, 2019).

Core UX questions (all participants)

- **Q1:** Do users understand SafePath is safety-first (not shortest/fastest only)?
- **Q2:** Do users correctly recognise and use risk indicators (overlays, hazard markers, legend, route colour meanings) when choosing a route?
- **Q3:** Will users trade time/distance for safety, and under what conditions (e.g., night travel, travelling alone, trip purpose)?
- **Q4:** Is the hazard reporting flow simple enough that users will contribute data?
- **Q5:** Can users discover, adjust, and understand safety preferences (e.g., lighting vs crime avoidance)?

Additional Questions (by user type)

These capture business and technical realism without forcing every participant to answer technical items:

For target users (business/adoption)

- **Q6:** Would you use SafePath in real life, and what would make you adopt it (or stop using it)?
- **Q7:** What minimum value should be available as a **guest** before login to justify signing up?

For expert reviewers (technical/research/UX)

- **Q8:** Are the information hierarchy, interaction patterns, and safety explanations strong enough to prevent misinterpretation and build trust?
- **Q9:** Are the architecture choices for real-time updates and privacy controls appropriate for a safety/location-based system at MVP scale?

(These “extra questions” are asked selectively during debrief to match the participant’s expertise, while task metrics remain comparable across sessions.)

Evaluation Process and Setup

Participant groups (how we compare results)

To compare outcomes across user types, participants are categorised into three groups:

- **G1: Experts** (mentor/reviewer type; optionally tagged as UX / Technical / Research)
- **G2: Target users – Tech-related**
- **G3: Target users – Non-technical** (some required guidance to learn the UI)

A city tag (**London** / **Dublin**) is recorded separately for context.

Tasks (scenario-driven)

Participants complete five core tasks, each embedded in a short scenario:

1. Plan a safer route between two points.
2. Interpret safety overlays/hazards and use the legend**.**
3. Report a hazard on/near the route.
4. Adjust safety preferences and observe how routes change.
5. Use FindBuddy to find a buddy nearby (location-based safety feature).

What we measure (task metrics + factor ratings)

To keep results comparable, the same tasks and the same rating factors are collected from all participants.

A) Task performance (objective)
For each task we record:

- **Task outcome:** Complete / Pass / Fail
  - Complete = completed independently (1.0)
  - Pass = completed with facilitator help (0.5)
  - Fail = not completed (0.0)
- **Completion time**
- **Prompts count** (number of interventions), with prompt type noted:
  - Navigation hint (where to click)
  - Concept explanation (e.g., "why is this route safer?")

B) Post-task feedback (subjective)
After tasks, participants provide:

- Short Likert ratings (e.g., clarity of safety info, trust in the suggested route, usefulness of reporting)
- Brief comments (what was confusing, what felt valuable, what should change)

C) Factor ratings (for your Positive/Neutral/Negative table)
Each participant also provides ratings for the same factors:

- UI
- Functionality
- Usability
- Color scheme
- Performance
- Timing

These are recorded as **Positive / Neutral / Negative** (or 1–5 then converted).

Participant Data and Results

Participant Data

Non-Expert User Evaluation

P1 (Mai, Dublin)

Participant profile. (Mai) is a Dublin-based daily walker and digital marketer. The session was conducted in-person with a moderated walkthrough.

Task performance. Mai completed the core scenario task **"**Plan a safer route and start navigation**"** successfully. She selected origin/destination easily, adjusted the safety preference controls, and initiated navigation. Total time to complete the flow was under 2 minutes**.** She initially found the meaning of some map colours unclear, but after brief guidance she proceeded confidently and completed the task without further difficulty.

Safety-first comprehension (controlled hazard scenario). To test whether SafePath's safety-first logic was understood (rather than being perceived as a routing error), a deliberate high-risk hazard was placed in the area of her intended route. When SafePath suggested a route that was slightly longer, Mai correctly interpreted the detour as an intentional safety decision and explained that the extra metres were acceptable because it

avoided the high-risk area. This supports Q1/Q3 by showing that users can understand and accept the “safer route may be longer” trade-off when risk cues are visible.

Qualitative feedback. Mai’s feedback was mainly UI/interaction-focused:

- Requested **“**show password**”** on the signup screen to reduce friction during account creation.
- Praised the dark mode as modern and visually appealing.
- Suggested increasing the map area on mobile and improving spacing/padding so the suggested routes list under the map is easier to view and interact with.

Impact on design decisions: Based on this session, three improvements were prioritised:

1. add a show-password toggle on signup to improve usability and reduce user error during registration.
2. adjust the mobile map + routes layout (more map viewport space and clearer padding/scroll behaviour) to improve route comparison;
3. strengthen the visibility of safety cues via a clearer legend/onboarding hint, since initial colour interpretation required guidance.

P2 (Esraa, Dublin)

Participant profile. P2 (Esraa) is a Dublin-based parent and PE teacher who regularly walks for school drop-off and local trips. The session was conducted in-person. She is comfortable with everyday apps but does not have a technical background.

Task performance. Esraa required guidance at the start to understand where to begin in the interface and how to report a hazard. After initial instruction, she successfully completed two core tasks: (1) report a hazard and (2) plan a route and start navigation. However, the overall time to complete these tasks and begin navigation was over 6 minutes, indicating higher friction than expected for a routine walking use case.

Observed breakdown: “Fastest = Safest” confusion (Q1/Q3/Q4).
During route planning, Esraa observed that the fastest route and safest route were identical**,** which caused confusion and reduced confidence in the safety-first concept. The moderator clarified that her reported hazard was a low-risk type**,** and the current routing behaviour primarily applies detours for high to critical hazards. This interaction highlights a UX gap: when SafePath does *not* alter a route, users still need a clear explanation of *why* (e.g., “no high-risk factors detected on this corridor” or “reported hazard severity is low, route unchanged”).

Qualitative feedback.

- Positive reactions to brand and visuals: liked the colours**,** logo**,** app name**,** and dark mode.
- Main usability concern: she needed more guidance and did not find the steps intuitive; she asked early questions about where to start and how hazard reporting works.

Impact on design decisions: Based on Esraa's session, the following changes are prioritised:

1. Add onboarding / first-run guidance (short tutorial or tooltips): "Start here", "How to report a hazard", "How to read safety layers."

2. Make hazard severity and routing impact explicit in the UI (especially after reporting):

    - show the hazard's **severity label** ("Low / Medium / High / Critical")

    - display a short message such as: **"**Low-risk hazard recorded; route may not change unless higher-risk hazards are present."

3. Improve route explanation panel to handle the "no detour" case, so users can understand that "fastest = safest" is a valid outcome rather than a failure.

Finally, Esraa liked the visual design and branding, but her session revealed important usability issues around onboarding and explainability. Her confusion when the route did not change provides evidence that SafePath needs clearer messaging about hazard severity and when safety-based rerouting will occur.

P3 (Mohamed, Dublin) - Tech-related

Participant profile. P3 (Mohamed) is a Dublin-based Database Support Engineer and a high-tech user. He cycles as a hobby and represents a tech-literate target user who can navigate new systems quickly and is sensitive to data correctness.

Task performance. Mohamed completed a broader set of tasks than earlier participants, including account creation**,** email verification**,** FindBuddy request flow**,** and route planning from the FindBuddy page. Overall, he completed the end-to-end tasks in approximately 4 minutes on average, indicating strong efficiency once he understood the workflow.

Key observation: location handling revealed a critical FindBuddy bug (Functionality / reliability).
Unlike other participants, Mohamed entered his current location during sign-up. During FindBuddy testing with a project team member, the session exposed a critical failure case: if a user does not provide a location at sign-up, the location field remains null, which led to invalid distance calculations between buddies. This test directly resulted in a functional fix: SafePath was updated so that location is captured and stored explicitly at the points where it is required (e.g., during route planning and within the FindBuddy flow), rather than relying solely on optional sign-up data. This change improved system robustness and prevented invalid or misleading proximity results.

Safety cues and routing interpretation (Q2).
Mohamed reported that the overall app colours are meaningful, but he still commented on the routing colour meanings and suggested clearer communication of what each route colour represents (e.g., safest vs fastest and/or risk level cues).

Workflow friction (Timing / usability).
He noted that the FindBuddy flow contains too many steps, and recommended reducing steps to support quick use in real walking/cycling contexts (where users may be on the move and less willing to navigate multi-step screens).

Impact on design decisions. Based on Mohamed's evaluation, the following changes were prioritised:

1. Fix location-null reliability issue (critical functionality fix): capture location at route and buddy flows, not only during sign-up, preventing invalid distance computations and improving FindBuddy trustworthiness.

2. Clarify routing colour semantics: improve legend/labels so route colour meaning is obvious without interpretation.

3. Streamline FindBuddy steps: reduce interaction steps (e.g., fewer screens, clearer primary action) to improve timing and real-world usability for cyclists/walkers.

Finally, Mohamed's session validated the main flows for a tech user and surfaced a critical edge-case bug that directly improved SafePath's FindBuddy reliability. His feedback also identified two UX improvements: clearer route colour meanings and a simplified FindBuddy workflow.

P4 (Nazim, London, remote) - Tech User Evaluation

Participant profile. P4 (Nazim) is an IT professional (expert tech user). The session was conducted remotely via Zoom. Nazim tested from London, which matches SafePath's controlled dataset context (London crime layer).

Session setup (remote adaptation). Because the session was remote, a short high-level explanation of the application was provided at the start to reduce the risk of "remote onboarding friction" impacting results (e.g., participants getting stuck due to screen-sharing limitations rather than the UI itself).

Task performance. Nazim registered and moved directly to the Plan Route feature. He completed the route-planning task successfully in approximately 4 minutes (including registration). He intentionally selected an area with real-time hazards during a rush-time context, and he was able to distinguish between Fastest vs Safest routes clearly without adjusting safety scoring controls. This supports the clarity of SafePath's route alternatives and indicates that the "safety-first vs fastest" concept can be understood through route outcomes and map cues alone in hazard-dense contexts.

Qualitative feedback and issues raised.

- **Navigation UX:** He questioned why navigation does not remain on the same page as route planning (suggesting a more seamless transition, especially for mobile use).

- **Account recovery:** Requested a Forgot Password feature, highlighting an expectation aligned with mainstream apps and a trust/usability requirement for real users.

- **Performance:** He reported the app felt slow when retrieving data, indicating a performance bottleneck impacting perceived quality.

Impact on design decisions: Based on Nazim's session, the following improvements were prioritised:

1. **Improve navigation continuity:** explore keeping navigation in the same route-planning context (e.g., collapsible bottom sheet, persistent map view) to reduce page switching and cognitive load.

2. **Add password recovery ("Forgot Password"):** necessary for production readiness and improves user trust and usability for account-based flows.

3. **Address perceived slowness:** investigate and optimise data retrieval latency (e.g., reduce API calls, caching, lazy-loading overlays, database indexing where relevant). This aligns with usability evidence that system responsiveness affects user satisfaction and trust (Nielsen, 1993).

Finally, (P4). Nazim successfully completed route planning and understood the safest–fastest distinction quickly, even without adjusting safety controls. His feedback highlighted three priority product upgrades: smoother navigation flow, account recovery, and performance optimisation.

P7 (Alina, London, remote) – Non – Tech User

Participant profile. P7 (Alina) is a student in London (LSE). The session was conducted remotely via Facebook Messenger video call. She is not a technical user, but she is familiar with mainstream navigation/routing apps and relies on walking/public transport in daily routines.

Tasks completed. Alina completed a hazard-focused workflow end-to-end:

1. **Sign-up and email verification**

2. **Report a roadworks hazard**: captured a photo, selected the hazard location on the map, entered hazard details, uploaded the image, and submitted

3. **Verify submission outcome**: confirmed the hazard appeared on the map and viewed the hazard card with its details

Task performance evidence.

- **Outcome:** Completed successfully without major functional failures

- **Time:** > 3 minutes for the full reporting flow (from report initiation to viewing the hazard card)

Qualitative feedback (key themes).

1. **Workflow length / page switching (Usability + Timing).**
   Alina perceived the flow as involving “many pages” and suggested a single-page, map-first interaction model similar to Google Maps, where core tasks happen without leaving the main map context. This indicates that page transitions create perceived friction, especially for fast “in-the-moment” reporting.

2. **Reporting text friction (Q4).**
   She recommended adding auto-complete / suggested text for the hazard description field because it is required and takes effort to write, particularly on mobile. This feedback points to avoidable input burden during reporting.

Impact on design decisions. Based on P7’s session, two improvements are prioritised:

- **Reduce reporting friction:** add quick-pick templates or auto-suggestions for hazard descriptions (e.g., “Roadworks blocking footpath”, “Lane closed”, “Temporary diversion”), and/or make description optional when a hazard type + photo are provided.

- **Minimise page switching:** redesign key flows (reporting and route planning) using a map-first, single-page pattern (e.g., bottom sheet panels) so users can complete tasks without losing context.

Finally, (P7) Alina successfully completed hazard reporting and confirmed the hazard display outcome, supporting functional validity of the reporting feature. Her feedback highlights usability improvements needed for real-world adoption: reduce steps/page switching and reduce typing effort in the report hazard flow.

P9 (Aref, Dublin, Just Eat delivery rider) - Target User

Participant profile: P9 (Aref) is a Dublin-based Just Eat food delivery rider who relies on mobile navigation for frequent, time-sensitive trips between customer locations and restaurant pickup points. The session was conducted in person as a short opportunistic interview during a delivery drop-off. Aref represents a high-mobility target user who typically prioritises speed, low interaction effort, and clear route guidance while on the move.

Tasks completed: Because this was a short, real-world interaction during a delivery drop-off, Aref was asked to choose a single task. After a brief walkthrough of the app, he selected Task 1: Plan a route**.**

Task performance (Plan route + start navigation)

Aref attempted to plan a route from the delivery location to his next pickup location (restaurant). He encountered difficulty early in the flow, mainly when adjusting the safety preference scores and most importantly accessing the suggested routes list after pressing “Plan Route.” On his mobile screen, the route-selection panel was not easily reachable due to a layout/reachability issue (content appearing out of view / hard to scroll to). With facilitator guidance and a quick UI/layout adjustment, he was able to reach the routes list, select a route, and start navigation successfully.

The total time recorded (>15 minutes) was therefore driven primarily by UI reachability and interaction struggle**,** rather than routing computation time, which is not acceptable for a delivery-style use case.

Safety-first comprehension outcome (Q1/Q3)

In this scenario, Aref did not observe a meaningful difference between route options (fastest vs safest), because there were no visible risk factors in the chosen corridor that produced a detour. Without a clear explanation of “why routes are the same,” he did not recognise SafePath’s value proposition and perceived it as similar to conventional navigation apps.

Qualitative feedback and observed issues

- **Usability / Timing:** struggled with safety score controls and required sustained guidance.
- **Mobile responsiveness:** route results were not immediately reachable after “Plan Route,” indicating a mobile layout/scroll issue.
- **Value clarity:** when no detour occurs, users need explicit messaging to understand that the system still evaluated safety.

Impact on design decisions

This participant highlights three high-priority changes:

1. **Fix mobile layout reachability for route results**
   Ensure the suggested routes panel is always reachable on small screens (stable bottom sheet, guaranteed scroll, no hidden overflow). This directly improves Usability and Timing.
2. **Simplify safety preference controls for fast-paced users**
   Add quick presets (Fastest / Balanced / Safest) and keep sliders optional, reducing setup effort for users who need immediate routing.
3. **Add "no difference" explanations to communicate value**
   When fastest and safest are identical, display a clear message such as:
   "Fastest and safest are the same here — no high-risk factors detected on this corridor."
   This supports Q1/Q3 by preventing users from assuming the app has "no purpose" in low-risk scenarios.

Finally,(P9) Aref's session revealed a critical mobile usability risk: on small screens, route results can be difficult to access after "Plan Route," causing excessive completion time even when routing functionality itself works. It also showed that SafePath must explicitly explain outcomes when route options do not diverge, otherwise users may not understand the safety-first purpose.

P10 (Yassin TY student cyclist, Dublin) - Target User Evaluation

Participant profile. P10 is a Transition Year (TY) secondary school student who cycles daily for school and local trips. The session was conducted informally with the project team present. P10 represents a younger daily cyclist who values quick, simple interactions and clear guidance.

Tasks completed. P10 tested FindBuddy with the team and then attempted Plan Route from within the FindBuddy page**.**

Task performance.

- **FindBuddy:** He was able to complete the FindBuddy task with team support (opening the feature and participating in the buddy interaction flow).

- **Plan Route inside FindBuddy:** He found the embedded route-planning flow **difficult to use** and struggled to progress through the steps without guidance. He explicitly asked for this part to be more straightforward in future.

Qualitative feedback (key theme).

- **Workflow complexity (Usability/Timing):** The main issue was not the idea of FindBuddy itself, but the *extra steps and unclear progression* when trying to plan a route from the FindBuddy context.

Impact on design decisions.

1. **Simplify "Plan Route in FindBuddy"** by reducing steps and making the next action obvious (single primary button, clearer labels, fewer screens).

2. **Keep map context consistent** (e.g., bottom sheet route planner within the same map view) so users do not feel lost when switching between buddy and routing actions.

3. **Add a short guided hint** the first time a user tries "Plan Route" from FindBuddy (especially helpful for younger/non-expert users).

Finally, P10 could engage with FindBuddy in a supported setting, but the “Plan Route” flow inside FindBuddy was confusing and required guidance. This suggests the FindBuddy Route transition should be redesigned to be shorter, clearer, and more map-first.

P11 (Sarah, Dublin, non-technical daily walker) - Target User

Participant profile. P11 (Sarah) is a neighbour of a project team member and a daily walker with non-technical experience. The session was conducted in-person while walking together, focusing mainly on the **FindBuddy** feature.

Tasks completed (FindBuddy end-to-end).

Sarah completed the core FindBuddy workflow:

1. **Register + login**
2. Open **FindBuddy** and view nearby users on the map
3. **Send a buddy request** to a nearby team member and receive acceptance
4. Open the **Buddies tab** (side panel) and **create a walking group** (“Friend Walk”)
5. Invite the team member to the group and confirm acceptance on the other device

Observed issues and edge case.

During the session, an edge-case behaviour was observed on the **team member’s device**: the team member had previously tested FindBuddy with another user in a different area, and those “nearby” results still appeared even though the users were no longer in the same current location. Although Sarah’s own flow completed up to joining the group, this observation suggests a potential issue with **presence/recency filtering** (e.g., stale location/discoverability state), which could reduce trust in the accuracy of “nearby” visibility.

Qualitative feedback.

- **Workflow complexity:** requested **fewer steps** to complete buddy/group actions.
- **Mobile clarity:** some buttons were not visually clear on mobile, indicating a need to improve spacing, labelling, and reachability.
- **Feature expectation:** asked for a **chatting facility** to coordinate walking once a buddy/group is created.
- **Overall perception:** liked the idea and found the feature valuable for walking safety.

Impact on design decisions.

- **High-severity bug fix: “nearby” presence/recency accuracy**
  A critical trust issue was identified: on the **team member’s device**, users from a previous test in a different area still appeared as “nearby.” This requires a **quick fix** by enforcing recency rules (e.g., last-updated timestamp), clearing stale sessions, and ensuring ST_DWithin queries are executed only against **current** locations/presence state. This is high priority because inaccurate “nearby” visibility can mislead users and reduce trust in FindBuddy.

- **Reduce steps in FindBuddy flows**
  Simplify request → accept → group creation into fewer actions/screens to support non-technical walkers.

- **Improve mobile UI clarity for FindBuddy**
  Increase button sizes, improve labels/icons, and ensure primary actions are always reachable on mobile.

- **Chat as a future enhancement (with privacy safeguards)**
  Capture lightweight messaging (or predefined quick messages) as a future enhancement to support coordination after buddy/group creation.

Finally, (P11). Sarah successfully completed buddy discovery, request/acceptance, and group creation, demonstrating that the FindBuddy concept is usable for non-technical walkers with guidance. However, she highlighted two key areas for improvement: (1) reducing steps and improving mobile clarity, and (2) ensuring the "nearby" concept is trustworthy through better presence/recency handling.

P12 (Project Manager, Dublin) - Target User Feedback

Participant profile. P12 is a project manager who walks 1–2 times per week. This interaction was a discussion/interview about the overall concept and product flow rather than a controlled usability testing session.

Type of evidence. Because no tasks were executed, this input is reported as qualitative product feedback (concept validation + adoption suggestions), not as task success/time metrics.

Key feedback (adoption and onboarding).

- P12 liked the overall app idea and suggested that some functionality should be accessible to guest users before login, so potential users can understand how SafePath works and see value before committing to registration.

Changes implemented based on this feedback.
To support "try before login" and reduce early drop-off, the following guest-visible elements were introduced:

- **Homepage features available to guests**, including:
  - viewing nearby hazards (real-time and/or community-reported where appropriate),
  - basic route search / planning entry point,
  - clear walk / cycle mode selection.
- An About page describing the main SafePath features and how to get started, providing a quick guide for first-time or guest users.

Impact on product direction.
This feedback influenced SafePath's onboarding strategy by introducing progressive engagement**:** users can preview core value (safety context + routing entry) before creating an account, while keeping privacy-sensitive features (e.g., FindBuddy details, personalised preferences) behind login.

Finally, (P12). Although not a test session, P12's feedback directly improved early user adoption design by making SafePath more understandable and useful to guest users through a more informative and functional entry experience.

P13 (Anika, Dublin, MSc colleague) - Target User

Participant profile. P13 (Anika) is a classmate from the MSc cohort who lives near the college. She participated as a target user in an in-person/close-context test focused on FindBuddy**.**

Tasks attempted.

Anika attempted the FindBuddy workflow (discover buddies, view map/list, create a route group, add a buddy, and proceed to route joining/coordination).

Bugs identified during testing (high impact)

1. **FindBuddy "View Map" button not functioning**
    - **Observed behaviour:** The **"View Map"** button in FindBuddy did not work as intended.
    - **User impact:** Users cannot visualise buddy locations in the expected geographic context and must rely only on list view/distance indicators.
    - **Why it matters:** This is a **core interaction** for a location-based safety feature; failure reduces usability and trust.
2. **Automatic group enrolment bypassing consent ("Join Route")**
    - **Observed behaviour:** When a user creates a routing group and adds an accepted buddy, the buddy is **automatically enrolled** without explicitly clicking **"Join Route."**
    - **User impact:** This bypasses the intended consent workflow, creates confusion, and introduces privacy/trust risks (users may feel "added" without agreeing).
    - **Why it matters:** For a safety/social feature, explicit consent is essential; inconsistent membership behaviour is a **high-severity issue**.
    - These issues were documented during late-stage testing and should be resolved before production deployment.

Qualitative feedback

Anika repeated a recurring issue: the FindBuddy workflow has too many steps and should be simplified to support quick use.

Impact on design decisions (what to change)

- **Fix "View Map" (priority bug fix):** restore expected map visualisation and ensure the button navigates correctly with a clear state (loading/empty/error).
- **Fix consent workflow (priority bug fix):** require explicit acceptance for group routing (buddy must confirm "Join Route" before being enrolled).
- **Reduce FindBuddy steps:** streamline discovery, request, accept, route/group into fewer screens and clearer primary actions (map-first bottom sheet pattern).

Finally, (P13) Anika's FindBuddy test revealed two high-severity functional bugs (broken "View Map" and consent bypass via auto-enrolment) and reinforced the need to reduce steps in the buddy workflow. These findings directly inform the "critical fixes before release" list for SafePath.

P14 (Omar, Dublin, Operations Supervisor) - Target User Feedback

Participant profile. P14 (Omar) is a Dublin-based Operations Supervisor who travels across the city for work and uses navigation apps regularly. This was an informal feedback session (walkthrough + discussion), not a timed task-based usability test.

Topics discussed. Omar focused on the routing logic and safety reasoning, especially whether SafePath adapts to real-world travel context.

Key issue raised: missing time-based safety factors (later fixed).
Omar noted that SafePath's early safety scoring appeared static and should adapt to time context. He highlighted missing considerations such as:

- time-of-day effects (e.g., night, higher lighting importance),
- rush-hour patterns,
- seasonal changes (daylight variation),
- day/night differences in risk patterns.

Impact on design decisions.

This feedback directly influenced the routing engine design by introducing time-aware weighting profiles (e.g., increasing lighting weight at night and adjusting priorities during rush periods). Seasonal and longer-term patterning was documented as a future enhancement, as it requires additional data and validation.

Finally,(P14). Omar's feedback strengthened SafePath's safety-first rationale by pushing the system toward context-aware scoring, ensuring the app behaves more like users expect in real travel conditions.

Target User Feedback — P15 (Liam, Dublin, TUD student)

Participant profile. P15 (Liam) is a Technological University Dublin student. We met him briefly at a Luas stop near the college and collected feedback via a short informal interview (no task session due to time/location constraints).

Topics discussed. Liam focused on what additional map information would increase perceived safety and decision confidence.

Key issue raised: missing POI-based safety context.
Liam suggested adding "safety-relevant POIs" and proximity cues to support decision-making, including:

- **safe spaces** (police stations, hospitals, well-lit public areas),
- explicit **high-risk areas** (crime hotspots),
- **CCTV coverage** (where reliable data exists),
- **emergency services proximity**.

Impact on design decisions.
This feedback was captured as a future enhancement category: adding optional **POI safety overlays** and "near safe spaces" decision support. Because POI coverage and CCTV datasets vary by city and may raise governance/privacy considerations, this was documented as a roadmap item to implement only where reliable datasets and policies support it.

Finally, (P15). Liam's feedback expanded SafePath's future direction beyond "avoid risk" toward "support reassurance," suggesting that proximity to safe spaces could improve user confidence in safety-first routing.

Expert Users -User Evaluation

P5 (Andrea, Mentor) - UI/UX Expert

Reviewer profile. P5 (Andrea) is the project mentor and a UX design expert. Her input was provided throughout early and mid-project stages as an expert review rather than a timed usability test session.

Review focus and approach. Andrea's feedback targeted core UX foundations: **mobile-first interaction**, first-impression value on entry, visual hierarchy, accessibility-oriented spacing and typography, and aligning design decisions with real user needs rather than developer assumptions.

Key findings and design changes implemented.

1. **Mobile-first direction (critical shift).**
   Early iterations were not designed primarily for mobile usage. Andrea emphasised treating mobile as the default view, increasing font sizes for readability, and improving spacing to support usability and accessibility. This influenced layout decisions across the application.

2. **Homepage purpose and first impression (from "landing page" to "functional start").**
   The initial homepage was a simple hero section with two buttons (e.g., report hazard / go safe). Andrea challenged this as low-value for a user's first interaction and recommended making the home screen immediately useful for primary tasks.
   Change implemented: the homepage was redesigned to include a **map**, a **route planning search bar**, and functional content for logged-in users (e.g., hazard cards and buddy availability cards).

3. **Visual identity and theme refinement.**
   The initial theme relied heavily on **green/amber** to represent safety and danger. Andrea recommended exploring a more modern routing-app aesthetic and ensuring the palette supports clarity rather than signalling only "danger."
   Change implemented: the theme was updated after further research to a **navy + cyan** palette, improving perceived modernity and visual cohesion.

4. **Transport-mode choice surfaced earlier.**
   Andrea advised that mode choice (walk/cycle) should be visible at the start of the journey, not hidden later in the flow.
   Change implemented: a walk/cycle control was added on the homepage, allowing users to begin route planning immediately and then transition to the Routes page.

Impact on SafePath. Andrea's expert review substantially improved SafePath's usability direction by shifting the product towards a **mobile-first, task-focused entry experience** and strengthening design consistency. Several additional suggestions were retained as future enhancements due to project time constraints.

Evidence type. This contribution is reported as an **expert review and iterative design critique**, not as a controlled participant task session, and therefore does not include task completion time or prompt-based metrics.

P6 (Damian, Mentor) - Technical / Research Expert

Reviewer profile. P6 (Damian) is a project mentor with expertise in software development and research design (academic/professional background). His input was provided as an expert review rather than a timed usability session.

Review focus. Damian's feedback focused on product framing and interaction clarity: feature naming and user expectations, consistency of visual language, accessibility-oriented layout decisions, and early product-definition choices (walk vs cycle vs both).

Key findings and changes implemented.

1. **Feature naming risk: "Find Mate" misframed the safety feature.**
   Damian identified that the original label "Find Mate" made the feature appear similar to a dating function, which conflicts with SafePath's safety/community intent and could reduce user trust.
   Change implemented: the feature was reframed and implemented as **FindBuddy**, emphasising community support and encouraging users not to walk/cycle alone.

2. **Consistency of visual language (colour semantics).**
   Damian recommended defining a consistent language for colours and states (e.g., clear mapping such as walk vs cycle, and stable route-type colours such as fastest/safest/balanced), so users can learn meanings quickly and avoid misinterpretation.
   Design implication: this feedback reinforced the need for a clear **legend**, consistent colour usage across screens, and avoidance of conflicting colour meanings.

3. **Brand and logo maturity.**
   He advised iterating on the logo beyond an early text-only/basic design to improve product identity and credibility.
   Design implication: additional logo iterations were explored to strengthen visual identity while staying aligned with the safety-first theme.

4. **Accessibility and readability improvements.**
   Damian consistently highlighted the need to increase font sizes and improve spacing (use of whitespace) to support readability and accessibility.
   Design implication: UI layouts were revised to include clearer spacing, larger typography on mobile, and reduced visual clutter.

5. **Product scope clarity: cycling vs walking vs both.**
   Damian pushed the team to decide the target mode(s) explicitly, since walking and cycling have different risk profiles and user needs. He recommended grounding this decision in user discussions and evaluation.
   Outcome: SafePath retained support for both walking and cycling, and FindBuddy was positioned as a safety/community feature applicable to both modes.

Impact on SafePath. Damian's review improved SafePath by reducing feature misinterpretation risk (naming), strengthening consistency in the UI's visual language, and reinforcing accessibility-friendly layout decisions. It also sharpened the project's framing: SafePath is not a "social matching" product; it is a safety-first routing system with a community support feature designed to reduce perceived risk when travelling alone.

Evidence type. This contribution is reported as an expert review (design/research/technical critique), therefore it is analysed as qualitative feedback rather than task-timing metrics.

P8 (Brendan, Mentor) - Technical / Architecture Expert

Reviewer profile. P8 (Brendan) is a technical mentor with strong expertise in databases and data architecture. His feedback was provided as an expert architecture/design review (not a timed task-based session).

Review focus. Brendan's review concentrated on architectural scalability, maintainability, real-time system design, safety-engine feasibility, and privacy/security controls for community features.

Key findings and changes implemented

1. **Caching decision: Redis not required for current needs**
   The team initially considered Redis caching for performance optimisation. Brendan advised that caching was not essential at this stage given the **moderate data volume** and the product goal of **real-time visibility and correctness**, not micro-optimised response times.
   Outcome: Redis caching was deprioritised, reducing operational complexity while keeping the system simpler to maintain.
2. **Real-time updates: move away from DB triggers to event-driven updates**
   Early iterations relied on database triggers to propagate hazard updates instantly. Brendan flagged this as creating tight coupling between data and logic layers and making behaviour harder to debug and evolve.
   Change implemented: the architecture shifted to an **event-driven real-time model** using **WebSockets (Socket.IO)** to publish hazard and buddy events from the application layer.
   Impact: improved maintainability, clearer system behaviour, easier debugging, and extensibility (e.g., filtered/room-based event channels).
3. **Suggested routes engine:** ML concept to rule-based safety engine
   The team explored a machine-learning approach for risk prediction, but Brendan reinforced concerns around labelled data availability**,** explainability, and added training/maintenance complexity.
   Change implemented: SafePath adopted a rule-based safety scoring engine that:
   - uses real crime datasets (UK open data),
   - computes route risk using weighted factors (e.g., crime, lighting, hazards),
   - generates alternatives via routing (e.g., OSRM),
   - supports user-defined safety preferences (profile weights).
4. **Authentication & onboarding: accounts enable personalisation and privacy control**
   Initially, the team planned to allow users to skip login to reduce friction. Brendan highlighted that user accounts create clear value by enabling:
   - **personalised safety weights** (e.g., crime vs lighting preferences),
   - explicit **opt-in location sharing controls** (important for FindBuddy),
   - stronger privacy/GDPR alignment through user-managed settings.
     Outcome: login/profile became a core part of the product maturity path.
5. **Map consistency: real-time hazards should appear across views**
   Brendan recommended showing real-time hazards not only where users report them, but consistently across relevant map views to strengthen trust and usefulness.
   Outcome: hazard visibility was treated as a cross-app map concern (not isolated to one screen).
6. **FindBuddy privacy: limit identifiable information for guests**
   Brendan raised privacy/security concerns about exposing buddy cards too openly.

Change implemented: FindBuddy details were restricted for non-logged-in users (e.g., guests see only an aggregate count of nearby buddies rather than identifiable cards/locations), while logged-in users access fuller features under explicit consent.

7. **Security governance: introduce admin monitoring capabilities**
   Brendan suggested an admin capability to monitor hazard submissions and buddy activity to reduce abuse and support moderation.
   Outcome: an admin panel / moderation workflow was captured as a security enhancement to support safe operation and discourage malicious behaviour.

Impact on SafePath. Brendan's review significantly influenced SafePath's technical direction: simplifying infrastructure choices (no premature Redis), improving real-time architecture (event-driven WebSockets), making the routing engine defensible and implementable (rule-based over ML), and strengthening privacy/security posture (FindBuddy visibility controls + moderation roadmap).

Evidence type. Expert technical/architecture review; reported as qualitative design decisions rather than task timing metrics.

Participant Overview

Below table display participants overview table (P1–P15), tagged by city and group, some entries are task sessions while others are expert reviews / interviews (no timing metrics).

ID	City	Group (G1/G2/G3)	Session type	Tasks attempted (T1–T5)	Notes
P1	Dublin	G3	Task session	T1	Controlled hazard used
P2	Dublin	G3	Task session	T3 + T1	"Fastest = Safest" confusion
P3	Dublin	G2	Task session	T1 + T5	Location-null bug found/fixed
P4	London	G2/G1	Task session (remote)	T1	Reported slowness
P5	—	G1	Expert UX review	N/A	Mobile-first + homepage redesign
P6	—	G1	Expert review	N/A	Naming + colour semantics + scope
P7	London	G3	Task session (remote)	T3	Reporting friction + page switching
P8	—	G1	Technical architecture review	N/A	WebSockets + privacy + governance
P9	Dublin	G3	Task session (opportunistic)	T1	Mobile reachability issue
P10	Dublin	G3	Task session	T5 + (Plan route in buddy)	FindBuddy→Route complexity
P11	Dublin	G3	Task session	T5	Presence/recency bug observed
P12	Dublin	G3	Interview feedback	N/A	Guest features recommendation
P13	Dublin	G3	Task session	T5	View Map bug + consent bypass bug
P14	Dublin	G2/G3	Interview feedback	N/A	Time-based factors requested
P15	Dublin	G3	Interview feedback	N/A	POI safety layer requested

Table 5 - Participant Overview

Task Results Summary

Task outcomes were recorded for each scenario task using a three-level scale (Complete / Pass / Fail), alongside completion time and the number of facilitator prompts required. Prompts were also labelled by type to distinguish simple navigation guidance (where to click) from concept clarification (e.g., why a route is considered safer). Not every participant attempted every task, because some sessions were shorter or opportunistic and some contributions were collected as interview-style feedback or expert reviews where timed task measurement was not applicable.

Task	Description	# Attempted	Complete	Pass	Fail	Observed time (range)	Common prompts / issues
T1	Plan safer route + start navigation	6 (P1, P2, P3, P4, P9, P10*)	3	2	0	<2 min → >15 min	Confusion when fastest = safest (needs explanation); route list reachability on small screens; safety-score controls not obvious
T2	Interpret overlays + legend	Not captured as a standalone task	—	—	—	—	Evidence appears indirectly (colour/legend meaning questions), but no consistent timing/outcome recorded per participant
T3	Report hazard (photo + pin + submit)	2 (P2, P7)	1	1	0	>3 min to submit + view card	Typing burden in required description; desire for templates/autocomplete; page switching friction
T4	Adjust safety preferences and observe route change	Not captured consistently	—	—	—	—	Some users adjusted scores (e.g., P1, P9), but not measured for all participants as a comparable standalone task
T5	FindBuddy (discover + request + group)	4 (P3, P10, P11, P13)	2	2	0	Not consistently timed	Too many steps; presence/recency ("nearby") trust issue; View Map button bug; consent workflow inconsistency ("Join Route")

Table 6 - Task Results Summary

**P10 attempted Plan Route inside FindBuddy, and reported it was difficult; this is counted under T1 as a* route-planning attempt within the buddy context.

Factor Ratings Summary

After each task-based session, participants were asked to rate the same six factors: UI, Functionality, Usability, Colour scheme, Performance, and Timing, using a simple Positive / Neutral / Negative scale. These factor ratings support cross-participant comparison by summarising overall sentiment beyond individual comments. Where a participant did not comment on a factor (or the session format did not include factor rating), it is recorded as N/A.

Factor ratings (task-based participants only, n = 9)

Factor	Positive	Neutral	Negative	N/A (not rated)
UI	2	4	3	0
Functionality	4	2	3	0
Usability	0	3	6	0
Colour scheme	3	2	0	4
Performance	0	5	1	3
Timing	1	3	2	3

Table 7 - Factor Rating Summary

Issues Log: Severity and Fix Status

Issue	Component	Severity	Evidence (P#	Status	Planned fix / implemented change
Route list not reachable on small screens	UI / T1	High	P9	Fixed / In progress	Bottom sheet + scroll + spacing
"View Map" button not working	FindBuddy	High	P13	In progress	Repair route + state handling
Auto-enrolment bypasses consent	FindBuddy	High	P13	In progress	Require explicit "Join Route"
Stale "nearby" visibility (presence bug)	FindBuddy	High	P11	Fixed	Recency filter + session cleanup
Location null causes invalid distance	FindBuddy	Critical	P3	Fixed	Capture location in route/buddy flows
"Fastest = safest" not explained	Route UX	Medium	P2, P9	Planned	Add "no high risk detected" message
Reporting requires long text	Reporting	Medium	P7	Planned	Templates/autofill
Forgot password missing	Auth	Medium	P4	Fixed	Add password reset
Time-based weights missing (initially)	Routing engin	Medium	P14	Fixed/Implemented	Time-aware weighting profile

Table 8 - Evaluation Issues Log

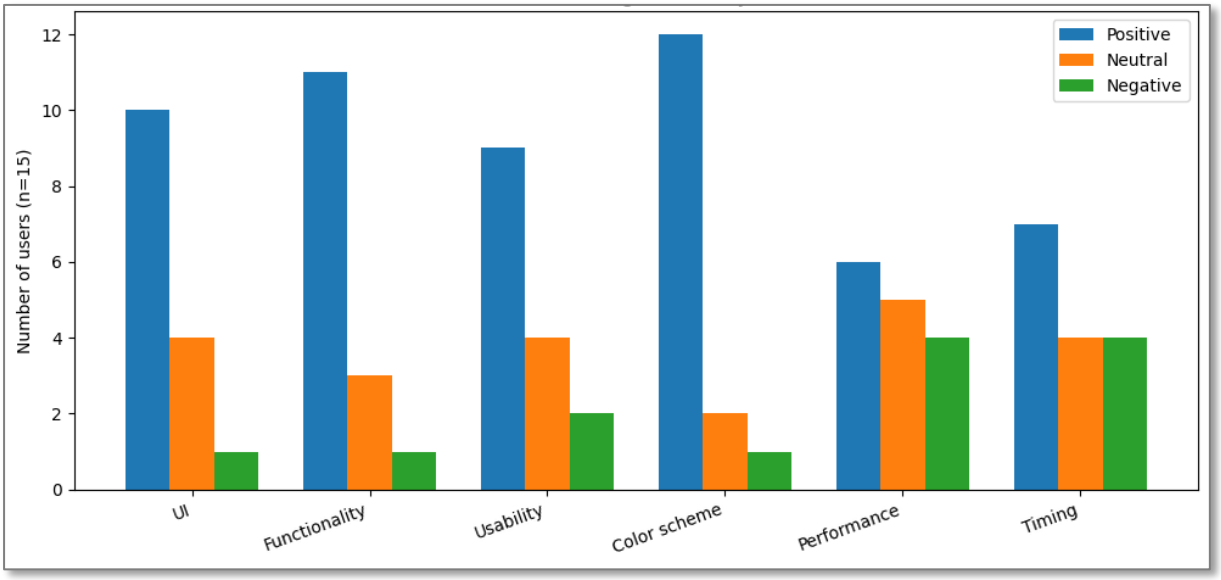

Figure 72 - Evaluation Summary Graph

Evaluation Conclusion

Overall, the factor ratings show that SafePath's visual design and core functionality are strong**,** while performance and time-efficiency are the main weaknesses. The most positive factor was Colour scheme (12/15 positive), followed by Functionality (11/15 positive) and **UI** (10/15 positive), suggesting the product is generally perceived as modern, understandable, and capable of delivering its core features. Usability is also largely positive (9/15), but the presence of negative ratings (2/15) indicates that some flows still require simplification and clearer guidance for first-time or non-technical users. In contrast, Performance and Timing have the highest negative counts (4/15 each), which aligns with user feedback about perceived slowness and multi-step interactions, especially on mobile. These results indicate that the next iteration should prioritise mobile flow efficiency (reduce steps, improve reachability) and responsiveness/latency improvements, while keeping the current visual identity and feature set as a strong foundation.

Conclusion: Review of the Project

What was our project management strategy?

We managed SafePath using an Agile methodology with two-week sprints throughout the entire development cycle. We used Jira for task tracking and GitHub for version control, which kept everyone aligned on progress and priorities. Regular stand-up meetings three times per week helped us identify blockers early and maintain momentum.

We divided responsibilities based on individual strengths: some team members focused on frontend development with Next.js and React, others handled backend architecture with Node.js and Express, and a dedicated subgroup managed the complex data processing pipeline including crime data ETL, PostGIS spatial queries, and external API integrations with OSRM and TomTom.

Throughout the project, we maintained three separate environments - development, staging, and production - which allowed thorough testing before releasing features.

Code reviews through pull requests became standard practice, ensuring quality and knowledge sharing across the team. This collaborative approach proved essential when integrating the various complex components like real-time hazard reporting, safety scoring algorithms, and the FindBuddy feature.

User research and feedback drove our development decisions. We conducted surveys with 55 respondents, created detailed personas based on real user needs, and ran multiple rounds of usability testing. This iterative feedback loop fundamentally shaped our design choices, leading to major interface redesigns and feature prioritization adjustments midway through the project.

What were the biggest challenges we faced and how did we address them?

The most significant technical challenge was integrating heterogeneous data sources into a unified safety routing system. Crime data from the UK Police API, real-time incidents from TomTom, lighting information from OpenStreetMap via Overpass, and user-generated hazard reports all arrived in different formats with varying spatial precision and update frequencies. Geographic misalignment was a constant issue - the same physical location could be represented differently across datasets.

We solved this by designing a grid-based spatial architecture. We divided the coverage area into approximately 1km grid cells and pre-processed all data sources to aggregate safety indicators at the cell level. Crime data underwent a Python ETL pipeline that cleaned, normalized, and applied time-decay weighting so recent incidents influenced scoring more heavily. This grid system transformed expensive real-time spatial queries into fast memory lookups, reducing route calculation time from several seconds to under one second.

Performance optimization presented another major hurdle. Our initial safety scoring implementation was too slow for interactive use - users expect route results within 2-3 seconds, but our algorithm was taking 10-15 seconds. The bottleneck was checking individual crime points and hazards for every sample point along multiple route options.

We addressed this through multiple optimizations: implementing PostGIS GiST spatial indexes for fast proximity queries, caching Overpass lighting data with a 30-day TTL to avoid repeated slow API calls, storing the crime grid in server memory for instant access, and

applying short-term caching (15 minutes) for TomTom traffic incidents. These combined changes brought route calculation time down to acceptable interactive levels.

User interface design proved more challenging than anticipated. Our early prototypes, which looked polished in Figma mock-ups, failed usability testing. Users found them cluttered, confusing, and difficult to navigate on mobile devices. The biggest complaint was that too many features competed for attention, making it hard to accomplish the primary task: finding a safe route.

We completely redesigned the interface to be mobile-first and map-centric. We removed the busy initial landing page and made the map the central element on every screen. We switched from a light colour scheme to a darker navy and green palette, which improved contrast against map tiles and enhanced outdoor readability. Route options were reorganized into clear side panels with progressive disclosure, showing essential information first and detailed breakdowns on demand. This redesign dramatically improved usability scores in subsequent testing.

Data quality and availability limitations also challenged us. Crime data was only available for London with monthly update cycles, which limited our coverage and freshness. STATS19 collision data integration, originally planned, proved too complex for the MVP timeline. Overpass API rate limits forced us to cache lighting data rather than fetch it live. We learned to be transparent about these limitations rather than hiding them, clearly communicating to users when data might be outdated or incomplete.

Team coordination across different time zones and schedules created workflow friction. Dependencies between frontend and backend work sometimes caused delays. We improved this by defining clear API contracts early, using mock data to allow parallel development, and maintaining detailed documentation so team members could pick up each other's work when needed.

One significant feature we were unable to implement due to time constraints was an admin dashboard for system moderation and safety monitoring. Following mentor feedback in our final sessions, we identified two critical administrative needs that remain unaddressed. First, the current hazard reporting system allows any user to report hazards that immediately appear to all other users. While this creates a responsive, community-driven safety map, it introduces risks of false reports, spam, or malicious content. Second, the FindBuddy matching feature connects users for joint travel but lacks any audit trail - if an incident occurred during a matched journey, we would have no record of who was connected with whom, when the match was made, or what route was shared. The planned admin dashboard would address both concerns: moderators could review and validate hazard reports before public display, with features for bulk moderation, user reputation tracking, automated suspicious pattern detection, and comprehensive audit logs. Simultaneously, it would maintain detailed records of all FindBuddy requests and matches, including timestamps, user profiles, shared routes, and match outcomes, providing crucial data for investigating any safety incidents while respecting user privacy through appropriate access controls and retention policies. Without these administrative capabilities, the system currently relies on community self-regulation for hazard quality and has no formal safety accountability mechanism for the buddy matching feature. Implementing this dashboard represents a critical requirement for production deployment, particularly given the safety-focused nature of the application and potential liability considerations.

What is the key contribution of our project and what future extensions do we envision?

SafePath's primary contribution is demonstrating that safety can be quantified and integrated directly into routing decisions rather than treated as an afterthought. We proved that heterogeneous risk signals - crime statistics, lighting conditions, infrastructure quality, real-time incidents, and community reports - can be unified into an actionable framework that produces understandable, trustworthy route recommendations.

Unlike existing navigation apps that optimize purely for time and distance, SafePath puts user safety first. The system doesn't just show a safety score - it explains why one route is safer: "This route has better street lighting and avoids an area with recent incidents." This transparency builds trust and helps users make informed decisions based on their personal risk tolerance.

The real-time community hazard reporting system with WebSocket-based instant notifications represents another significant innovation. When someone reports a pothole, broken streetlight, or dangerous junction, nearby users receive immediate alerts rather than discovering the hazard themselves. This creates a living, continuously updated safety map that improves as more people participate.

The FindBuddy feature addresses an often-overlooked aspect of safety: the psychological and practical benefits of not traveling alone. By matching people heading in similar directions, SafePath combines route safety with social safety, which user research showed was highly valued, especially for evening and night travel.

For future extensions, we envision several key improvements:

- **Geographic expansion:** Extending coverage beyond London to other UK cities and eventually internationally by automating the data ETL pipeline and making it configurable for different regional data sources.

- **Machine learning integration:** Building predictive models that learn patterns from historical data to forecast where and when risks are likely to emerge. For example, identifying junctions that become dangerous during rush hour or areas where lighting failures frequently occur.

- **Enhanced data sources:** Integrating air quality, weather impacts, noise levels, and detailed accessibility features (curb cuts, ramp quality, surface conditions) to provide holistic route recommendations that consider multiple aspects of journey quality.

- **City partnership programs:** Working with local councils and transport authorities to share aggregated, anonymized safety data that can inform infrastructure investment decisions and policy making. If data shows consistent hazard reports at specific locations, this becomes actionable intelligence for urban planning.

- **Advanced verification mechanisms:** Developing trust and reputation systems for community reports, where verified users or multiply-confirmed hazards carry more weight in safety scoring, reducing the impact of false or spam reports.

- **Multimodal routing:** Adding public transport integration and mixed-mode journeys (e.g., walk to station, take bus, cycle final leg) to provide comprehensive door-to-door guidance.

What lessons have we learned by developing this system?

The most important lesson was that working with spatial data requires specialized knowledge and tools. Geographic calculations, coordinate systems, and spatial queries are fundamentally different from regular database operations. We learned to rely on established libraries like PostGIS rather than attempting custom implementations, and to always validate spatial calculations thoroughly because small errors propagate into completely wrong results.

Real-time performance requirements shaped every architectural decision. Users expect instant responses from navigation apps, which forced us to pre-compute what we could, cache aggressively, and optimize database queries relentlessly. We learned that sometimes a slightly less accurate but much faster approach delivers better user experience than perfect but slow calculations.

User feedback fundamentally redirected our development. Our initial designs, created in isolation, looked professional but failed real-world usability testing. We learned that designing for actual usage contexts - people walking outdoors, using phones with one hand, in varying light conditions - requires testing with real users in realistic situations, not just reviewing mock-ups in meetings.

Data quality is never perfect in real-world systems. Missing values, errors, inconsistencies, and outdated information are inevitable. We learned to build graceful degradation into every component - the system should provide useful results even when some data sources are unavailable or incomplete, and it should communicate uncertainty honestly rather than presenting false confidence.

Building a safety-focused application carries special ethical responsibility. People might make important decisions based on our recommendations, so accuracy and transparency are not optional features -- they are fundamental requirements. We learned to clearly communicate limitations, avoid overpromising, and remind users that no app can guarantee safety, it can only provide information to support informed decision-making.

Finally, our perspective shifted from viewing this as primarily a technical challenge to understanding it as a human-centred design problem. The hardest parts weren't implementing algorithms or integrating APIs - they were understanding what users actually needed, designing interfaces that communicated complex information simply and clearly, and building systems that remained trustworthy under real-world conditions. Technical excellence is necessary but not sufficient, understanding people and their contexts is equally critical to building useful systems

Reference

[1] AbilityNet. (2024, December 12). Web Content Accessibility Guidelines (WCAG) 2.2. https://www.w3.org/TR/WCAG22/

[2] Backstrom, L., Sun, E., & Marlow, C. (2010). Find me if you can: Improving geographical prediction with social and spatial proximity. Proceedings of the 19th International Conference on World Wide Web, 61–70. https://doi.org/10.1145/1772690.1772698

[3] bSafe. (n.d.). How bSafe works. https://www.getbsafe.com/how-it-works

[4] Bhowmick, D., Winter, S., Stevenson, M., & Vortisch, P. (2021). Investigating the practical viability of walk-sharing in improving pedestrian safety. Computational Urban Science. https://doi.org/10.1007/s43762-021-00020-z

[5] Bradner, S. (1997). Key words for use in RFCs to indicate requirement levels (RFC 2119). https://doi.org/10.17487/RFC2119

[6] Branion-Calles, M., Nelson, T., & Winters, M. (2017). Comparing crowdsourced near-miss and collision cycling data and official bike safety reporting. Transportation Research Record, 2662(1), 1–11. https://doi.org/10.3141/2662-01

[7] Castells-Graells, D., Salahub, C., & Pournaras, E. (2020). On cycling risk and discomfort: Urban safety mapping and bike route recommendations. Computing, 102, 1259–1274. https://doi.org/10.1007/s00607-019-00771-y

[8] Chen, H., Wu, Y., Wang, W., Zheng, Z., Ma, J., & Zhou, B. (2024). Optimizing patrolling route with a risk-aware reinforcement learning model. https://doi.org/10.2139/ssrn.4752931

[9] data.police.uk. (n.d.). About / open data. https://data.police.uk/

[10] Delling, D., Goldberg, A. V., Pajor, T., & Werneck, R. F. (2017). Customizable route planning in road networks. Transportation Science, 51, 566–591. https://doi.org/10.1287/trsc.2014.0579

[11] Department for Transport. (2025, September 25). Reported road casualties Great Britain, annual report: 2024. https://www.gov.uk/government/statistics/reported-road-casualties-great-britain-annual-report-2024

[12] Diaz, N. (2024). Waze details new safety alerts and navigational tips arriving “this month”. Yahoo Tech. https://tech.yahoo.com/transportation/articles/waze-details-safety-alerts-navigational-201013065.html

[13] Foster, S., Giles-Corti, B., & Knuiman, M. (2014). Does fear of crime discourage walkers? A social-ecological exploration of fear as a deterrent to walking. Environment and Behavior, 46, 698–717. https://doi.org/10.1177/0013916512465176

[14] GOV.UK. (2017, October 3). Using moderated usability testing. https://www.gov.uk/service-manual/user-research/using-moderated-usability-testing

[15] Google Developers. (n.d.). How to use Waze deep links. Retrieved December 17, 2025, from https://developers.google.com/waze/deeplinks

[16] Google Developers. (n.d.). Waze for Developers: Developer tools. Retrieved December 17, 2025, from https://developers.google.com/waze

[17] ISO. (2018). ISO 9241-11:2018 Ergonomics of human-system interaction — Part 11: Usability: Definitions and concepts. International Organization for Standardization.

[18] Kiyohara, M., & Mondri, E. (2025, April 28). Coolight: Enhancing nighttime safety for urban student commuters. CHI Conference on Human Factors in Computing Systems (CHI '25). https://arxiv.org/html/2503.20888v1

[19] Lazar, J., Feng, J. H., & Hochheiser, H. (2017). Research methods in human-computer interaction. Morgan Kaufmann. https://www.sciencedirect.com/book/9780128053904/research-methods-in-human-computer-interaction

[20] Levy, S., Xiong, W., Belding, E., & Wang, W. Y. (2020). SafeRoute: Learning to navigate streets safely in an urban environment. ACM Transactions on Intelligent Systems and Technology, 11(6), 66:1–66:17. https://doi.org/10.1145/3402818

[21] Li, Q., Wang, Z., Kolla, R. D. T. N., Li, M., Yang, R., Lin, P. S., & Li, X. (2023). Modeling effects of roadway lighting photometric criteria on nighttime pedestrian crashes on roadway segments. Journal of Safety Research, 86, 253–261. https://doi.org/10.1016/j.jsr.2023.07.004

[22] Life360. (2025). Life360 | Location sharing & safety app. https://www.life360.com/en-ie

[23] Luxen, D., & Vetter, C. (2011). OSRM-backend. Proceedings of the ACM International Symposium on Advances in Geographic Information Systems, 513–516. https://doi.org/10.1145/2093973.2094062

[24] Moran, K. (2019, December 1). Usability (user) testing 101. Nielsen Norman Group. https://www.nngroup.com/articles/usability-testing-101/

[25] Myhrmann, M. S., & Mabit, S. E. (2023). Assessing bicycle crash risks controlling for detailed exposure: A Copenhagen case study. Accident Analysis & Prevention, 192, 107226. https://doi.org/10.1016/j.aap.2023.107226

[26] Nelson, T. A., Denouden, T., Jestico, B., Laberee, K., & Winters, M. (2015). BikeMaps.org: A global tool for collision and near miss mapping. Frontiers in Public Health, 3. https://doi.org/10.3389/fpubh.2015.00053

[27] Nielsen, J. (1993). Usability engineering. Morgan Kaufmann. (Excerpt and related guidance: https://www.nngroup.com/articles/response-times-3-important-limits/)

[28] Nielsen, J. (2001, January 20). Usability metrics. Nielsen Norman Group. https://www.nngroup.com/articles/usability-metrics/

[29] Nielsen, J. (2012, October 7). User satisfaction vs. performance metrics. Nielsen Norman Group. https://www.nngroup.com/articles/satisfaction-vs-performance-metrics/

[30] Noonlight. (2025). Personal safety—Powered by Noonlight. https://www.noonlight.com/solutions/personal-safety

[31] Open Source Geospatial Foundation / PostGIS Project. (n.d.). PostGIS geography types (DB management). Retrieved December 16, 2025, from https://postgis.net/docs/using_postgis_dbmanagement.html#PostGIS_Geography

[32] PostGIS Project. (n.d.). ST_Distance. Retrieved December 16, 2025, from https://postgis.net/docs/ST_Distance.html

[33] PostGIS Project. (n.d.). ST_DWithin. Retrieved December 16, 2025, from https://postgis.net/docs/ST_DWithin.html

[34] PostGIS Project. (n.d.). Spatial indexing. Retrieved December 16, 2025, from https://postgis.net/docs/using_postgis_dbmanagement.html

[35] Pazdan, S. (2020). The impact of weather on bicycle risk exposure. Archives of Transport, 56(4), 89–105. https://doi.org/10.5604/01.3001.0014.5629

[36] Project OSRM. (n.d.). OSRM documentation. Retrieved December 17, 2025, from https://project-osrm.org/docs/

[37] React Team. (n.d.). Quick start. Retrieved December 17, 2025, from https://react.dev/learn

[38] React Leaflet. (n.d.). Introduction. Retrieved December 17, 2025, from https://react-leaflet.js.org/docs/start-introduction/

[39] Redis. (n.d.). Documentation. Retrieved December 17, 2025, from https://redis.io/docs/latest/

[40] Socket.IO. (2025). Using multiple nodes. https://socket.io/docs/v4/using-multiple-nodes/

[41] Sohrabi, S., Weng, Y., Das, S., & German Paal, S. (2022). Safe route-finding: A review of literature and future directions. Accident Analysis & Prevention, 177, 106816. https://doi.org/10.1016/j.aap.2022.106816

[42] Solé, L., Sammarco, M., Detyniecki, M., & Miguel, M. E. (2022). Towards drivers' safety with multi-criteria car navigation systems. Future Generation Computer Systems, 135, 1–9. https://doi.org/10.1016/j.future.2022.04.019

[43] Stanford Crypto. (n.d.). Location privacy. Retrieved December 16, 2025, from https://crypto.stanford.edu/locpriv/

[44] Teschke, K., Harris, M. A., Reynolds, C. C. O., Winters, M., Babul, S., Chipman, M., Cusimano, M. D., Brubacher, J. R., Hunte, G., Friedman, S. M., Monro, M., Shen, H., Vernich, L., & Cripton, P. A. (2012). Route infrastructure and the risk of injuries to bicyclists: A case-crossover study. American Journal of Public Health, 102(12), 2336–2343. https://doi.org/10.2105/AJPH.2012.300762

[45] TomTom. (2024, August 8). Traffic incidents service | Traffic API. https://developer.tomtom.com/traffic-api/documentation/tomtom-maps/traffic-incidents/traffic-incidents-service

[46] Vidal-Tortosa, E., & Lovelace, R. (2024). Road lighting and cycling: A review of the academic literature and policy guidelines. Journal of Cycling and Micromobility Research, 2, 100008. https://doi.org/10.1016/j.jcmr.2023.100008

[47] Wang, G., Wang, B., Wang, T., Nika, A., Zheng, H., & Zhao, B. Y. (2016). Waze. MobiSys 2016 Companion, 146. https://doi.org/10.1145/2938559.2938610

[48] Waze Help. (n.d.). Spot emergency vehicles in your area. Retrieved December 17, 2025, from https://support.google.com/waze/answer/14716475?hl=en

[49] W3C. (2024, December 12). Web Content Accessibility Guidelines (WCAG) 2.2. https://www.w3.org/TR/WCAG22/

[50] Zhang, M., & Bandara, A. K. (2024). Understanding pedestrians' perception of safety and safe mobility practices. Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems, 1–17. https://doi.org/10.1145/3613904.3642896

[51] Google Workspace Admin Help. (2025). Send email from a printer, scanner, or app. [Add URL]

[52] Introduction to PostGIS. (2023).

[53] Kim, S., Joo, Y., & Park, S. (2011). Pedestrian Path Findings using Multi-Factors Affected Walking.

[54] Next.js. (n.d.). Next.js docs.

[55] OSRM API documentation: General options

[56] Upload. (2025).

[57] What is usability evaluation. (2016, June 6).

[58] Yu, J. (2022). 4 reasons why I start using Tailwind CSS in every project.